

Groupe de travail sur l'ingénierie Internet (IETF) R. Fielding, éd.

Demande de commentaires : 7231 Adobe

Obsolètes : [2616](#)

J. Reschke, éd.

Mises à jour : [2 817](#)

greenbytes

Catégorie : Parcours de normalisation juin 2014

ISSN : 2070-1721

Protocole de transfert hypertexte (HTTP/1.1) : sémantique et contenu

Abstrait

Le protocole de transfert hypertexte (HTTP) est une application sans état. protocole de niveau pour les informations distribuées, collaboratives et hypertextes systèmes. Ce document définit la sémantique des messages HTTP/1.1, comme exprimé par les méthodes de requête, les champs d'en-tête de requête, la réponse codes d'état et champs d'en-tête de réponse, ainsi que la charge utile de messages (métadonnées et contenu du corps) et mécanismes de contenu négociation.

Statut de ce mémo

Il s'agit d'un document de suivi des normes Internet.

Ce document est un produit de l'Internet Engineering Task Force (IETF). Il représente le consensus de la communauté IETF. Il a reçu un examen public et a été approuvé pour publication par le Groupe de pilotage de l'ingénierie Internet (IESG). Plus d'informations sur Les normes Internet sont disponibles dans [la section 2 de la RFC 5741](#) .

Informations sur l'état actuel de ce document, les éventuels errata, et comment fournir des commentaires à ce sujet peuvent être obtenus à l'adresse <http://www.rfc-editor.org/info/rfc7231> .

[RFC 7231](#)

HTTP/1.1 Sémantique et contenu Juin 2014

Avis de droit d'auteur

Copyright (c) 2014 IETF Trust et les personnes identifiées comme Auteurs du document. Tous droits réservés.

Ce document est soumis au [BCP 78](#) et aux conditions juridiques de l'IETF Trust.

Dispositions relatives aux documents de l'IETF (<http://trustee.ietf.org/license-info>) en vigueur à la date de publication de ce document. Veuillez consulter ces documents soigneusement, car ils décrivent vos droits et restrictions en ce qui concerne à ce document. Les composants de code extraits de ce document doivent inclure le texte de la licence BSD simplifiée tel que décrit dans la section 4.e de les dispositions légales de la fiducie et sont fournies sans garantie décrit dans la licence BSD simplifiée.

Ce document peut contenir des éléments provenant de documents de l'IETF ou de l'IETF

Contributions publiées ou rendues publiques avant novembre 10, 2008. La ou les personnes contrôlant les droits d'auteur sur certains de ces

le matériel n'a peut-être pas accordé à l'IETF Trust le droit d'autoriser modifications de ce matériel en dehors du processus de normalisation de l'IETF.

Sans obtenir une licence adéquate de la part de la ou des personnes contrôlant

le droit d'auteur sur ces matériaux, ce document ne peut pas être modifié en dehors du processus de normalisation de l'IETF, et des œuvres dérivées de celui-ci peuvent

ne pas être créé en dehors du processus de normalisation de l'IETF, sauf pour formater

pour publication sous forme de RFC ou pour le traduire dans d'autres langues que l'anglais.

Piste des normes Fielding & Reschke [Page 2]

[RFC 7231](#)

HTTP/1.1 Sémantique et contenu Juin 2014

Table des matières

1.	Introduction	6
1.1	. Conformité et gestion des erreurs	6
1.2	. Notation syntaxique	6
2.	Ressources	7
3.	Représentations	7
3.1	. Métadonnées de représentation	8
3.1.1	. Traitement des données de représentation	8
3.1.2	. Codage pour la compression ou l'intégrité	11
3.1.3	. Langue du public	13
3.1.4	. Identification	14
3.2	. Données de représentation	17
3.3	. Sémantique de la charge utile	17
3.4	. Négociation de contenu	18
3.4.1	. Négociation proactive	19
3.4.2	. Négociation réactive	20
4.	Méthodes de requête	21
4.1	. Présentation	21
4.2	. Propriétés communes des méthodes	22
4.2.1	. Méthodes sûres	22
4.2.2	. Méthodes idempotentes	23
4.2.3	. Méthodes pouvant être mises en cache	24
4.3	. Définitions de méthodes	24
4.3.1	. GET	24
4.3.2	. HEAD	25
4.3.3	. POST	25
4.3.4	. PUT	26
4.3.5	. DELETE	29
4.3.6	. CONNECT	30
4.3.7	. OPTIONS	31
4.3.8	. TRACE	32
5.	Champs d'en-tête de requête	33
5.1	. Contrôles	33
5.1.1	. Expect	34
5.1.2	. Max-Forwards	36
5.2	. Conditionnels	36
5.3	. Négociation de contenu	37
5.3.1	. Valeurs de qualité	37
5.3.2	. Accept	38
5.3.3	. Accept-Charset	40
5.3.4	. Accept-Encoding	41
5.3.5	. Accept-Language	42
5.4	. Informations d'authentification	44
5.5	. Contexte de la requête	44
5.5.1	. De	44
5.5.2	. Référent	45

46	5.5.3 . Agent utilisateur
--------------------	---

Piste des normes Fielding & Reschke [Page 3]

[RFC 7231](#)

HTTP/1.1 Sémantique et contenu Juin 2014

6.	Codes d'état de réponse	
47		
6.1	. Présentation des codes d'état	48
6.2	. Informationnel 1xx	
		50
6.2.1	. 100 Continuer	50
6.2.2	. 101 Changement de protocoles	
		50
6.3	. Réussi 2xx	51
6.3.1	. 200 OK	51
6.3.2	. 201 Créé	
52		
6.3.3	. 202 Accepté	
52		
6.3.4	. 203 Informations non autorisées	52
6.3.5	. 204 Aucun contenu	53
6.3.6	. 205 Réinitialiser le contenu	
		53
6.4	. Redirection 3xx	54
6.4.1	. 300 Choix multiples	55
6.4.2	. 301 Déplacé définitivement	56
6.4.3	. 302 Trouvé	56
6.4.4	. 303 Voir autre	57
6.4.5	. 305 Utiliser un proxy	58
6.4.6	. 306 (Inutilisé)	58
6.4.7	. 307 Redirection temporaire	58
6.5	. Erreur client 4xx	58
6.5.1	. 400 Requête incorrecte	58
6.5.2	. 402 Paiement requis	59
6.5.3	. 403 Interdit	59
6.5.4	. 404 Introuvable	59
6.5.5	. 405 Méthode non autorisée	
59		
6.5.6	. 406 Non acceptable	60
6.5.7	. 408 Délai d'expiration de la requête	60
6.5.8	. 409 Conflit	60
6.5.9	. 410 Disparu	60
6.5.10	. 411 Longueur requise	61
6.5.11	. 413 Charge utile trop grande	
		61
6.5.12	. 414 URI trop long	61
6.5.13	. 415 Type de média non pris en charge	62
6.5.14	. 417 Échec de l'attente	62
6.5.15	. 426 Mise à niveau requise	
62		
6.6	. Erreur de serveur 5xx	62
6.6.1	. 500 Erreur interne du serveur	
		63
6.6.2	. 501 Non implémenté	63
6.6.3	. 502 Passerelle incorrecte	
		63
6.6.4	. 503 Service indisponible	63
6.6.5	. 504 Délai d'expiration de la passerelle	

.....	63	
6.6.6	. 505 Version HTTP non prise en charge	
64		
7	. Champs d'en-tête de réponse	64
7.1	. Données de contrôle	64
ed 7.1.1.	Date d'origine	65
7.1.2	. Emplacement	68
7.1.3	. Réessayer après	69

Piste des normes Fielding & Reschke [Page 4]

[RFC 7231](#)

HTTP/1.1 Sémantique et contenu Juin 2014

7.1.4	. Vary	70
7.2	. Champs d'en-tête du validateur	71
7.3	. Défis d'authentification	72
7.4	. Contexte de réponse	72
7.4.1	. Allow	72
7.4.2	. Server	73
8	. Considérations relatives à l'IANA	73
8.1	. Registre des méthodes	73
8.1.1	. Procédure	74
8.1.2	. Considérations relatives aux nouvelles méthodes	74
8.1.3	. Enregistrements	75
8.2	. Registre des codes d'état	75
8.2.1	. Procédure	75
8.2.2	. Considérations relatives aux nouveaux codes d'état	76
8.2.3	. Enregistrements	76
8.3	. Registre des champs d'en-tête	77
8.3.1	. Considérations relatives aux nouveaux champs d'en-tête ..	78
8.3.2	. Enregistrements	80
8.4	. Registre de codage de contenu	81
8.4.1	. Procédure	81
8.4.2	. Enregistrements	81
9	. Considérations relatives à la sécurité	81
9.1	. Attaques basées sur les noms de fichiers et de chemins d'accès	82
9.2	. Attaques basées sur l'injection de commandes, de codes ou de requêtes ..	82
9.3	. Divulgence d'informations personnelles	83
9.4	. Divulgence d'informations sensibles dans les URI	83
9.5	. Divulgence de fragments après redirections	84
9.6	. Divulgence des informations sur les produits	84
9.7	. Empreinte digitale du navigateur	84
10	. Remerciements	85
11	. Références	85
11.1	. Références normatives	85
11.2	. Références informatives	86
Annexe A	. Différences entre HTTP et MIME	89
A.1	. Version MIME	89
A.2	. Conversion en forme canonique	89
A.3	. Conversion des formats de date	90
A.4	. Conversion du codage de contenu	90
A.5	. Conversion du codage de transfert de contenu	90
A.6	. MHTML et limitations de longueur de ligne	90
Annexe B	. Modifications par rapport à la RFC 2616	91
Annexe C	. ABNF importé	93
Annexe D	. Recueil ABNF	94
Index		97

Piste des normes Fielding & Reschke [Page 5]

1. Introduction

Chaque message HTTP (Hypertext Transfer Protocol) est soit une requête ou une réponse. Un serveur écoute une requête sur une connexion, analyse chaque message reçu, interprète la sémantique du message dans par rapport à la cible de la demande identifiée et y répond requête avec un ou plusieurs messages de réponse. Un client construit demande des messages pour communiquer des intentions spécifiques, examine reçu des réponses pour voir si les intentions ont été réalisées, et détermine comment interpréter les résultats. Ce document définit Sémantique des requêtes et réponses HTTP/1.1 en termes d'architecture défini dans [[RFC7230](#)].

HTTP fournit une interface uniforme pour interagir avec une ressource ([Section 2](#)), quel que soit son type, sa nature ou sa mise en œuvre, via la manipulation et le transfert des représentations ([Section 3](#)).

La sémantique HTTP inclut les intentions définies par chaque méthode de requête ([Section 4](#)), extensions à ces sémantiques qui pourraient être décrites dans champs d'en-tête de demande ([section 5](#)), la signification des codes d'état à indiquer une réponse lisible par machine ([Section 6](#)), et la signification de autres données de contrôle et métadonnées de ressources qui pourraient être fournies dans champs d'en-tête de réponse ([Section 7](#)).

Ce document définit également les métadonnées de représentation qui décrivent comment une charge utile est destinée à être interprétée par un destinataire, la requête champs d'en-tête susceptibles d'influencer la sélection du contenu, ainsi que les différents algorithmes de sélection collectivement appelés « contenu » négociation" ([Section 3.4](#)).

1.1 . Conformité et gestion des erreurs

Les mots clés « DOIT », « NE DOIT PAS », « OBLIGATOIRE », « DEVRA », « NE DEVRA PAS », « DEVRAIT », « NE DEVRAIT PAS », « RECOMMANDÉ », « PEUT » et « FACULTATIF » dans ce Les documents doivent être interprétés comme décrit dans [[RFC2119](#)].

Les critères de conformité et les considérations concernant la gestion des erreurs sont défini dans [la section 2.5 de \[RFC7230\]](#) .

1.2 . Notation syntaxique

Cette spécification utilise la forme Backus-Naur augmentée (ABNF)

notation de [[RFC5234](#)] avec une extension de liste, définie dans [la section 7 de \[RFC7230\]](#), qui permet une définition compacte des valeurs séparées par des virgules
listes utilisant un opérateur « # » (similaire à la façon dont l'opérateur « * » indique répétition). [L'annexe C](#) décrit les règles importées d'autres documents. [L'annexe D](#) présente la grammaire collectée avec toute la liste opérateurs étendus à la notation ABNF standard.

Piste des normes Fielding & Reschke [Page 6]

[RFC 7231](#)

HTTP/1.1 Sémantique et contenu Juin 2014

Cette spécification utilise les termes « caractère », « codage de caractères »
schéma", "jeu de caractères" et "élément de protocole" tels qu'ils sont
définis dans
[[RFC6365](#)].

2. Ressources

La cible d'une requête HTTP est appelée « ressource ». HTTP ne
limiter la nature d'une ressource ; elle définit simplement une interface qui
peuvent être utilisés pour interagir avec les ressources. Chaque ressource
est
identifié par un identifiant de ressource uniforme (URI), comme décrit dans
[Section 2.7 de \[RFC7230\]](#) .

Lorsqu'un client construit un message de requête HTTP/1.1, il envoie le
URI cible sous l'une des différentes formes, telles que définies dans ([Section 5.3 de \[RFC7230\]](#)). Lorsqu'une requête est reçue, le serveur reconstruit une
URI de demande effective pour la ressource cible ([Section 5.5 de \[RFC7230\]](#)).

L'un des objectifs de conception de HTTP est de séparer l'identification des
ressources de
sémantique de la requête, rendue possible par l'investissement de la requête
sémantique dans la méthode de requête ([section 4](#)) et quelques
Champs d'en-tête de modification de requête ([section 5](#)). En cas de conflit
entre la sémantique de la méthode et toute sémantique impliquée par l'URI
elle-même, comme décrit dans [la section 4.2.1](#) , la sémantique de la méthode
prend
priorité.

3. Représentations

Considérant qu'une ressource peut être n'importe quoi, et que l'uniforme
L'interface fournie par HTTP est similaire à une fenêtre à travers laquelle
on
ne peut observer et agir sur une telle chose que par la communication
de messages à un acteur indépendant de l'autre côté, un
l'abstraction est nécessaire pour représenter (« prendre la place de ») le
courant
ou l'état souhaité de cette chose dans nos communications. Cela
l'abstraction est appelée une représentation [[REST](#)].

Aux fins du protocole HTTP, une « représentation » est une information qui
est
destiné à refléter un état passé, actuel ou souhaité d'un
ressource, dans un format qui peut être facilement communiqué via le
protocole, et qui consiste en un ensemble de métadonnées de représentation et
un
flux potentiellement illimité de données de représentation.

Un serveur d'origine peut être fourni avec, ou être capable de générer,

représentations multiples qui sont chacune destinées à refléter la l'état actuel d'une ressource cible. Dans ce cas, un algorithme est utilisé par le serveur d'origine pour sélectionner l'une de ces représentations comme le plus applicable à une demande donnée, généralement basé sur le contenu négociation. Cette « représentation sélectionnée » est utilisée pour fournir

Piste des normes Fielding & Reschke [Page 7]

[RFC 7231](#)

HTTP/1.1 Sémantique et contenu Juin 2014

données et métadonnées pour évaluer les demandes conditionnelles [[RFC7232](#)] et construction de la charge utile pour 200 (OK) et 304 (non modifié) réponses à GET ([Section 4.3.1](#)).

[3.1](#) . Métadonnées de représentation

Les champs d'en-tête de représentation fournissent des métadonnées sur la représentation. Lorsqu'un message inclut un corps de charge utile, le Les champs d'en-tête de représentation décrivent comment interpréter les données de représentation incluses dans le corps de la charge utile. En réponse à une Requête HEAD, les champs d'en-tête de représentation décrivent le données de représentation qui auraient été incluses dans le corps de la charge utile si la même requête avait été un GET.

Les champs d'en-tête suivants transmettent des métadonnées de représentation :

```
+-----+-----+
| Nom du champ d'en-tête | Défini dans... |
+-----+-----+
| Type de contenu | Section 3.1.1.5 |
| Codage de contenu | Section 3.1.2.2 |
| Contenu-Langue | Section 3.1.3.2 |
| Contenu-Emplacement | Section 3.1.4.2 |
+-----+-----+
```

[3.1.1](#) . Traitement des données de représentation**[3.1.1.1](#) . Type de support**

HTTP utilise les types de médias Internet [[RFC2046](#)] dans le Content-Type ([Section 3.1.1.5](#)) et Accepter ([Section 5.3.2](#)) les champs d'en-tête dans l'ordre

pour fournir un typage de données et une négociation de type ouverts et extensibles.

Les types de médias définissent à la fois un format de données et différents modèles de traitement :

comment traiter ces données en fonction de chaque contexte dans lequel elles est reçu.

```
type-média = type "/" sous-type *( OWS ";" paramètre OWS )
type = jeton
sous-type = jeton
```

Le type/sous-type PEUT être suivi de paramètres sous la forme de paires nom=valeur.

```
paramètre = jeton "=" ( jeton / chaîne entre guillemets )
```

Piste des normes Fielding & Reschke [Page 8]

[RFC 7231](#)

HTTP/1.1 Sémantique et contenu Juin 2014

Les jetons de type, de sous-type et de nom de paramètre ne sont pas sensibles à la casse.

Les valeurs des paramètres peuvent être sensibles à la casse ou non, selon la sémantique du nom du paramètre. La présence ou l'absence d'un le paramètre peut être important pour le traitement d'un type de média, en fonction de sa définition dans le registre des types de médias.

Une valeur de paramètre qui correspond à la production de jetons peut être transmis soit sous forme de jeton, soit dans une chaîne entre guillemets. et les valeurs sans guillemets sont équivalentes. Par exemple, les exemples sont tous équivalents, mais le premier est préféré pour cohérence:

```
texte/html;charset=utf-8
texte/html;charset=UTF-8
Texte/HTML ; Charset = utf-8 »
texte/html; charset="utf-8"
```

Les types de médias Internet doivent être enregistrés auprès de l'IANA conformément à
les procédures définies dans [[BCP13](#)].

Remarque : contrairement à certaines constructions similaires dans d'autres champs d'en-tête, les médias
les paramètres de type n'autorisent pas les espaces (même les « mauvais » espaces)
autour du caractère "=".

[3.1.1.2](#) . Jeu de caractères

HTTP utilise des noms de jeux de caractères pour indiquer ou négocier le caractère

Schéma de codage d'une représentation textuelle [[RFC6365](#)]. Un jeu de caractères est
identifié par un jeton insensible à la casse.

jeu de caractères = jeton

Les noms de jeux de caractères doivent être enregistrés dans les « jeux de caractères » de l'IANA.

registre (< <http://www.iana.org/assignments/character-sets> >) selon
aux procédures définies dans [[RFC2978](#)].

[3.1.1.3](#) . Canonisation et valeurs par défaut du texte

Les types de médias Internet sont enregistrés sous une forme canonique afin de

être interopérable entre des systèmes ayant différents formats d'encodage natifs.

Les représentations sélectionnées ou transférées via HTTP doivent être dans forme canonique, pour plusieurs des mêmes raisons décrites par le

Extensions de messagerie Internet polyvalentes (MIME) [[RFC2045](#)].
Cependant,

caractéristiques de performance des déploiements de messagerie électronique (c.-à-d., stocker et (transférer des messages à ses pairs) sont significativement différentes de celles commun à HTTP et au Web (services d'information basés sur un serveur). De plus, les contraintes de MIME pour des raisons de compatibilité avec les anciens protocoles de transfert de courrier ne s'appliquent pas à HTTP (voir [l'annexe A](#)).

Piste des normes Fielding & Reschke [Page 9]

[RFC 7231](#)

HTTP/1.1 Sémantique et contenu Juin 2014

La forme canonique de MIME exige que les sous-types de médias de type « texte »

Utilisez CRLF comme saut de ligne. HTTP permet le transfert de texte. médias avec CR ou LF seul représentant un saut de ligne, lorsque tel Les sauts de ligne sont cohérents pour une représentation entière. Un HTTP l'expéditeur PEUT générer, et un destinataire DOIT être capable d'analyser, la ligne

ruptures dans les supports textuels constitués de CRLF, CR nu ou LF nu.

De plus, les médias textuels dans HTTP ne se limitent pas aux jeux de caractères qui utilisent

octets 13 et 10 pour CR et LF, respectivement. Cette flexibilité concernant les sauts de ligne s'applique uniquement au texte dans une représentation

auquel un type de média « texte » a été attribué ; cela ne s'applique pas à types « multipart » ou éléments HTTP en dehors du corps de la charge utile (par exemple, champs d'en-tête).

Si une représentation est codée avec un codage de contenu, le sous-jacent les données doivent être sous une forme définie ci-dessus avant d'être codées.

[3.1.1.4](#) . Types multipartites

MIME fournit un certain nombre de types « multiparties » : des encapsulations de

une ou plusieurs représentations dans un même corps de message. Tous les types multipartites partagent une syntaxe commune, telle que définie dans [la section 5.1.1 de](#)

[\[RFC2046\]](#) et inclure un paramètre de limite dans le cadre du type de média valeur. Le corps du message est lui-même un élément de protocole ; un expéditeur DOIT

générer uniquement CRLF pour représenter les sauts de ligne entre les parties du corps.

Le cadrage des messages HTTP n'utilise pas la limite multipartite comme indicateur de la longueur du corps du message, bien qu'il puisse être utilisé par

implémentations qui génèrent ou traitent la charge utile. Par exemple, le type « multipart/form-data » est souvent utilisé pour transporter des données de formulaire

dans une requête, comme décrit dans [[RFC2388](#)], et le « multipart/

Le type « byteranges » est défini par cette spécification pour être utilisé dans certains 206

(Contenu partiel) réponses [[RFC7233](#)].

[3.1.1.5](#) . Type de contenu

Le champ d'en-tête « Content-Type » indique le type de média du représentation associée : soit la représentation incluse dans le charge utile du message ou la représentation sélectionnée, telle que déterminée par le

sémantique du message. Le type de média indiqué définit à la fois les données

format et comment ces données sont destinées à être traitées par un destinataire,
dans le cadre de la sémantique du message reçu, après tout contenu les codages indiqués par Content-Encoding sont décodés.

Type de contenu = type de média

Piste des normes Fielding & Reschke [Page 10]

[RFC 7231](#)

HTTP/1.1 Sémantique et contenu Juin 2014

Les types de médias sont définis à [la section 3.1.1.1](#) . Un exemple de champ est

Type de contenu : texte/html ; jeu de caractères = ISO-8859-4

Un expéditeur qui génère un message contenant un corps de charge utile DEVRAIT

générer un champ d'en-tête Content-Type dans ce message, sauf si le type de support prévu de la représentation jointe est inconnu du expéditeur. Si aucun champ d'en-tête Content-Type n'est présent, le destinataire

PEUT soit assumer un type de média « application/octet-stream » ([\[RFC2046\]](#), [Section 4.5.1](#)) ou examiner les données pour déterminer leur type.

Dans la pratique, les propriétaires de ressources ne configurent pas toujours correctement leurs

serveur d'origine pour fournir le type de contenu correct pour un représentation, avec pour résultat que certains clients examineront une le contenu de la charge utile et remplacent le type spécifié. Les clients qui le font

risquent donc de tirer des conclusions erronées, ce qui pourrait exposer des risques de sécurité (par exemple, « escalade de privilèges »). De plus, il est

impossible de déterminer l'intention de l'expéditeur en examinant les données format : de nombreux formats de données correspondent à plusieurs types de supports qui ne diffèrent que

dans le traitement sémantique. Les implémenteurs sont encouragés à fournir une

moyens de désactiver un tel « reniflage de contenu » lorsqu'il est utilisé.

[3.1.2](#) . Codage pour la compression ou l'intégrité

[3.1.2.1](#) . Codages de contenu

Les valeurs de codage de contenu indiquent une transformation de codage qui a ont été ou peuvent être appliqués à une représentation. Les codages de contenu sont

principalement utilisé pour permettre à une représentation d'être compressée ou

autrement utilement transformé sans perdre l'identité de son type de média sous-jacent et sans perte d'information. Fréquemment, la représentation est stockée sous forme codée, transmise directement, et décodé uniquement par le destinataire final.

codage de contenu = jeton

Toutes les valeurs de codage de contenu ne sont pas sensibles à la casse et doivent être

enregistré dans le « HTTP Content Coding Registry », tel que défini dans [Section 8.4](#) . Ils sont utilisés dans l'encodage Accept-Encoding ([Section 5.3.4](#)).

et les champs d'en-tête Content-Encoding ([Section 3.1.2.2](#)).

Piste des normes Fielding & Reschke [Page 11]

[RFC 7231](#)

HTTP/1.1 Sémantique et contenu Juin 2014

Les valeurs de codage de contenu suivantes sont définies par ceci spécification:

compresser (et x-compress) : voir [la section 4.2.1 de \[RFC7230\]](#) .

dégonfler : voir [la section 4.2.2 de \[RFC7230\]](#) .

gzip (et x-gzip) : voir [la section 4.2.3 de \[RFC7230\]](#) .

[3.1.2.2](#) . Codage du contenu

Le champ d'en-tête « Content-Encoding » indique quels codages de contenu ont été appliquées à la représentation, au-delà de celles inhérentes à la type de média, et donc quels mécanismes de décodage doivent être appliqués afin d'obtenir des données dans le type de média référencé par le Content-Type champ d'en-tête. Content-Encoding est principalement utilisé pour permettre à un les données de représentation doivent être compressées sans perdre l'identité de son type de média sous-jacent.

Content-Encoding = 1#content-coding

Un exemple de son utilisation est

Codage de contenu : gzip

Si un ou plusieurs codages ont été appliqués à une représentation, le l'expéditeur qui a appliqué les encodages DOIT générer un encodage de contenu champ d'en-tête qui répertorie les codages de contenu dans l'ordre dans lequel ils ont été appliqués. Informations complémentaires sur le codage les paramètres peuvent être fournis par d'autres champs d'en-tête non définis par celui-ci spécification.

Contrairement au codage de transfert ([section 3.3.1 de \[RFC7230\]](#)), les codages répertoriés dans Content-Encoding sont une caractéristique du représentation ; la représentation est définie en termes de codé forme, et toutes les autres métadonnées sur la représentation concernent la forme codée, sauf indication contraire dans la définition des métadonnées. En règle générale, la représentation n'est décodée que juste avant le rendu ou usage analogue.

Si le type de média inclut un codage inhérent, tel qu'un codage de données format qui est toujours compressé, alors cet encodage ne serait pas reformulé dans Content-Encoding même s'il s'avère que c'est le même algorithme comme l'un des codages de contenu. Un tel codage de contenu ne sera répertorié que si, pour une raison étrange, il est appliqué une seconde fois temps de former la représentation. De même, un serveur d'origine pourrait

choisir de publier les mêmes données sous forme de représentations multiples ne diffèrent que par le fait que le codage est défini comme faisant partie du type de contenu

Piste des normes Fielding & Reschke [Page 12]

[RFC 7231](#)

HTTP/1.1 Sémantique et contenu Juin 2014

ou Content-Encoding, car certains agents utilisateurs se comporteront différemment dans leur traitement de chaque réponse (par exemple, ouvrir une boîte de dialogue « Enregistrer sous ... » au lieu de la décompression et du rendu automatiques du contenu).

Un serveur d'origine PEUT répondre avec un code d'état de 415 (non pris en charge) Type de média) si une représentation dans le message de demande a un contenu codage qui n'est pas acceptable.

[3.1.3](#) . Langue du public

[3.1.3.1](#) . Balises de langue

Une balise de langue, telle que définie dans [[RFC5646](#)], identifie une langue naturelle langue parlée, écrite ou autrement transmise par les êtres humains pour communication d'informations à d'autres êtres humains. Ordinateur les langues sont explicitement exclues.

HTTP utilise des balises de langue dans Accept-Language et Champs d'en-tête Content-Language. Accept-Language utilise la définition plus large production de gamme linguistique définie dans [la section 5.3.5](#) , tandis que Content-Language utilise la production de balises de langue définie ci-dessous.

language-tag = <Language-Tag, voir [\[RFC5646\]](#), [Section 2.1](#) >

Une balise de langue est une séquence d'une ou plusieurs sous-balises insensibles à la casse, chacun séparé par un trait d'union (« - », %x2D). Dans la plupart des cas, La balise de langue se compose d'une sous-balise de langue principale qui identifie une vaste famille de langues apparentées (par exemple, « en » = anglais), qui est éventuellement suivi d'une série de sous-balises qui affinent ou restreignent cela la gamme de la langue (par exemple, « en-CA » = la variété de l'anglais tel que communiqué au Canada). Les espaces ne sont pas autorisés dans une langue. balise. Exemples de balises :

fr, en-US, es-419, az-Arab, x-pig-latin, man-Nkoo-GN

Voir [[RFC5646](#)] pour plus d'informations.

[3.1.3.2](#) . Contenu-Langue

Le champ d'en-tête « Content-Language » décrit la ou les langues naturelles du public visé par la représentation. Notez que ceci pourrait ne pas être équivalent à toutes les langues utilisées dans le représentation.

Contenu-Langue = 1#balise-langue

Piste des normes Fielding & Reschke [Page 13]

[RFC 7231](#)

HTTP/1.1 Sémantique et contenu Juin 2014

Les balises de langue sont définies dans [la section 3.1.3.1](#) . L'objectif principal de

Le langage de contenu doit permettre à un utilisateur d'identifier et de différencier

représentations selon la langue préférée des utilisateurs.

Ainsi, si le contenu est destiné uniquement à un public alphabétisé en danois,

le champ approprié est

Contenu-Language : da

Si aucun Content-Language n'est spécifié, la valeur par défaut est que le contenu

est destiné à tous les publics linguistiques. Cela pourrait signifier que l'expéditeur ne considère pas qu'il soit spécifique à une langue naturelle, ou que l'expéditeur ne sait pas à quelle langue il est destiné.

Plusieurs langues PEUVENT être répertoriées pour le contenu destiné à publics multiples. Par exemple, une interprétation du « Traité de Waitangi », présenté simultanément en version originale maorie et anglaise versions, nécessiteraient

Contenu-Language : mi, en

Cependant, ce n'est pas parce que plusieurs langues sont présentes dans un représentation ne signifie pas qu'elle est destinée à plusieurs publics linguistiques. Un exemple serait un cours de langue pour débutants amorcé, comme « Une première leçon de latin », qui est clairement destiné à un public anglophone. Dans ce cas, Content-Language n'inclurait correctement que « en ».

Le langage de contenu PEUT être appliqué à tout type de média -- il n'est pas limité aux documents textuels.

[3.1.4](#) . Identification

[3.1.4.1](#) . Identification d'une représentation

Lorsqu'une représentation complète ou partielle est transférée dans un message

charge utile, il est souvent souhaitable que l'expéditeur fournisse, ou le destinataire à déterminer, un identifiant pour une ressource correspondant à cette représentation.

Pour un message de demande :

o Si la requête comporte un champ d'en-tête Content-Location, alors le l'expéditeur affirme que la charge utile est une représentation du Ressource identifiée par la valeur du champ Content-Location. Cependant, une telle affirmation ne peut être fiable que si elle peut être vérifiée par d'autres moyens (non définis par la présente spécification). Les informations

pourrait encore être utile pour les liens vers l'historique des révisions.

Piste des normes Fielding & Reschke [Page 14]

o Sinon, la charge utile n'est pas identifiée.

Pour un message de réponse, les règles suivantes sont appliquées dans l'ordre jusqu'à ce qu'une correspondance soit trouvée :

1. Si la méthode de demande est GET ou HEAD et le code d'état de la réponse est 200 (OK), 204 (Aucun contenu), 206 (Contenu partiel) ou 304 (Non Modifié), la charge utile est une représentation de la ressource identifié par l'URI de la demande effective ([Section 5.5 de \[RFC7230\]](#)).

2. Si la méthode de demande est GET ou HEAD et le code d'état de la réponse est 203 (Information non autorisée), la charge utile est un représentation potentiellement modifiée ou améliorée de la cible ressource fournie par un intermédiaire.

3. Si la réponse comporte un champ d'en-tête Content-Location et son field-value est une référence au même URI que le champ effectif URI de requête, la charge utile est une représentation de la ressource identifié par l'URI de la demande effective.

4. Si la réponse comporte un champ d'en-tête Content-Location et son field-value est une référence à un URI différent de l'URI effectif URI de la demande, puis l'expéditeur affirme que la charge utile est un représentation de la ressource identifiée par le Content-Location valeur du champ. Cependant, une telle assertion ne peut être fiable que

si

cela peut être vérifié par d'autres moyens (non définis par ce spécification).

5. Sinon, la charge utile n'est pas identifiée.

[3.1.4.2](#) . Contenu-Emplacement

Le champ d'en-tête « Content-Location » fait référence à un URI qui peut être utilisé

comme identifiant d'une ressource spécifique correspondant à la représentation dans la charge utile de ce message. Autrement dit, si l'on devaient effectuer une requête GET sur cet URI au moment de cette la génération du message, alors une réponse 200 (OK) contiendrait le même représentation qui est incluse comme charge utile dans ce message.

Emplacement du contenu = URI absolu / URI partiel

La valeur Content-Location ne remplace pas la valeur effective URI de requête ([section 5.5 de \[RFC7230\]](#)). Il s'agit d'une représentation métadonnées. Sa syntaxe et sa sémantique sont identiques à celles du champ d'en-tête.

du même nom défini pour les parties de corps MIME dans [la section 4 de \[RFC2557\]](#) . Cependant, son apparition dans un message HTTP présente certaines implications particulières pour les destinataires HTTP.

Piste des normes Fielding & Reschke [Page 15]

[RFC 7231](#)

HTTP/1.1 Sémantique et contenu Juin 2014

Si Content-Location est inclus dans une réponse 2xx (réussie) message et sa valeur se réfèrent (après conversion en forme absolue) à un URI qui est le même que l'URI de la demande effective, puis le destinataire PEUT considérer la charge utile comme une représentation actuelle de celle-ci ressource à l'heure indiquée par la date d'origine du message. Pour une requête GET ([Section 4.3.1](#)) ou HEAD ([Section 4.3.2](#)), c'est le identique à la sémantique par défaut lorsqu'aucun Content-Location n'est fourni par

le serveur. Pour une requête de changement d'état comme PUT ([Section 4.3.4](#)) ou

POST ([Section 4.3.3](#)), cela implique que la réponse du serveur contient la nouvelle représentation de cette ressource, la distinguant ainsi à partir de représentations qui pourraient seulement rendre compte de l'action (par exemple,

« Ça a marché ! »). Cela permet aux créateurs d'applications de mettre à jour leurs

copies locales sans avoir besoin d'une requête GET ultérieure.

Si Content-Location est inclus dans une réponse 2xx (réussie) message et sa valeur de champ font référence à un URI différent du URI de demande efficace, alors le serveur d'origine affirme que l'URI est un identifiant pour une ressource différente correspondant à la ressource jointe

représentation. Une telle affirmation ne peut être fiable que si les deux identifiants

partagent le même propriétaire de ressource, ce qui ne peut pas être programmiquement déterminé via HTTP.

- o Pour une réponse à une requête GET ou HEAD, ceci est une indication que l'URI de la demande effective fait référence à une ressource qui est sous réserve de négociation du contenu et de l'emplacement du contenu field-value est un identifiant plus spécifique pour le champ sélectionné représentation.

- o Pour une réponse 201 (Créé) à une méthode de changement d'état, un Champ Content-Location : valeur identique à l'emplacement field-value indique que cette charge utile est une charge utile actuelle représentation de la ressource nouvellement créée.

- o Sinon, un tel Content-Location indique que cette charge utile est une représentation faisant état de l'état de l'action demandée et que le même rapport est disponible (pour un accès futur avec GET) à l'URI donnée. Par exemple, une transaction d'achat effectuée via un La requête POST peut inclure un document de réception comme charge utile de la réponse 200 (OK) ; le champ Content-Location-value fournit un identifiant permettant de récupérer une copie de ce même reçu dans le avenir.

Un agent utilisateur qui envoie Content-Location dans un message de demande est

indiquant que sa valeur fait référence à l'endroit où l'agent utilisateur

s'est initialement positionné
obtenu le contenu de la représentation ci-jointe (avant toute
modifications apportées par cet agent utilisateur). Autrement dit,
l'utilisateur
l'agent fournit un lien retour vers la source de l'original
représentation.

Piste des normes Fielding & Reschke [Page 16]

[RFC 7231](#)

HTTP/1.1 Sémantique et contenu Juin 2014

Un serveur d'origine qui reçoit un champ Content-Location dans une requête le message DOIT traiter l'information comme un contexte de demande transitoire plutôt que comme des métadonnées à sauvegarder textuellement dans le cadre de représentation. Un serveur d'origine PEUT utiliser ce contexte pour guider traiter la demande ou de la sauvegarder pour d'autres utilisations, par exemple dans Liens sources ou métadonnées de versionnage. Cependant, un serveur d'origine DOIT N'utilisez PAS ces informations contextuelles pour modifier la sémantique de la demande.

Par exemple, si un client fait une requête PUT sur une ressource négociée et le serveur d'origine accepte ce PUT (sans redirection), alors le nouvel état de cette ressource devrait être cohérent avec le une représentation fournie dans ce PUT ; le Content-Location ne peut pas être utilisé comme une forme d'identifiant de sélection de contenu inversé pour mettre à jour une seule des représentations négociées. Si l'agent utilisateur avait voulait la dernière sémantique, il aurait appliqué le PUT directement à l'URI Content-Location.

[3.2](#) . Données de représentation

Les données de représentation associées à un message HTTP sont soit fourni comme corps de charge utile du message ou référencé par le sémantique du message et URI de la requête effective. La représentation les données sont dans un format et un codage définis par la représentation champs d'en-tête de métadonnées.

Le type de données des données de représentation est déterminé via l'en-tête champs Content-Type et Content-Encoding. Ceux-ci définissent une couche à deux niveaux, modèle de codage ordonné :

représentation-données := Content-Encoding(Content-Type(bits))

[3.3](#) . Sémantique de la charge utile

Certains messages HTTP transfèrent une représentation complète ou partielle sous forme de le message « charge utile ». Dans certains cas, une charge utile peut contenir uniquement les champs d'en-tête de la représentation associée (par exemple, les réponses à HEAD) ou seulement une ou plusieurs parties des données de représentation (par exemple, le 206 (Contenu partiel) code d'état).

Le but d'une charge utile dans une requête est défini par la méthode sémantique. Par exemple, une représentation dans la charge utile d'un PUT la requête ([section 4.3.4](#)) représente l'état souhaité de la cible ressource si la demande est appliquée avec succès, alors qu'une

représentation dans la charge utile d'une requête POST ([Section 4.3.3](#))
représente les informations à traiter par la ressource cible.

Piste des normes Fielding & Reschke [Page 17]

[RFC 7231](#)

HTTP/1.1 Sémantique et contenu Juin 2014

Dans une réponse, l'objectif de la charge utile est défini à la fois par la demande

méthode et le code d'état de la réponse. Par exemple, la charge utile d'un La réponse 200 (OK) à GET ([Section 4.3.1](#)) représente l'état actuel de la ressource cible, telle qu'observée au moment du message date d'origine ([section 7.1.1.2](#)), tandis que la charge utile du même Le code d'état dans une réponse à POST peut représenter soit le résultat du traitement ou le nouvel état de la ressource cible après Application du traitement. Messages de réponse avec un code d'erreur contiennent généralement une charge utile qui représente la condition d'erreur, telle que qu'il décrit l'état d'erreur et les prochaines étapes suggérées pour le résoudre.

Champs d'en-tête qui décrivent spécifiquement la charge utile, plutôt que la représentation associée, sont appelés « en-tête de charge utile » champs". Les champs d'en-tête de charge utile sont définis dans d'autres parties de ce spécification, en raison de leur impact sur l'analyse des messages.

```
+-----+-----+
| Nom du champ d'en-tête | Défini dans... |
+-----+-----+
| Longueur du contenu | Section 3.3.2 de \[RFC7230\] |
| Plage de contenu | Section 4.2 de \[RFC7233\] |
| Bande-annonce | Section 4.4 de \[RFC7230\] |
| Codage de transfert | Section 3.3.1 de \[RFC7230\] |
+-----+-----+
```

[3.4](#) . Négociation du contenu

Lorsque les réponses transmettent des informations de charge utile, qu'elles indiquent une

succès ou une erreur, le serveur d'origine a souvent différentes manières de représentant ces informations ; par exemple, dans différents formats, langues ou encodages. De même, différents utilisateurs ou agents utilisateurs peuvent avoir des capacités, des caractéristiques ou des préférences différentes

qui pourrait influencer la représentation, parmi celles disponibles, serait préférable de livrer. Pour cette raison, HTTP fournit des mécanismes pour la négociation de contenu.

Cette spécification définit deux modèles de négociation de contenu qui peut être rendu visible dans le protocole : « proactif », où le le serveur sélectionne la représentation en fonction des informations déclarées par l'agent utilisateur

préférences et négociation « réactive », où le serveur fournit une Liste de représentations parmi lesquelles l'agent utilisateur peut choisir. Autre

les modèles de négociation de contenu incluent le « contenu conditionnel », où

la représentation se compose de plusieurs parties qui sont sélectivement rendu en fonction des paramètres de l'agent utilisateur, « contenu actif »,

où le
la représentation contient un script qui fait des ajouts (plus
demandes spécifiques) basées sur les caractéristiques de l'agent utilisateur,
et
« Négociation de contenu transparent » ([[RFC2295](#)]), où le contenu

Piste des normes Fielding & Reschke [Page 18]

[RFC 7231](#)

HTTP/1.1 Sémantique et contenu Juin 2014

La sélection est effectuée par un intermédiaire. Ces modèles ne sont pas mutuellement exclusifs, et chacun présente des compromis en termes d'applicabilité et praticité.

Notez que, dans tous les cas, HTTP n'est pas conscient de la sémantique des ressources.

La cohérence avec laquelle un serveur d'origine répond aux requêtes, au fil du temps et des différentes dimensions de la négociation de contenu, et

ainsi la « similitude » des représentations observées d'une ressource sur le temps est entièrement déterminé par l'entité ou l'algorithme sélectionné ou génère ces réponses. HTTP ne prête aucune attention à l'homme derrière le rideau.

[3.4.1](#) . Négociation proactive

Lorsque les préférences de négociation de contenu sont envoyées par l'agent utilisateur dans un demande d'encourager un algorithme situé sur le serveur à sélectionner le représentation privilégiée, on parle alors de négociation proactive (c'est-à-dire

négociation pilotée par le serveur). La sélection est basée sur les données disponibles représentations d'une réponse (les dimensions sur lesquelles elle pourrait varier, comme la langue, le codage du contenu, etc.) par rapport à divers informations fournies dans la demande, y compris les informations explicites champs de négociation de [la section 5.3](#) et caractéristiques implicites, telles comme l'adresse réseau du client ou des parties du champ User-Agent.

La négociation proactive est avantageuse lorsque l'algorithme de il est difficile de choisir parmi les représentations disponibles décrire à un agent utilisateur, ou lorsque le serveur souhaite envoyer son « meilleure estimation » à l'agent utilisateur avec la première réponse (en espérant pour éviter le délai aller-retour d'une demande ultérieure si le « meilleur « deviner » est suffisant pour l'utilisateur). Afin d'améliorer selon la supposition du serveur, un agent utilisateur PEUT envoyer des champs d'en-tête de requête qui décrire ses préférences.

La négociation proactive présente de sérieux inconvénients :

- o Il est impossible pour le serveur de déterminer avec précision ce qui pourrait être « le meilleur » pour un utilisateur donné, car cela nécessiterait une connaissance des capacités de l'agent utilisateur et de utilisation prévue de la réponse (par exemple, l'utilisateur souhaite-t-il la voir) à l'écran ou l'imprimer sur papier ?);

- o Demander à l'agent utilisateur de décrire ses capacités dans chaque requête

peut être à la fois très inefficace (étant donné que seul un faible pourcentage des réponses ont des représentations multiples) et un risque potentiel à la vie privée de l'utilisateur ;

- o Cela complique la mise en œuvre d'un serveur d'origine et algorithmes pour générer des réponses à une demande ; et,

Piste des normes Fielding & Reschke [Page 19]

[RFC 7231](#)

HTTP/1.1 Sémantique et contenu Juin 2014

o Cela limite la réutilisabilité des réponses pour la mise en cache partagée.

Un agent utilisateur ne peut pas compter sur des préférences de négociation proactives.

Le serveur d'origine pourrait ne pas implémenter la négociation proactive pour la ressource demandée ou pourrait décider que l'envoi d'une réponse non conforme à celle de l'agent utilisateur est acceptable. Les préférences sont meilleures que l'envoi d'une réponse 406 (non acceptable).

Un champ d'en-tête Vary ([Section 7.1.4](#)) est souvent envoyé dans une réponse sous réserve de négociation proactive pour indiquer quelles parties des informations demandées ont été utilisées dans l'algorithme de sélection.

[3.4.2](#) . Négociation réactive

Avec la négociation réactive (également appelée négociation pilotée par l'agent), la sélection de la meilleure représentation de réponse (quelle que soit la code d'état) est exécuté par l'agent utilisateur après avoir reçu une réponse initiale du serveur d'origine qui contient une liste de ressources pour des représentations alternatives. Si l'agent utilisateur n'est pas satisfait par la représentation de réponse initiale, il peut effectuer une Requête GET sur une ou plusieurs des ressources alternatives sélectionnées basé sur les métadonnées incluses dans la liste, pour obtenir une forme différente de représentation de cette réponse. La sélection d'alternatives pourrait être effectuée automatiquement par l'agent utilisateur ou manuellement par l'utilisateur à partir d'un menu généré (éventuellement hypertexte).

Notez que ce qui précède fait référence aux représentations de la réponse, dans les représentations de la ressource. L'alternative les représentations ne sont considérées que comme des représentations de la cible ressource si la réponse dans laquelle ces alternatives sont fournies a la sémantique d'être une représentation de la ressource cible (par exemple, une réponse 200 (OK) à une requête GET) ou a la sémantique de fournir des liens vers des représentations alternatives de la cible ressource (par exemple, une réponse 300 (choix multiples) à une requête GET).

Un serveur peut choisir de ne pas envoyer de représentation initiale, d'autres que la liste des alternatives, et indiquent ainsi que la réaction La négociation par l'agent utilisateur est privilégiée. Par exemple, alternatives énumérées dans les réponses avec le 300 (choix multiples) et Les codes d'état 406 (non acceptable) incluent des informations sur les représentations disponibles afin que l'utilisateur ou l'agent utilisateur puisse réagir en

faire une sélection.

La négociation réactive est avantageuse lorsque la réponse varie sur les dimensions couramment utilisées (telles que le type, la langue ou l'encodage), lorsque le serveur d'origine est incapable de déterminer l'identité d'un agent utilisateur, les capacités d'examiner la demande, et généralement lorsque le public les caches sont utilisés pour répartir la charge du serveur et réduire l'utilisation du réseau.

Piste des normes Fielding & Reschke [Page 20]

[RFC 7231](#)

HTTP/1.1 Sémantique et contenu Juin 2014

La négociation réactive souffre des inconvénients de la transmission d'une liste d'alternatives à l'agent utilisateur, qui dégrade la perception de l'utilisateur latence si transmis dans la section d'en-tête et nécessitant une seconde demande d'obtenir une représentation alternative. De plus, cette la spécification ne définit pas de mécanisme permettant de prendre en charge automatiquement sélection, bien qu'elle n'empêche pas un tel mécanisme d'être développé comme une extension.

[4. Méthodes de requête](#)

[4.1 . Aperçu](#)

Le jeton de méthode de requête est la principale source de sémantique de requête ;
il indique le but pour lequel le client a fait cette demande
et ce que le client attend comme résultat réussi.

La sémantique de la méthode de requête peut être davantage spécialisée par sémantique de certains champs d'en-tête lorsqu'ils sont présents dans une requête ([Section 5](#))
si ces sémantiques supplémentaires n'entrent pas en conflit avec la méthode.
par exemple, un client peut envoyer des champs d'en-tête de requête conditionnelle
([Section 5.2](#)) pour rendre l'action demandée conditionnelle à l'état actuel état de la ressource cible ([[RFC7232](#)]).

méthode = jeton

HTTP a été conçu à l'origine pour être utilisable comme interface pour systèmes d'objets distribués. La méthode de requête a été envisagée comme appliquer la sémantique à une ressource cible de la même manière que l'invocation d'une méthode définie sur un objet identifié s'appliquerait sémantique. Le jeton de méthode est sensible à la casse, car il pourrait être utilisé comme passerelle vers des systèmes basés sur des objets avec une méthode sensible à la casse noms.

Contrairement aux objets distribués, les méthodes de requête standardisées dans HTTP
ne sont pas spécifiques aux ressources, puisque les interfaces uniformes assurent
meilleure visibilité et réutilisation dans les systèmes basés sur le réseau [[REST](#)]. Une fois
définie, une méthode standardisée devrait avoir la même sémantique lorsque appliqué à n'importe quelle ressource, bien que chaque ressource détermine pour elle-même
si ces sémantiques sont implémentées ou autorisées.

Cette spécification définit un certain nombre de méthodes normalisées qui sont
couramment utilisé dans HTTP, comme indiqué dans le tableau suivant.

convention, les méthodes standardisées sont définies en majuscules
Lettres US-ASCII.

Piste des normes Fielding & Reschke [Page 21]

[RFC 7231](#)

HTTP/1.1 Sémantique et contenu Juin 2014

+-----+-----+-----+-----+-----+				
Méthode Description Sec.				
+-----+-----+-----+-----+-----+				
GET	Transférer une représentation actuelle de la cible	4.3.1		
	ressource.			
HEAD	Identique à GET, mais ne transfère que la ligne d'état	4.3.2		
	et section d'en-tête.			
POST	Effectuer un traitement spécifique aux ressources sur le	4.3.3		
	demande de charge utile.			
PUT	Remplacer toutes les représentations actuelles du	4.3.4		
	ressource cible avec la charge utile de la requête.			
SUPPRIMER	Supprimer toutes les représentations actuelles du	4.3.5		
	ressource cible.			
CONNECTER	Établir un tunnel vers le serveur identifié par	4.3.6		
	la ressource cible.			
OPTIONS	Décrivez les options de communication pour le	4.3.7		
	ressource cible.			
TRACE	Effectuer un test de boucle de message le long du chemin	4.3.8		
	à la ressource cible.			
+-----+-----+-----+-----+-----+				

Tous les serveurs à usage général DOIVENT prendre en charge les méthodes GET et HEAD.

Toutes les autres méthodes sont FACULTATIVES.

Des méthodes supplémentaires, en dehors du cadre de cette spécification, ont été normalisées pour une utilisation dans HTTP. Toutes ces méthodes doivent être

enregistré dans la méthode « Hypertext Transfer Protocol (HTTP) « Registre » maintenu par l'IANA, tel que défini dans [la section 8.1](#) .

L'ensemble des méthodes autorisées par une ressource cible peut être répertorié dans un

Autoriser le champ d'en-tête ([section 7.4.1](#)). Cependant, l'ensemble des champs autorisés

Les méthodes peuvent changer dynamiquement. Lorsqu'une méthode de requête est reçue

qui n'est pas reconnu ou non implémenté par un serveur d'origine, le le serveur d'origine DEVRAIT répondre avec le statut 501 (non implémenté) code. Lorsqu'une méthode de requête connue d'une origine est reçue serveur mais non autorisé pour la ressource cible, le serveur d'origine DEVRAIT répondre avec le code d'état 405 (méthode non autorisée).

[4.2](#) . Propriétés communes des méthodes

[4.2.1](#) . Méthodes sûres

Les méthodes de requête sont considérées comme « sûres » si leur sémantique définie est

essentiellement en lecture seule ; c'est-à-dire que le client ne demande pas et ne

ne vous attendez pas à un changement d'état sur le serveur d'origine à la suite de

appliquer une méthode sûre à une ressource cible. De même, une méthode raisonnable l'utilisation d'une méthode sûre ne devrait pas causer de préjudice, de perte de propriété ou charge inhabituelle sur le serveur d'origine.

Piste des normes Fielding & Reschke [Page 22]

[RFC 7231](#)

HTTP/1.1 Sémantique et contenu Juin 2014

Cette définition des méthodes sûres n'empêche pas une implémentation d'inclure un comportement potentiellement dangereux, qui n'est pas entièrement en lecture seule, ou qui provoque des effets secondaires lors de l'invocation d'une méthode sécurisée méthode. Ce qui est important, cependant, c'est que le client n'ait pas demander un comportement supplémentaire et ne peut être tenu responsable de Par exemple, la plupart des serveurs ajoutent des informations de requête pour accéder fichiers journaux à la fin de chaque réponse, quelle que soit la méthode, et qui est considérée comme sûre même si le stockage des journaux peut se remplir et font planter le serveur. De même, une requête sécurisée initiée en sélectionnant une publicité sur le Web aura souvent le côté effet de la facturation d'un compte publicitaire.

Parmi les méthodes de requête définies par cette spécification, GET, HEAD, Les méthodes OPTIONS et TRACE sont définies comme étant sûres.

Le but de la distinction entre les méthodes sûres et dangereuses est de permettre des processus de récupération automatisés (spiders) et des performances de cache optimisation (pré-extraction) pour fonctionner sans crainte de causer des dommages. en outre, il permet à un agent utilisateur d'appliquer des contraintes appropriées sur l'utilisation automatisée de méthodes dangereuses lors du traitement de données potentiellement contenu non fiable.

Un agent utilisateur DOIT faire la distinction entre les méthodes sûres et non sûres lorsque présenter des actions potentielles à un utilisateur, de telle sorte que l'utilisateur puisse être être informé d'une action dangereuse avant qu'elle ne soit demandée.

Lorsqu'une ressource est construite de telle sorte que les paramètres à l'intérieur de la l'URI de requête efficace a pour effet de sélectionner une action, c'est la responsabilité du propriétaire de la ressource de s'assurer que l'action est conforme à la sémantique de la méthode de requête. Par exemple, Il est courant que les logiciels d'édition de contenu Web utilisent des actions dans paramètres de requête, tels que « page?do=delete ». Si l'objectif d'une telle requête la ressource doit effectuer une action dangereuse, alors le propriétaire de la ressource DOIT désactiver ou interdire cette action lorsqu'elle est accessible à l'aide d'un coffre-fort méthode de requête. Ne pas le faire entraînerait des conséquences fâcheuses. effets lorsque les processus automatisés effectuent une opération GET sur chaque référence URI

pour des raisons de maintenance des liens, de pré-extraction, de création d'une recherche indice, etc.

4.2.2 . Méthodes idempotentes

Une méthode de requête est considérée comme « idempotente » si l'effet escompté sur le serveur de plusieurs requêtes identiques avec cette méthode est le même effet que pour une seule requête de ce type. Parmi les méthodes de requête définies par cette spécification, PUT, DELETE et les méthodes de requête sécurisées sont idempotents.

Piste des normes Fielding & Reschke [Page 23]

[RFC 7231](#)

HTTP/1.1 Sémantique et contenu Juin 2014

Comme la définition de sûr, la propriété idempotente s'applique uniquement à ce qui a été demandé par l'utilisateur ; un serveur est libre d'enregistrer chaque

demande séparément, conserver un historique de contrôle des révisions ou mettre en œuvre autres effets secondaires non idempotents pour chaque demande idempotente.

Les méthodes idempotentes se distinguent car la requête peut être répétée automatiquement si une panne de communication se produit avant le client peut lire la réponse du serveur. Par exemple, si un client envoie une requête PUT et la connexion sous-jacente est fermée avant qu'une réponse ne soit reçue, le client peut alors établir un nouveau connexion et relancer la requête idempotente. Il sait que la répétition la demande aura le même effet escompté, même si l'original la demande a réussi, même si la réponse peut différer.

[4.2.3](#) . Méthodes pouvant être mises en cache

Les méthodes de requête peuvent être définies comme « pouvant être mises en cache » pour indiquer que

les réponses à ces questions peuvent être stockées pour une réutilisation ultérieure ; par

exigences spécifiques, voir [[RFC7234](#)]. En général, les méthodes sûres qui ne dépendent pas d'une réponse actuelle ou faisant autorité sont définis comme

pouvant être mis en cache ; cette spécification définit GET, HEAD et POST comme

pouvant être mis en cache, bien que la grande majorité des caches les implémentations ne prennent en charge que GET et HEAD.

[4.3](#) . Définitions des méthodes

[4.3.1](#) . OBTENIR

La méthode GET demande le transfert d'une représentation actuellement sélectionnée

pour la ressource cible. GET est le principal mécanisme d'information la récupération et l'objet de presque toutes les optimisations de performances.

Ainsi, lorsque les gens parlent de récupérer des informations identifiables via HTTP, ils font généralement référence à une requête GET.

Il est tentant de considérer les identifiants de ressources comme des systèmes de fichiers distants.

noms de chemin et de représentations comme étant une copie du contenu de de tels fichiers. En fait, c'est ainsi que de nombreuses ressources sont implémentées (voir

[Section 9.1](#) pour les considérations de sécurité connexes). Cependant, il existe

En pratique, aucune limitation de ce type n'est observée. L'interface HTTP d'une ressource

est tout aussi susceptible d'être implémenté qu'un arbre d'objets de contenu, un

vue programmatique sur divers enregistrements de bases de données, ou une passerelle vers d'autres systèmes d'information. Même lorsque le mécanisme de mappage d'URI est lié à un système de fichiers, un serveur d'origine peut être configuré pour exécuter le fichiers avec la requête en entrée et envoyer la sortie en tant que représentation plutôt que de transférer les fichiers directement. Quoi qu'il en soit, seul le serveur d'origine a besoin de savoir comment chacune de ses ressources

Piste des normes Fielding & Reschke [Page 24]

[RFC 7231](#)

HTTP/1.1 Sémantique et contenu Juin 2014

les identifiants correspondent à une implémentation et à la manière dont chacun

l'implémentation parvient à sélectionner et à envoyer une représentation actuelle de la ressource cible dans une réponse à GET.

Un client peut modifier la sémantique de GET pour qu'elle soit une « demande de plage »,

demande le transfert d'une ou plusieurs parties seulement du fichier sélectionné

représentation, en envoyant un champ d'en-tête Range dans la requête

([[RFC7233](#)]).

Une charge utile dans un message de requête GET n'a pas de sémantique définie ;

l'envoi d'un corps de charge utile sur une requête GET peut entraîner des erreurs existantes

implémentations pour rejeter la demande.

La réponse à une requête GET peut être mise en cache ; un cache PEUT l'utiliser pour

satisfaire les requêtes GET et HEAD ultérieures, sauf indication contraire par le champ d'en-tête Cache-Control ([Section 5.2 de \[RFC7234\]](#)).

[4.3.2](#) . TÊTE

La méthode HEAD est identique à GET sauf que le serveur NE DOIT PAS envoyer un corps de message dans la réponse (c'est-à-dire que la réponse se termine à

la fin de la section d'en-tête). Le serveur DEVRAIT envoyer le même champs d'en-tête en réponse à une requête HEAD comme il l'aurait fait si la requête était un GET, sauf que les champs d'en-tête de charge utile ([Section 3.3](#)) PEUT être omise. Cette méthode peut être utilisée pour

obtenir

métadonnées sur la représentation sélectionnée sans transférer les données de représentation et est souvent utilisé pour tester les liens hypertextes pour

validité, accessibilité et modification récente.

Une charge utile dans un message de requête HEAD n'a pas de sémantique définie ;

l'envoi d'un corps de charge utile sur une requête HEAD peut entraîner des erreurs existantes

implémentations pour rejeter la demande.

La réponse à une requête HEAD peut être mise en cache ; un cache PEUT l'utiliser pour

satisfaire les demandes HEAD ultérieures, sauf indication contraire du Champ d'en-tête Cache-Control ([Section 5.2 de \[RFC7234\]](#)). Un HEAD

La réponse peut également avoir un effet sur les réponses précédemment mises en cache

OBTENIR ; voir [la section 4.3.5 de \[RFC7234\]](#) .

4.3.3 . POSTE

La méthode POST demande que la ressource cible traite le représentation incluse dans la demande selon les ressources sémantique spécifique. Par exemple, POST est utilisé pour les cas suivants : fonctions (entre autres) :

- o Fournir un bloc de données, comme les champs saisis dans un HTML forme, à un processus de traitement de données ;

Piste des normes Fielding & Reschke [Page 25]

[RFC 7231](#)

HTTP/1.1 Sémantique et contenu Juin 2014

- o Publier un message sur un babillard, un groupe de discussion, une liste de diffusion,
blog, ou groupe d'articles similaire ;
- o Création d'une nouvelle ressource qui n'a pas encore été identifiée par le serveur d'origine ; et
- o Ajout de données à la ou aux représentations existantes d'une ressource.

Un serveur d'origine indique la sémantique de réponse en choisissant une code d'état approprié en fonction du résultat du traitement du Requête POST ; presque tous les codes d'état définis par cette spécification peut être reçue dans une réponse à POST (les exceptions étant 206 (contenu partiel), 304 (non modifié) et 416 (plage non modifiée) Satisfaisant)).

Si une ou plusieurs ressources ont été créées sur le serveur d'origine en tant que résultat du traitement réussi d'une requête POST, le serveur d'origine DEVRAIT envoyer une réponse 201 (Créé) contenant un en-tête Location champ qui fournit un identifiant pour la ressource principale créée ([Section 7.1.2](#)) et une représentation qui décrit le statut du demande en faisant référence à la ou aux nouvelles ressources.

Les réponses aux requêtes POST ne peuvent être mises en cache que lorsqu'elles incluent informations de fraîcheur explicites (voir [la section 4.2.1 de \[RFC7234\]](#)). Cependant, la mise en cache POST n'est pas largement implémentée. Dans les cas où un le serveur d'origine souhaite que le client puisse mettre en cache le résultat d'une POST d'une manière qui peut être réutilisée par un GET ultérieur, le serveur d'origine PEUT envoyer une réponse 200 (OK) contenant le résultat et un Champ d'en-tête Content-Location qui a la même valeur que celle du POST URI de demande effective ([Section 3.1.4.2](#)).

Si le résultat du traitement d'un POST était équivalent à un représentation d'une ressource existante, un serveur d'origine PEUT rediriger l'agent utilisateur à cette ressource en envoyant une réponse 303 (Voir autre) avec l'identifiant de la ressource existante dans le champ Emplacement. Ceci présente l'avantage de fournir à l'agent utilisateur un identifiant de ressource et transférer la représentation via une méthode plus adaptée à mise en cache partagée, mais au prix d'une requête supplémentaire si l'utilisateur l'agent n'a pas déjà la représentation mise en cache.

[4.3.4](#) . METTRE

La méthode PUT demande que l'état de la ressource cible soit créé ou remplacé par l'état défini par la représentation

inclus dans la charge utile du message de requête. Un PUT réussi d'une requête donnée
représentation suggérerait qu'une GET ultérieure sur ce même la ressource cible entraînera une représentation équivalente envoyé une réponse 200 (OK). Cependant, rien ne garantit que

Piste des normes Fielding & Reschke [Page 26]

[RFC 7231](#)

HTTP/1.1 Sémantique et contenu Juin 2014

un tel changement d'état sera observable, puisque la ressource cible pourrait être traitée par d'autres agents utilisateurs en parallèle, ou pourrait être soumise à un traitement dynamique par le serveur d'origine, avant toute réponse GET ultérieure est reçue. Une réponse réussie implique seulement que l'intention de l'agent utilisateur a été réalisée au moment de son traitement par le serveur d'origine.

Si la ressource cible n'a pas de représentation actuelle et que PUT en crée une avec succès, puis le serveur d'origine DOIT en informer le l'agent utilisateur en envoyant une réponse 201 (Créé). Si la cible la ressource a une représentation actuelle et cette représentation est modifiée avec succès conformément à l'état du document ci-joint représentation, alors le serveur d'origine DOIT envoyer soit un 200 (OK) soit une réponse 204 (Aucun contenu) pour indiquer la réussite de l'demande.

Un serveur d'origine DOIT ignorer les champs d'en-tête non reconnus reçus une requête PUT (c'est-à-dire ne pas les enregistrer dans le cadre de l'état de la ressource).

Un serveur d'origine DOIT vérifier que la représentation PUT est cohérent avec toutes les contraintes imposées par le serveur à la cible ressource qui ne peut pas ou ne sera pas modifiée par le PUT. Ceci est particulièrement important lorsque le serveur d'origine utilise des informations de configuration liées à l'URI afin de définir les valeurs pour les métadonnées de représentation dans les réponses GET. Lorsqu'une réponse PUT la représentation est incompatible avec la ressource cible, l'origine le serveur DEVRAIT soit les rendre cohérents, en transformant la représentation ou modification de la configuration des ressources, ou réponse avec un message d'erreur approprié contenant suffisamment d'informations pour expliquer pourquoi la représentation est inappropriée. Le 409 (Conflit) ou les codes d'état 415 (type de support non pris en charge) sont suggérés, avec le ce dernier étant spécifique aux contraintes sur les valeurs Content-Type.

Par exemple, si la ressource cible est configurée pour toujours avoir un Le type de contenu « text/html » et la représentation étant PUT ont un Type de contenu « image/jpeg », le serveur d'origine doit effectuer l'une des opérations suivantes :

- a. reconfigurer la ressource cible pour refléter le nouveau type de média ;
- b. transformer la représentation PUT en un format cohérent avec celui-ci de la ressource avant de l'enregistrer en tant que nouvel état de ressource ; ou,
- c. rejeter la demande avec une réponse 415 (type de média non pris en charge) indiquant que la ressource cible est limitée à « texte/html », peut-être en incluant un lien vers une autre ressource qui serait un

cible appropriée pour la nouvelle représentation.

Piste des normes Fielding & Reschke [Page 27]

[RFC 7231](#)

HTTP/1.1 Sémantique et contenu Juin 2014

HTTP ne définit pas exactement comment une méthode PUT affecte l'état d'un serveur d'origine au-delà de ce qui peut être exprimé par l'intention de l'utilisateur requête de l'agent et la sémantique de la réponse du serveur d'origine. ne définit pas ce que pourrait être une ressource, dans aucun sens du terme, au-delà de l'interface fournie via HTTP. Elle ne définit pas comment l'état de la ressource est « stocké », ni comment ce stockage pourrait changer en tant que résultat d'un changement d'état de la ressource, ni comment le serveur d'origine traduit l'état des ressources en représentations. En général, tous les détails d'implémentation derrière l'interface de ressource sont intentionnellement caché par le serveur.

Un serveur d'origine NE DOIT PAS envoyer un champ d'en-tête de validateur ([Section 7.2](#)), comme un champ ETag ou Last-Modified, dans une réponse réussie à PUT à moins que les données de représentation de la requête a été sauvé sans aucune transformation appliquée au corps (c'est-à-dire le les nouvelles données de représentation de la ressource sont identiques à la représentation données reçues dans la requête PUT) et la valeur du champ validateur reflète la nouvelle représentation. Cette exigence permet à un utilisateur agent pour savoir quand le corps de représentation qu'il a en mémoire reste courant à la suite du PUT, donc n'ayant pas besoin d'être récupéré à nouveau depuis le serveur d'origine, et que le(s) nouveau(x) validateur(s) a/ont reçu dans la réponse peut être utilisé pour de futures demandes conditionnelles afin pour éviter les écrasements accidentels ([Section 5.2](#)).

La différence fondamentale entre les méthodes POST et PUT est mis en évidence par l'intention différente de la représentation ci-jointe. La ressource cible dans une requête POST est destinée à gérer le représentation fermée selon la sémantique propre à la ressource, alors que la représentation incluse dans une requête PUT est définie comme remplaçant l'état de la ressource cible. D'où l'objectif de PUT est idempotent et visible pour les intermédiaires, même si l'exact l'effet n'est connu que du serveur d'origine.

L'interprétation correcte d'une requête PUT suppose que l'agent utilisateur sait quelle ressource cible est souhaitée. Un service qui sélectionne une URI approprié au nom du client, après avoir reçu un changement d'état la demande DEVRAIT être implémentée en utilisant la méthode POST plutôt que PUT.

Si le serveur d'origine ne parvient pas à effectuer le changement d'état PUT demandé en la ressource cible et souhaite plutôt qu'elle soit appliquée à un ressource différente, par exemple lorsque la ressource a été déplacée vers un URI différent, alors le serveur d'origine DOIT envoyer un 3xx approprié Réponse (de redirection) ; l'agent utilisateur PEUT alors prendre sa propre décision concernant la nécessité ou non de rediriger la demande.

Une requête PUT appliquée à la ressource cible peut avoir des effets secondaires sur d'autres ressources. Par exemple, un article peut avoir un URI pour identifier « la version actuelle » (une ressource) qui est distincte de les URI identifiant chaque version particulière (différentes ressources

Piste des normes Fielding & Reschke [Page 28]

[RFC 7231](#)

HTTP/1.1 Sémantique et contenu Juin 2014

qui à un moment donné partageait le même état que la version actuelle (ressource). Une requête PUT réussie sur l'URI « version actuelle » pourrait donc créer une nouvelle ressource de version en plus de modifier l'état de la ressource cible, et peut également provoquer des liens ajoutés entre les ressources liées.

Un serveur d'origine qui autorise PUT sur une ressource cible donnée DOIT envoyer une réponse 400 (Bad Request) à une requête PUT qui contient un Champ d'en-tête Content-Range ([Section 4.2 de \[RFC7233\]](#)), puisque le la charge utile est susceptible d'être un contenu partiel qui a été placé par erreur en tant que représentation complète. Des mises à jour partielles du contenu sont possibles en ciblant une ressource identifiée séparément avec un état qui chevauche une partie de la ressource la plus importante, ou en utilisant une méthode différente qui a été spécifiquement défini pour les mises à jour partielles (par exemple, la Méthode PATCH définie dans [[RFC5789](#)]).

Les réponses à la méthode PUT ne peuvent pas être mises en cache. Si une réponse PUT est réussie, la requête passe par un cache contenant une ou plusieurs réponses stockées pour l'URI de demande effective, ces réponses stockées seront invalidé (voir [la section 4.4 de \[RFC7234\]](#)).

[4.3.5](#) . SUPPRIMER

La méthode DELETE demande au serveur d'origine de supprimer le association entre la ressource cible et sa ressource actuelle fonctionnalité. En effet, cette méthode est similaire à la commande rm sous UNIX : il exprime une opération de suppression sur le mappage URI du serveur d'origine plutôt qu'une attente selon laquelle le serveur précédemment les informations associées soient supprimées.

Si la ressource cible possède une ou plusieurs représentations actuelles, elles pourrait ou non être détruit par le serveur d'origine, et le le stockage associé peut ou non être récupéré, selon entièrement sur la nature de la ressource et sa mise en œuvre par le serveur d'origine (qui sont hors du champ d'application de cette spécification).

De même, d'autres aspects de la mise en œuvre d'une ressource peuvent devoir être désactivé ou archivé à la suite d'une SUPPRESSION, comme une base de données ou connexions de passerelle. En général, on suppose que l'origine le serveur autorisera uniquement la SUPPRESSION sur les ressources pour lesquelles il dispose d'un mécanisme prescrit pour réaliser la suppression.

Relativement peu de ressources autorisent la méthode DELETE - son utilisation

principale
est destiné aux environnements de création à distance, où l'utilisateur dispose de
direction concernant son effet. Par exemple, une ressource qui a été précédemment créé à l'aide d'une requête PUT, ou identifié via le Champ d'en-tête d'emplacement après une réponse 201 (Créé) à un POST demande, pourrait permettre une demande DELETE correspondante pour annuler ces actions. De même, les implémentations d'agents utilisateurs personnalisés qui implémentent

Piste des normes Fielding & Reschke [Page 29]

[RFC 7231](#)

HTTP/1.1 Sémantique et contenu Juin 2014

une fonction de création, telle que les clients de contrôle de révision utilisant HTTP

pour les opérations à distance, peut utiliser DELETE en supposant que l'espace URI du serveur a été conçu pour correspondre à une version dépôt.

Si une méthode DELETE est appliquée avec succès, le serveur d'origine DEVRAIT envoyer un code d'état 202 (Accepté) si l'action est susceptible de réussir mais n'a pas encore été promulguée, un code d'état 204 (Aucun contenu) si le l'action a été mise en œuvre et aucune information supplémentaire ne doit être fournie,

ou un code d'état 200 (OK) si l'action a été exécutée et que Le message de réponse comprend une représentation décrivant l'état.

Une charge utile dans un message de demande DELETE n'a pas de sémantique définie ;

l'envoi d'un corps de charge utile sur une requête DELETE peut entraîner des erreurs existantes implémentations pour rejeter la demande.

Les réponses à la méthode DELETE ne peuvent pas être mises en cache. Si une réponse DELETE

la requête passe par un cache contenant une ou plusieurs réponses stockées pour l'URI de demande effective, ces réponses stockées seront invalidé (voir [la section 4.4 de \[RFC7234\]](#)).

[4.3.6](#) . CONNECT

La méthode CONNECT demande au destinataire d'établir un tunnel vers le serveur d'origine de destination identifié par la cible de la requête et, en cas de succès, restreindre ensuite son comportement à la transmission aveugle

de paquets, dans les deux sens, jusqu'à la fermeture du tunnel. Tunnels sont couramment utilisés pour créer une connexion virtuelle de bout en bout, via

un ou plusieurs proxys, qui peuvent ensuite être sécurisés à l'aide de TLS (Transport Sécurité de la couche, [[RFC5246](#)]).

CONNECT est destiné uniquement à être utilisé dans les requêtes adressées à un proxy. Une origine

le serveur qui reçoit une requête CONNECT pour lui-même PEUT répondre avec un Code d'état 2xx (réussi) pour indiquer qu'une connexion est établie établi. Cependant, la plupart des serveurs d'origine n'implémentent pas CONNECT.

Un client envoyant une demande CONNECT DOIT envoyer le formulaire d'autorisation de

cible de la requête ([section 5.3 de \[RFC7230\]](#)) ; c'est-à-dire la cible de la requête

se compose uniquement du nom d'hôte et du numéro de port du tunnel destination, séparés par deux points. Par exemple,

```
CONNECT server.example.com:80 HTTP/1.1  
Hôte : server.example.com:80
```

Le proxy destinataire peut établir un tunnel soit directement se connecter à la cible de la requête ou, si configuré pour en utiliser une autre proxy, en transmettant la requête CONNECT au prochain proxy entrant. Toute réponse 2xx (réussie) indique que l'expéditeur (et tous les

Piste des normes Fielding & Reschke [Page 30]

[RFC 7231](#)

HTTP/1.1 Sémantique et contenu Juin 2014

les proxys entrants) passeront en mode tunnel immédiatement après le ligne vide qui conclut la section d'en-tête de la réponse réussie ; les données reçues après cette ligne vide proviennent du serveur identifié par

la cible de la requête. Toute réponse autre qu'une réponse positive indique que le tunnel n'a pas encore été formé et que le la connexion reste régie par HTTP.

Un tunnel est fermé lorsqu'un intermédiaire de tunnel détecte que côté a fermé sa connexion : l'intermédiaire DOIT tenter d'envoyer toutes les données en suspens provenant du côté fermé vers l'autre côté, fermez les deux connexions, puis supprimez toutes les données restantes laissé non livré.

L'authentification par proxy peut être utilisée pour établir l'autorité de créer un tunnel. Par exemple,

```
CONNECT server.example.com:80 HTTP/1.1
Hôte : server.example.com:80
Autorisation de proxy : aGVsbG86d29ybGQ de base =
```

Il y a des risques importants à établir un tunnel vers des décisions arbitraires.

serveurs, en particulier lorsque la destination est un serveur connu ou Port TCP réservé, non destiné au trafic web. Par exemple : une CONNEXION à une cible de requête de « exemple.com:25 » suggérerait que le proxy se connecte au port réservé au trafic SMTP ; si autorisé, qui pourraient tromper le proxy pour qu'il relaie des courriers indésirables.

Les proxys qui

Le support CONNECT DEVRAIT restreindre son utilisation à un ensemble limité de ports ou une liste blanche configurable de cibles de requêtes sûres.

Un serveur NE DOIT PAS envoyer d'en-tête Transfer-Encoding ou Content-Length champs dans une réponse 2xx (réussie) à CONNECT. Un client DOIT ignorer tous les champs d'en-tête Content-Length ou Transfer-Encoding reçus dans une réponse réussie à CONNECT.

Une charge utile dans un message de demande CONNECT n'a pas de sémantique définie ;

l'envoi d'un corps de charge utile sur une requête CONNECT peut entraîner des erreurs existantes implémentations pour rejeter la demande.

Les réponses à la méthode CONNECT ne peuvent pas être mises en cache.

[4.3.7](#) . OPTIONS

La méthode OPTIONS demande des informations sur la communication options disponibles pour la ressource cible, soit à l'origine serveur ou un intermédiaire. Cette méthode permet à un client pour déterminer les options et/ou les exigences associées à un ressource, ou les capacités d'un serveur, sans impliquer une

action sur les ressources.

Piste des normes Fielding & Reschke [Page 31]

[RFC 7231](#)

HTTP/1.1 Sémantique et contenu Juin 2014

Une requête OPTIONS avec un astérisque (« * ») comme cible de la requête ([La section 5.3 de \[RFC7230\]](#)) s'applique au serveur en général plutôt qu'au serveur d'un serveur dépendent généralement de la ressource, la requête « * » n'est utile qu'en tant que méthode de type « ping » ou « no-op » ; elle ne fait rien d'autre que permettre au client pour tester les capacités du serveur. Par exemple, cela peut être utilisé pour tester un proxy pour la conformité HTTP/1.1 (ou son absence).

Si la cible de la requête n'est pas un astérisque, la requête OPTIONS s'applique aux options disponibles lors de la communication avec la cible ressource.

Un serveur générant une réponse réussie à OPTIONS DEVRAIT envoyer n'importe quel champs d'en-tête qui pourraient indiquer des fonctionnalités facultatives implémentées par le serveur et applicable à la ressource cible (par exemple, Autoriser), y compris les extensions potentielles non définies par la présente spécification. La charge utile de réponse, le cas échéant, peut également décrire la communication options dans une représentation lisible par machine ou par l'homme. Une norme le format d'une telle représentation n'est pas défini par ce spécification, mais pourrait être définie par de futures extensions de HTTP. le serveur DOIT générer un champ Content-Length avec une valeur de « 0 » si aucun le corps de la charge utile doit être envoyé dans la réponse.

Un client PEUT envoyer un champ d'en-tête Max-Forwards dans une requête OPTIONS pour cibler un destinataire spécifique dans la chaîne de requête (voir [Section 5.1.2](#)). Un proxy NE DOIT PAS générer d'en-tête Max-Forwards. champ lors de la transmission d'une demande, sauf si cette demande a été reçue avec un champ Max-Forwards.

Un client qui génère une requête OPTIONS contenant un corps de charge utile DOIT envoyer un champ d'en-tête Content-Type valide décrivant le type de support de représentation. Bien que cette spécification ne définir une utilisation pour une telle charge utile, les futures extensions de HTTP pourraient utilisez le corps OPTIONS pour effectuer des requêtes plus détaillées sur la cible ressource.

Les réponses à la méthode OPTIONS ne peuvent pas être mises en cache.

[4.3.8](#) . TRACE

La méthode TRACE demande une boucle de retour à distance au niveau de l'application le message de requête. Le destinataire final de la requête DEVRAIT refléter le message reçu, à l'exclusion de certains champs décrits ci-dessous, de retour au client sous forme de corps de message d'une réponse 200 (OK) avec un Type de contenu de « message/http » ([section 8.3.1 de \[RFC7230\]](#)). Le destinataire final est soit le serveur d'origine, soit le premier serveur à recevoir une valeur Max-Forwards de zéro (0) dans la requête ([Section 5.1.2](#)).

Piste des normes Fielding & Reschke [Page 32]

[RFC 7231](#)

HTTP/1.1 Sémantique et contenu Juin 2014

Un client NE DOIT PAS générer de champs d'en-tête dans une requête TRACE contenant des données sensibles qui pourraient être divulguées par la réponse.

Par exemple, il serait stupide pour un agent utilisateur d'envoyer des informations utilisateur stockées informations d'identification [[RFC7235](#)] ou cookies [[RFC6265](#)] dans une requête TRACE.

le destinataire final de la demande DOIT exclure tout en-tête de demande champs susceptibles de contenir des données sensibles lorsque ce destinataire génère le corps de la réponse.

TRACE permet au client de voir ce qui est reçu à l'autre fin de la chaîne de requête et utiliser ces données à des fins de test ou de diagnostic

information. La valeur du champ d'en-tête Via ([Section 5.7.1 de \[RFC7230\]](#)) présente un intérêt particulier, car il agit comme une trace de la

chaîne de requêtes. L'utilisation du champ d'en-tête Max-Forwards permet client pour limiter la longueur de la chaîne de requête, ce qui est utile pour

tester une chaîne de proxys transmettant des messages dans une boucle infinie.

Un client NE DOIT PAS envoyer de corps de message dans une requête TRACE.

Les réponses à la méthode TRACE ne peuvent pas être mises en cache.

[5. Champs d'en-tête de demande](#)

Un client envoie des champs d'en-tête de requête pour fournir plus d'informations

concernant le contexte de la demande, rendre la demande conditionnelle en fonction du

état de la ressource cible, suggérer les formats préférés pour la réponse, fournir des informations d'authentification ou modifier la demande attendue traitement. Ces champs agissent comme des modificateurs de requête, similaires à

paramètres sur une invocation de méthode de langage de programmation.

[5.1 . Contrôles](#)

Les contrôles sont des champs d'en-tête de requête qui dirigent la gestion spécifique des la demande.

+-----+-----+	
Nom du champ d'en-tête	Défini dans...
+-----+-----+	
Cache-Control	Section 5.2 de [RFC7234]
Attendre	Section 5.1.1
Hôte	Section 5.4 de [RFC7230]
Max-Forwards	Section 5.1.2
Pragma	Section 5.4 de [RFC7234]

| Plage | [Section 3.1 de \[RFC7233\]](#). |
| TE | [Section 4.3 de \[RFC7230\]](#). |
+-----+-----+

Piste des normes Fielding & Reschke [Page 33]

[5.1.1](#) . Attendez-vous à

Le champ d'en-tête « Expect » dans une requête indique un certain ensemble de comportements (attentes) qui doivent être pris en charge par le serveur dans afin de traiter correctement cette demande. La seule attente de ce type défini par cette spécification est 100-continu.

Attendre = "100-continuer"

La valeur du champ Expect n'est pas sensible à la casse.

Un serveur qui reçoit une valeur de champ Expect autre que 100-continue PEUT répondre avec un code d'état 417 (échec de l'attente) pour indiquer que l'attente inattendue ne peut être satisfaite.

Une attente de 100 continus informe les destinataires que le client est sur le point d'envoyer un corps de message (probablement volumineux) dans cette requête et souhaite recevoir une réponse provisoire de 100 (Continuer) si le Les champs de ligne de demande et d'en-tête ne sont pas suffisants pour provoquer une succès immédiat, redirection ou réponse d'erreur. Cela permet à le client doit attendre une indication qu'il vaut la peine d'envoyer le corps du message avant de le faire réellement, ce qui peut améliorer l'efficacité lorsque le corps du message est volumineux ou lorsque le client anticipe qu'un une erreur est probable (par exemple, lors de l'envoi d'une méthode de changement d'état, pour le (première fois, sans informations d'authentification préalablement vérifiées).

Par exemple, une demande qui commence par

```
PUT /quelque part/amusant HTTP/1.1
Hôte : origin.example.com
Type de contenu : vidéo/h264
Contenu-Longueur : 1234567890987
Attendez-vous à : 100-continuer
```

permet au serveur d'origine de répondre immédiatement avec une erreur message, tel que 401 (non autorisé) ou 405 (méthode non autorisée), avant que le client ne commence à remplir les tuyaux avec des données inutiles transfert.

Exigences pour les clients :

- o Un client NE DOIT PAS générer une attente de 100 continues dans une demande qui n'inclut pas de corps de message.
- o Un client qui attendra une réponse 100 (Continuer) avant

l'envoi du corps du message de demande DOIT envoyer un champ d'en-tête
Expect
contenant une espérance de 100 continue.

Piste des normes Fielding & Reschke [Page 34]

[RFC 7231](#)

HTTP/1.1 Sémantique et contenu Juin 2014

- o Un client qui envoie une attente de 100 continus n'est pas obligé de attendre une durée spécifique ; un tel client PEUT procéder à envoyer le corps du message même s'il n'a pas encore reçu de réponse. De plus, étant donné que 100 réponses (Continuer) ne peuvent pas être envoyées via un intermédiaire HTTP/1.0, un tel client NE DEVRAIT PAS attendre une période indéterminée avant l'envoi du corps du message.
- o Un client qui reçoit un code d'état 417 (échec d'attente) dans la réponse à une demande contenant une attente de 100 continus DEVRAIT répéter cette demande sans s'attendre à 100%, puisque la réponse 417 indique simplement que la chaîne de réponse ne prendra en charge les attentes (par exemple, il passe par un serveur HTTP/1.0).

Exigences pour les serveurs :

- o Un serveur qui reçoit une attente de 100 continues dans un HTTP/1.0 la demande DOIT ignorer cette attente.
- o Un serveur PEUT omettre d'envoyer une réponse 100 (Continuer) s'il a déjà reçu une partie ou la totalité du corps du message pour la demande correspondante, ou si le cadrage indique qu'il y a pas de corps de message.
- o Un serveur qui envoie une réponse 100 (Continuer) DOIT finalement envoyer un code d'état final, une fois le corps du message reçu et traité, sauf si la connexion est fermée prématurément.
- o Un serveur qui répond avec un code d'état final avant de lire le corps du message entier DEVRAIT indiquer dans cette réponse s'il a l'intention de fermer la connexion ou de continuer à lire et de supprimer le message de demande (voir [la section 6.6 de \[RFC7230\]](#)).

Un serveur d'origine DOIT, lors de la réception d'un HTTP/1.1 (ou version ultérieure) ligne de demande et une section d'en-tête complète qui contient un 100-continue expectative et indique qu'un corps de message de demande sera suivre, soit envoyer une réponse immédiate avec un code d'état final, si ce statut peut être déterminé en examinant uniquement la ligne de demande et les champs d'en-tête, ou envoyez une réponse immédiate 100 (Continuer) à encourager le client à envoyer le corps du message de la requête. L'origine le serveur NE DOIT PAS attendre le corps du message avant d'envoyer le 100 (Continuer) réponse.

Un proxy DOIT, lors de la réception d'une ligne de requête HTTP/1.1 (ou ultérieure) et une section d'en-tête complète qui contient une attente de 100 continues et indique qu'un corps de message de demande suivra, soit envoyer une réponse immédiate avec un code d'état final, si cet état peut être déterminé en examinant uniquement les champs de ligne de demande et d'en-tête, ou

commencer à transmettre la demande au serveur d'origine en envoyant un

Piste des normes Fielding & Reschke [Page 35]

[RFC 7231](#)

HTTP/1.1 Sémantique et contenu Juin 2014

ligne de demande et section d'en-tête correspondantes à la prochaine entrée serveur. Si le proxy croit (d'après la configuration ou des erreurs passées) interaction) que le prochain serveur entrant ne prend en charge que HTTP/1.0, le

- le proxy PEUT générer une réponse immédiate de 100 (Continuer) pour encourager
- le client pour commencer à envoyer le corps du message.

Remarque : le champ d'en-tête Expect a été ajouté après l'original publication de HTTP/1.1 [[RFC2068](#)] comme moyen de demander un réponse provisoire 100 (Continuer) et le mécanisme général pour indiquant les extensions indispensables. Cependant, l'extension mécanisme n'a pas été utilisé par les clients et doit être compris les exigences n'ont pas été implémentées par de nombreux serveurs, ce qui rend

le mécanisme d'extension est inutile. Cette spécification a supprimé le mécanisme d'extension afin de simplifier la définition et traitement de 100-continu.

[5.1.2](#) . Max-Forwards

Le champ d'en-tête « Max-Forwards » fournit un mécanisme avec le TRACE ([Section 4.3.8](#)) et OPTIONS ([Section 4.3.7](#)) demandent des méthodes pour limiter

le nombre de fois que la requête est transmise par des proxys. Ceci peut être utile lorsque le client tente de tracer une demande qui semble échouer ou boucler au milieu de la chaîne.

Max-Forwards = 1*DIGIT

La valeur Max-Forwards est un entier décimal indiquant le reste nombre de fois que ce message de demande peut être transmis.

Chaque intermédiaire qui reçoit une demande TRACE ou OPTIONS contenant un champ d'en-tête Max-Forwards DOIT vérifier et mettre à jour sa valeur avant

transmettre la requête. Si la valeur reçue est zéro (0), le l'intermédiaire NE DOIT PAS transmettre la demande ; à la place, l'intermédiaire DOIT répondre en tant que destinataire final. Si le Max-Forwards reçu la valeur est supérieure à zéro, l'intermédiaire DOIT générer une mise à jour Champ Max-Forwards dans le message transféré avec une valeur de champ qui est le moindre de a) la valeur reçue diminuée d'un (1) ou b) la valeur maximale prise en charge par le destinataire pour Max-Forwards.

Un destinataire PEUT ignorer un champ d'en-tête Max-Forwards reçu avec n'importe quel autres méthodes de demande.

[5.2](#) . Conditionnels

Les champs d'en-tête de requête conditionnelle HTTP [[RFC7232](#)] permettent à un client

pour placer une condition préalable sur l'état de la ressource cible, de sorte que l'action correspondant à la sémantique de la méthode ne sera pas appliquée si la précondition est fausse. Chaque précondition définie par

Piste des normes Fielding & Reschke [Page 36]

[RFC 7231](#)

HTTP/1.1 Sémantique et contenu Juin 2014

cette spécification consiste en une comparaison entre un ensemble de validateurs obtenus à partir de représentations antérieures de la ressource cible

à l'état actuel des validateurs pour la représentation sélectionnée ([Section 7.2](#)). Par conséquent, ces conditions préalables évaluent si l'état

de la ressource cible a changé depuis un état donné connu par le client. L'effet d'une telle évaluation dépend de la méthode sémantique et choix du conditionnel, tel que défini dans [la section 5 de \[RFC7232\]](#) .

```
+-----+-----+
| Nom du champ d'en-tête | Défini dans... |
+-----+-----+
| If-Match | Section 3.1 de \[RFC7232\] |
| If-None-Match | Section 3.2 de \[RFC7232\] |
| Si-Modifié-Depuis | Section 3.3 de \[RFC7232\] |
| Si-Non-Modifié-Depuis | Section 3.4 de \[RFC7232\] |
| If-Range | Section 3.2 de \[RFC7233\] |
+-----+-----+
```

[5.3](#) . Négociation du contenu

Les champs d'en-tête de requête suivants sont envoyés par un agent utilisateur à

s'engager dans une négociation proactive du contenu de la réponse, tel que défini

dans [la section 3.4.1](#) . Les préférences envoyées dans ces champs s'appliquent à tout

contenu de la réponse, y compris les représentations de la cible ressource, représentations d'erreur ou d'état de traitement, et potentiellement même les diverses chaînes de texte qui pourraient apparaître dans le cadre du protocole.

```
+-----+-----+
| Nom du champ d'en-tête | Défini dans... |
+-----+-----+
| Accepter | Section 5.3.2 |
| Accept-Charset | Section 5.3.3 |
| Accepter-Encodage | Section 5.3.4 |
| Accept-Language | Section 5.3.5 |
+-----+-----+
```

[5.3.1](#) . Valeurs de qualité

De nombreux champs d'en-tête de demande pour la négociation proactive utilisent un

paramètre commun, nommé « q » (insensible à la casse), pour attribuer un « poids » à la préférence pour ce type de contenu associé. Ceci

le poids est appelé « valeur de qualité » (ou « qvalue ») car le

le même nom de paramètre est souvent utilisé dans les configurations de serveur pour

attribuer un poids à la qualité relative des différents

représentations qui peuvent être sélectionnées pour une ressource.

Piste des normes Fielding & Reschke [Page 37]

[RFC 7231](#)

HTTP/1.1 Sémantique et contenu Juin 2014

Le poids est normalisé à un nombre réel compris entre 0 et 1, où 0,001 est le moins préféré et 1 est le plus préféré ;
 La valeur 0 signifie « non acceptable ». Si le paramètre « q » n'est pas présent,
 le poids par défaut est 1.

```
poids = OWS ";" OWS "q=" qvalue
qvalue = ( "0" [ "." 0*3DIGIT ] )
         / ( "1" [ "." 0*3("0") ] )
```

Un expéditeur de qvalue NE DOIT PAS générer plus de trois chiffres après le point décimal. La configuration utilisateur de ces valeurs doit être limitée de la même manière.

5.3.2 . Accept

Le champ d'en-tête « Accepter » peut être utilisé par les agents utilisateurs pour spécifier types de supports de réponse acceptables. Les champs d'en-tête acceptés peuvent être utilisés pour indiquer que la demande est spécifiquement limitée à un petit ensemble de types souhaités, comme dans le cas d'une demande de image en ligne.

```
Accepter = #( plage-média [ paramètres-accept ] )

plage-média = ( "*/*"
                / ( taper "/" "*" )
                / ( type "/" sous-type )
                ) *( OWS ";" paramètre OWS )
accept-params = poids *( accept-ext )
accept-ext = OWS ";" jeton OWS [ "=" ( jeton / chaîne entre guillemets ) ]
```

Le caractère astérisque « * » est utilisé pour regrouper les types de médias en plages, avec "*/*" indiquant tous les types de médias et "type/*" indiquant tous sous-types de ce type. La plage de supports peut inclure le type de support paramètres applicables à cette plage.

Chaque plage de supports peut être suivie de zéro ou plusieurs supports applicables paramètres de type (par exemple, jeu de caractères), un paramètre « q » facultatif pour indiquant un poids relatif ([Section 5.3.1](#)), puis zéro ou plus paramètres d'extension. Le paramètre « q » est nécessaire si les extensions (accept-ext) sont présentes, car elles agissent comme un séparateur entre les deux ensembles de paramètres.

Remarque : utilisation du nom de paramètre « q » pour séparer le type de média les paramètres de l'extension Accept sont dus à des facteurs historiques pratique. Bien que cela empêche tout paramètre de type de média nommé

« q » étant utilisé avec une plage de médias, un tel événement est considéré
peu probable étant donné l'absence de paramètres « q » dans l'IANA

Piste des normes Fielding & Reschke [Page 38]

[RFC 7231](#)

HTTP/1.1 Sémantique et contenu Juin 2014

registre des types de médias et utilisation rare de tout type de média paramètres dans Accept. Les futurs types de médias sont déconseillés enregistrement de tout paramètre nommé « q ».

L'exemple

```
Accepter : audio/* ; q=0,2, audio/basic
```

est interprété comme « Je préfère l'audio/basique, mais envoyez-moi n'importe quel type d'audio "si c'est le meilleur disponible après une réduction de 80 % sur la qualité".

Une requête sans champ d'en-tête Accept implique que l'agent utilisateur acceptera tout type de média en réponse. Si le champ d'en-tête est présent dans une demande et aucune des représentations disponibles pour la réponse a un type de média répertorié comme acceptable, le serveur d'origine peut honorer le champ d'en-tête en envoyant une erreur 406 (non Réponse acceptable) ou ignorer le champ d'en-tête en traitant le réponse comme si elle n'était pas soumise à une négociation de contenu.

Un exemple plus élaboré est

```
Accepter : texte/plain ; q=0,5, texte/html,  
          texte/x-dvi ; q=0,8, texte/xc
```

Verbalement, cela serait interprété comme « text/html et text/xc sont les types de médias également préférés, mais s'ils n'existent pas, alors envoyer la représentation texte/x-dvi, et si elle n'existe pas, envoyer le texte/la représentation simple".

Les plages de médias peuvent être remplacées par des plages de médias plus spécifiques ou types de supports spécifiques. Si plusieurs gammes de supports s'appliquent à un

Pour un type donné, la référence la plus spécifique prime. Par exemple :

```
Accepter : texte/*, texte/plain, texte/plain ; format = flowed, */*
```

ont la priorité suivante :

1. texte/plain;format=flowed
2. texte/brut
3. texte/*
4. */*

Le facteur de qualité du type de support associé à un type donné est déterminé en trouvant la plage de médias avec la priorité la plus élevée qui correspond au type. Par exemple,

Piste des normes Fielding & Reschke [Page 39]

[RFC 7231](#)

HTTP/1.1 Sémantique et contenu Juin 2014

Accepter : texte/* ; q=0,3, texte/html ; q=0,7, texte/html ; niveau=1, texte/html;niveau=2;q=0.4, */*;q=0.5

entraînerait l'association des valeurs suivantes :

+-----+-----+		
Type de média	Qualité	Valeur
+-----+-----+		
texte/html;niveau=1	1	
texte/html	0.7	
texte/brut	0,3	
image/jpeg	0,5	
texte/html;niveau=2	0.4	
texte/html;niveau=3	0,7	
+-----+-----+		

Remarque : un agent utilisateur peut être doté d'un ensemble de paramètres de qualité par défaut.

valeurs pour certaines plages de médias. Cependant, à moins que l'agent utilisateur ne soit un système fermé qui ne peut pas interagir avec d'autres agents de rendu, ce L'ensemble par défaut doit être configurable par l'utilisateur.

[5.3.3](#) . Accept-Charset

Le champ d'en-tête « Accept-Charset » peut être envoyé par un agent utilisateur à

indiquer quels jeux de caractères sont acceptables dans le contenu des réponses textuelles.

Ce champ permet aux agents utilisateurs capables de comprendre davantage jeux de caractères complets ou à usage spécial pour signaler cette capacité à un serveur d'origine capable de représenter des informations dans ces jeux de caractères.

Accept-Charset = 1#((charset / "*") [poids])

Les noms de jeux de caractères sont définis dans [la section 3.1.1.2](#) . Un agent utilisateur PEUT

associer une valeur de qualité à chaque jeu de caractères pour indiquer l'utilisateur

préférence relative pour ce jeu de caractères, comme défini dans [la section 5.3.1](#) .

Un exemple est

Accept-Charset : iso-8859-5, unicode-1-1 ; q = 0,8

La valeur spéciale « * », si elle est présente dans le champ Accept-Charset, correspond à tous les jeux de caractères qui ne sont pas mentionnés ailleurs dans le

Champ Accept-Charset. Si aucun « * » n'est présent dans un champ Accept-Charset

champ, alors tous les jeux de caractères non explicitement mentionnés dans le champ sont

considéré comme « inacceptable » par le client.

Une requête sans champ d'en-tête Accept-Charset implique que le

L'agent utilisateur acceptera n'importe quel jeu de caractères en réponse.
Usage le plus général

les agents utilisateurs n'envoient pas Accept-Charset, sauf indication
contraire

Piste des normes Fielding & Reschke [Page 40]

configuré pour le faire, car une liste détaillée des jeux de caractères pris en charge permet à un serveur d'identifier plus facilement un individu en vertu de les caractéristiques de la requête de l'agent utilisateur ([Section 9.7](#)).

Si un champ d'en-tête Accept-Charset est présent dans une requête et qu'aucun d'entre eux les représentations disponibles pour la réponse ont un jeu de caractères qui est

répertorié comme acceptable, le serveur d'origine peut soit honorer l'en-tête champ, en envoyant une réponse 406 (Non acceptable), ou ignorer le champ d'en-tête en traitant la ressource comme si elle n'était pas soumise à négociation de contenu.

[5.3.4](#) . Accept-Encoding

Le champ d'en-tête « Accept-Encoding » peut être utilisé par les agents utilisateurs pour indiquer quels sont les codages de contenu de réponse ([section 3.1.2.1](#)) acceptable dans la réponse. Un jeton « identité » est utilisé comme synonyme pour « pas d'encodage » afin de communiquer lorsqu'aucun encodage n'est préféré.

```
Accept-Encoding = #( codages [ poids ] )
codages = codage de contenu / "identité" / "*"
```

Chaque valeur de codage PEUT se voir attribuer une valeur de qualité associée représentant la préférence pour cet encodage, tel que défini dans [Section 5.3.1](#) . L'astérisque « * » dans un champ Accept-Encoding correspond à tout codage de contenu disponible non explicitement répertorié dans le champ d'en-tête.

Par exemple,

```
Accepter-Encodage : compresser, gzip
Accepter-Encodage :
Accepter-Encodage : *
Accepter-Encodage : compresser ; q = 0,5, gzip ; q = 1,0
Accepter-Encodage : gzip ; q = 1,0, identité ; q = 0,5, * ; q = 0
```

Une requête sans champ d'en-tête Accept-Encoding implique que le L'agent utilisateur n'a aucune préférence concernant le codage du contenu. cela permet au serveur d'utiliser n'importe quel codage de contenu dans une réponse, n'implique pas que l'agent utilisateur sera en mesure de traiter correctement tous les encodages.

Un serveur teste si un codage de contenu pour une représentation donnée est acceptable en utilisant ces règles :

1. Si aucun champ Accept-Encoding n'est présent dans la requête, tout codage de contenu

est considéré comme acceptable par l'agent utilisateur.

Piste des normes Fielding & Reschke [Page 41]

[RFC 7231](#)

HTTP/1.1 Sémantique et contenu Juin 2014

2. Si la représentation n'a pas de codage de contenu, alors elle est acceptable par défaut, sauf exclusion spécifique par le Champ Accept-Encoding indiquant soit « identity;q=0 » soit « *;q=0 » sans entrée plus spécifique pour « identité ».
3. Si le codage du contenu de la représentation est l'un des les codages de contenu répertoriés dans le champ Accept-Encoding, alors il est acceptable à moins qu'il ne soit accompagné d'une valeur q de 0. (Comme défini dans [la section 5.3.1](#) , une valeur q de 0 signifie « non acceptable ».)
4. Si plusieurs codages de contenu sont acceptables, alors le contenu acceptable le codage de contenu avec la valeur q non nulle la plus élevée est préféré.

Un champ d'en-tête Accept-Encoding avec une valeur de champ combinée qui est vide implique que l'agent utilisateur ne veut aucun codage de contenu dans réponse. Si un champ d'en-tête Accept-Encoding est présent dans une requête et aucune des représentations disponibles pour la réponse n'a de codage de contenu répertorié comme acceptable, le serveur d'origine DEVRAIT envoyer une réponse sans aucun codage de contenu.

Remarque : la plupart des applications HTTP/1.0 ne reconnaissent pas ou ne respectent pas les qvalues associés aux codages de contenu. Cela signifie que les qvalues peuvent ne fonctionnent pas et ne sont pas autorisés avec x-gzip ou x-compress.

[5.3.5](#) . Accept-Language

Le champ d'en-tête « Accept-Language » peut être utilisé par les agents utilisateurs pour indiquer l'ensemble des langues naturelles qui sont préférées dans le réponse. Les balises de langue sont définies dans [la section 3.1.3.1](#) .

Accept-Language = 1#(plage-de-langues [poids])
 plage de langues =
 <plage de langues, voir [\[RFC4647\]](#), [Section 2.1](#) >

Chaque gamme de langues peut se voir attribuer une valeur de qualité associée représentant une estimation de la préférence de l'utilisateur pour les langues spécifié par cette plage, comme défini à [la section 5.3.1](#) . Par exemple,

Accepter la langue : da, en-gb;q=0.8, en;q=0.7

signifierait : « Je préfère le danois, mais j'accepterai l'anglais britannique et autres types d'anglais ».

Une requête sans champ d'en-tête Accept-Language implique que le L'agent utilisateur acceptera n'importe quelle langue en réponse. Si le champ

d'en-tête

est présent dans une demande et aucune des représentations disponibles pour la réponse a une balise de langue correspondante, le serveur d'origine peut soit ignorer le champ d'en-tête en traitant la réponse comme si elle

Piste des normes Fielding & Reschke [Page 42]

[RFC 7231](#)

HTTP/1.1 Sémantique et contenu Juin 2014

n'est pas soumis à la négociation de contenu ou au respect du champ d'en-tête par envoi d'une réponse 406 (non acceptable). Cependant, cette dernière n'est pas encouragé, car cela peut empêcher les utilisateurs d'accéder à du contenu qui ils pourraient être en mesure de l'utiliser (avec un logiciel de traduction, par exemple).

Notez que certains destinataires traitent l'ordre dans lequel les balises de langue sont répertorié comme une indication de priorité décroissante, en particulier pour les balises auxquels sont attribuées des valeurs de qualité égales (aucune valeur n'est identique à q=1). Cependant, on ne peut pas se fier à ce comportement. Pour des raisons de cohérence et de maximiser l'interopérabilité, de nombreux agents utilisateurs attribuent à chaque balise de langue une valeur de qualité unique tout en les listant par ordre décroissant qualité. Une discussion supplémentaire sur les listes de priorités linguistiques peut être trouvé dans [la section 2.3 de \[RFC4647\]](#) .

Pour la correspondance, [la section 3 de \[RFC4647\]](#) définit plusieurs correspondances schémas. Les implémentations peuvent offrir la correspondance la plus appropriée schéma adapté à leurs besoins. Le schéma « Filtrage de base » ([\[RFC4647\]](#), [Section 3.3.1](#)) est identique au schéma de correspondance qui a été précédemment défini pour HTTP dans [la section 14.4 de \[RFC2616\]](#) .

Il pourrait être contraire aux attentes de confidentialité de l'utilisateur d'envoyer un champ d'en-tête Accept-Language avec la langue complète préférences de l'utilisateur dans chaque requête ([Section 9.7](#)).

L'intelligibilité dépendant fortement de l'utilisateur individuel, les agents utilisateurs doivent permettre à l'utilisateur de contrôler ses préférences linguistiques (soit par la configuration de l'agent utilisateur lui-même, soit par (par défaut, un paramètre système contrôlable par l'utilisateur). Un agent utilisateur qui ne fournit pas un tel contrôle à l'utilisateur NE DOIT PAS envoyer un Champ d'en-tête Accept-Language.

Remarque : les agents utilisateurs doivent fournir des conseils aux utilisateurs lors de la configuration une préférence, car les utilisateurs sont rarement familiers avec les détails de correspondance linguistique comme décrit ci-dessus. Par exemple, les utilisateurs pourraient supposent qu'en sélectionnant « en-gb », ils recevront n'importe quel type de Document en anglais si l'anglais britannique n'est pas disponible. Un

utilisateur

L'agent pourrait suggérer, dans un tel cas, d'ajouter « en » à la liste pour meilleur comportement de correspondance.

Piste des normes Fielding & Reschke [Page 43]

5.4 . Informations d'authentification

Deux champs d'en-tête sont utilisés pour transporter les informations d'authentification, tel que défini dans [[RFC7235](#)]. Notez que divers mécanismes personnalisés pour l'authentification de l'utilisateur utilise le champ d'en-tête Cookie à cette fin, comme défini dans [[RFC6265](#)].

```
+-----+-----+
| Nom du champ d'en-tête | Défini dans... |
+-----+-----+
| Autorisation | Section 4.2 de \[RFC7235\] |
| Autorisation de proxy | Section 4.4 de \[RFC7235\] |
+-----+-----+
```

5.5 . Contexte de la demande

Les champs d'en-tête de demande suivants fournissent des informations supplémentaires sur le contexte de la demande, y compris les informations sur l'utilisateur, l'utilisateur agent et ressource derrière la demande.

```
+-----+-----+
| Nom du champ d'en-tête | Défini dans... |
+-----+-----+
| De | Section 5.5.1 |
| Référent | Section 5.5.2 |
| Agent utilisateur | Section 5.5.3 |
+-----+-----+
```

5.5.1 . De

Le champ d'en-tête « De » contient une adresse e-mail Internet pour un utilisateur humain contrôlant l'agent utilisateur demandeur. L'adresse doit être utilisable par machine, tel que défini par « boîte aux lettres » dans [la section 3.4 de \[RFC5322\]](#) :

De = boîte aux lettres

boîte aux lettres = <boîte aux lettres, voir [\[RFC5322\]](#), [Section 3.4](#) >

Un exemple est :

De : webmaster@example.org

Le champ d'en-tête De est rarement envoyé par des agents utilisateurs non robotiques.

l'agent utilisateur NE DOIT PAS envoyer un champ d'en-tête De sans configuration par l'utilisateur, car cela pourrait entrer en conflit avec les

intérêts en matière de confidentialité ou de politique de sécurité de leur site.

Piste des normes Fielding & Reschke [Page 44]

[RFC 7231](#)

HTTP/1.1 Sémantique et contenu Juin 2014

Un agent utilisateur robotique DEVRAIT envoyer un champ d'en-tête De valide afin que

la personne responsable de l'exécution du robot peut être contactée si des problèmes surviennent sur les serveurs, par exemple si le robot envoie des messages excessifs, demandes indésirables ou invalides.

Un serveur NE DOIT PAS utiliser le champ d'en-tête De pour le contrôle d'accès ou

authentification, car la plupart des destinataires supposeront que le champ la valeur est une information publique.

[5.5.2](#) . Référent

Le champ d'en-tête « Referer » [sic] permet à l'agent utilisateur de spécifier un

Référence URI pour la ressource à partir de laquelle l'URI cible a été obtenue

(c'est-à-dire le « référent », bien que le nom du champ soit mal orthographié). Un utilisateur

l'agent NE DOIT PAS inclure les composants fragment et userinfo du Référence URI [[RFC3986](#)], le cas échéant, lors de la génération du champ Referer valeur.

Référent = URI-absolu / URI-partiel

Le champ d'en-tête Referer permet aux serveurs de générer des liens retour vers

autres ressources pour des analyses simples, la journalisation, la mise en cache optimisée,

etc. Il permet également de trouver des liens obsolètes ou mal orthographiés pour

maintenance. Certains serveurs utilisent le champ d'en-tête Referer comme moyen de

refuser les liens provenant d'autres sites (appelés « liens profonds ») ou restreindre la falsification de requêtes intersites (CSRF), mais pas toutes les requêtes

le contenir.

Exemple:

Référent : <http://www.example.org/hypertext/Overview.html>

Si l'URI cible a été obtenue à partir d'une source qui n'a pas son propre URI (par exemple, une entrée à partir du clavier de l'utilisateur ou une entrée dans le

signets/favoris de l'utilisateur), l'agent utilisateur DOIT soit exclure le Champ de référence ou envoyez-le avec une valeur de « about:blank ».

Le champ Référent a le potentiel de révéler des informations sur le contexte de demande ou historique de navigation de l'utilisateur, ce qui est une confidentialité

préoccupation si l'identifiant de la ressource référente révèle des informations personnelles des informations (comme un nom de compte) ou une ressource censée être confidentiel (comme derrière un pare-feu ou à l'intérieur d'un service sécurisé). La plupart des agents utilisateurs à usage général n'envoient pas le Champ d'en-tête de référence lorsque la ressource référente est un « fichier » local ou URI « data ». Un agent utilisateur NE DOIT PAS envoyer un champ d'en-tête Referer dans une requête HTTP non sécurisée si la page de référence a été reçue avec un Protocole sécurisé. Voir [la section 9.4](#) pour plus de sécurité. considérations.

Piste des normes Fielding & Reschke [Page 45]

[RFC 7231](#)

HTTP/1.1 Sémantique et contenu Juin 2014

Certains intermédiaires sont connus pour supprimer sans discernement
 Champs d'en-tête de référence des requêtes sortantes.
 effet secondaire malheureux de l'interférence avec la protection contre le
 CSRF
 attaques, qui peuvent être bien plus néfastes pour leurs utilisateurs.
 Intermédiaires et extensions d'agent utilisateur qui souhaitent limiter
 la divulgation d'informations dans Referer devrait limiter leurs
 modifications à
 modifications spécifiques, telles que le remplacement des noms de domaine
 internes par
 pseudonymes ou en tronquant les composants de la requête et/ou du chemin.
 l'intermédiaire NE DOIT PAS modifier ou supprimer le champ d'en-tête Referer
 lorsque la valeur du champ partage le même schéma et le même hôte que la
 demande
 cible.

[5.5.3](#) . Agent utilisateur

Le champ d'en-tête « User-Agent » contient des informations sur l'utilisateur
 agent à l'origine de la requête, qui est souvent utilisé par les serveurs
 pour aider
 identifier la portée des problèmes d'interopérabilité signalés, pour
 travailler
 contourner ou adapter les réponses pour éviter un agent utilisateur
 particulier
 limitations et pour les analyses concernant le navigateur ou le système
 d'exploitation
 utilisation. Un agent utilisateur DOIT envoyer un champ User-Agent dans
 chaque requête
 sauf configuration spécifique pour ne pas le faire.

User-Agent = produit *(RWS (produit / commentaire))

La valeur du champ User-Agent se compose d'un ou plusieurs produits
 identifiants, chacun suivi de zéro ou plusieurs commentaires ([Section 3.2 de](#)
[\[RFC7230\]](#)), qui identifient ensemble le logiciel de l'agent utilisateur et
 son
 sous-produits significatifs. Par convention, les identifiants de produits
 sont
 classés par ordre décroissant d'importance pour l'identification des
 logiciel agent utilisateur. Chaque identifiant de produit est composé d'un
 nom et
 version optionnelle.

produit = jeton ["/" version-produit]
 version-du-produit = jeton

Un expéditeur DEVRAIT limiter les identifiants de produit générés à ce qui
 est
 nécessaire pour identifier le produit ; un expéditeur NE DOIT PAS générer
 publicité ou autres informations non essentielles dans le produit
 identifiant. Un expéditeur NE DOIT PAS générer d'informations dans
 version du produit qui n'est pas un identifiant de version (c'est-à-dire

successif)

les versions du même nom de produit ne doivent différer que par la partie version du produit de l'identifiant du produit).

Exemple:

Agent utilisateur : CERN-LineMode/2.15 libwww/2.17b3

Piste des normes Fielding & Reschke [Page 46]

[RFC 7231](#)

HTTP/1.1 Sémantique et contenu Juin 2014

Un agent utilisateur NE DOIT PAS générer un champ User-Agent contenant des détails inutilement fins et DEVRAIT limiter l'ajout de sous-produits de tiers. Agent utilisateur trop long et détaillé. les valeurs de champ augmentent la latence des requêtes et le risque qu'un utilisateur soit identifiés contre leur gré (« empreintes digitales »).

De même, les implémentations sont encouragées à ne pas utiliser le produit jetons d'autres implémentations afin de déclarer la compatibilité avec eux, car cela contourne l'objectif du champ. Si un utilisateur l'agent se fait passer pour un agent utilisateur différent, les destinataires peuvent supposer que l'utilisateur désire intentionnellement voir des réponses adaptées à cet agent utilisateur identifié, même s'il ne fonctionne peut-être pas aussi bien pour l'agent utilisateur réel utilisé.

6. Codes d'état de réponse

L'élément de code d'état est un code entier à trois chiffres donnant le résultat de la tentative de comprendre et de satisfaire la demande.

Les codes d'état HTTP sont extensibles. Les clients HTTP ne sont pas tenus de comprendre la signification de tous les codes de statut enregistrés, bien que de tels La compréhension est évidemment souhaitable. Cependant, un client DOIT comprendre la classe de tout code d'état, comme indiqué par le premier chiffre et traiter un code d'état non reconnu comme étant équivalent à le code d'état x00 de cette classe, à l'exception d'un le destinataire NE DOIT PAS mettre en cache une réponse avec un code d'état non reconnu.

Par exemple, si un code d'état non reconnu de 471 est reçu par un client, le client peut supposer qu'il y avait quelque chose qui n'allait pas avec son demande et traiter la réponse comme si elle avait reçu un 400 (mauvais Code d'état de la demande. Le message de réponse contient généralement un représentation qui explique le statut.

Le premier chiffre du code d'état définit la classe de réponse. Les deux derniers chiffres n'ont aucun rôle de catégorisation. Il y a cinq valeurs pour le premier chiffre :

- o 1xx (Informationnel) : La demande a été reçue, le processus se poursuit
- o 2xx (Réussi) : La demande a été reçue avec succès, compris et accepté
- o 3xx (Redirection) : Des mesures supplémentaires doivent être prises afin de compléter la demande
- o 4xx (Erreur client) : la requête contient une syntaxe incorrecte ou ne peut pas être

accompli

Piste des normes Fielding & Reschke [Page 47]

[RFC 7231](#)

HTTP/1.1 Sémantique et contenu Juin 2014

o 5xx (Erreur du serveur) : Le serveur n'a pas réussi à exécuter une tâche apparemment demande valide

[6.1](#) . Aperçu des codes d'état

Les codes d'état répertoriés ci-dessous sont définis dans cette spécification,

[Section 4 de \[RFC7232\]](#) , [Section 4 de \[RFC7233\]](#) et [Section 3 de \[RFC7235\]](#) . Les phrases de raison listées ici ne sont que des recommandations.

-- ils peuvent être remplacés par des équivalents locaux sans affecter la protocole.

Réponses avec des codes d'état définis comme pouvant être mis en cache par défaut

(par exemple, 200, 203, 204, 206, 300, 301, 404, 405, 410, 414 et 501 dans cette spécification) peut être réutilisée par un cache avec une heuristique expiration sauf indication contraire dans la définition de la méthode ou contrôles de cache explicites [[RFC7234](#)] ; tous les autres codes d'état ne sont pas pouvant être mis en cache par défaut.

Piste des normes Fielding & Reschke [Page 48]

[RFC 7231](#)

HTTP/1.1 Sémantique et contenu Juin 2014

+-----+-----+-----+-----+-----+-----+			
Code	Phrase de raison	Défini dans...	
+-----+-----+-----+-----+-----+-----+			
100	Continuer	Section 6.2.1	
101	Protocoles de commutation	Section 6.2.2	
200	OK	Section 6.3.1	
201	Créé	Section 6.3.2	
202	Accepté	Section 6.3.3	
203	Informations non autorisées	Section 6.3.4	
204	Aucun contenu	Section 6.3.5	
205	Réinitialiser le contenu	Section 6.3.6	
206	Contenu partiel	Section 4.1 de [RFC7233]	
300	Choix multiples	Section 6.4.1	
301	Déplacé définitivement	Section 6.4.2	
302	Trouvé	Section 6.4.3	
303	Voir Autre	Section 6.4.4	
304	Non modifié	Section 4.1 de [RFC7232]	
305	Utiliser un proxy	Section 6.4.5	
307	Redirection temporaire	Section 6.4.7	
400	Mauvaise requête	Section 6.5.1	
401	Non autorisé	Section 3.1 de [RFC7235]	
402	Paieement requis	Section 6.5.2	
403	Interdit	Section 6.5.3	
404	Non trouvé	Section 6.5.4	
405	Méthode non autorisée	Section 6.5.5	
406	Non acceptable	Section 6.5.6	
407	Authentification proxy requise	Section 3.2 de [RFC7235]	
408	Délai d'expiration de la demande	Section 6.5.7	
409	Conflit	Section 6.5.8	
410	Disparu	Section 6.5.9	
411	Longueur requise	Section 6.5.10	
412	Échec de la condition préalable	Section 4.2 de [RFC7232]	
413	Charge utile trop importante	Section 6.5.11	
414	URI trop long	Section 6.5.12	
415	Type de média non pris en charge	Section 6.5.13	
416	Plage non satisfaisante	Section 4.4 de [RFC7233]	
417	Attente échouée	Section 6.5.14	
426	Mise à niveau requise	Section 6.5.15	
500	Erreur interne du serveur	Section 6.6.1	
501	Non implémenté	Section 6.6.2	
502	Mauvaise passerelle	Section 6.6.3	
503	Service indisponible	Section 6.6.4	
504	Délai d'expiration de la passerelle	Section 6.6.5	
505	Version HTTP non prise en charge	Section 6.6.6	
+-----+-----+-----+-----+-----+-----+			

[RFC 7231](#)

HTTP/1.1 Sémantique et contenu Juin 2014

Notez que cette liste n'est pas exhaustive - elle n'inclut pas codes d'état d'extension définis dans d'autres spécifications. L'intégralité La liste des codes d'état est gérée par l'IANA. Voir [la section 8.2](#) pour plus d'informations. détails.

[6.2](#) . Informationnel 1xx

La classe de code d'état 1xx (Informationnel) indique un état intermédiaire réponse pour communiquer l'état de la connexion ou la progression de la demande

avant de terminer l'action demandée et d'envoyer une réponse finale réponse. Les réponses 1xx sont terminées par la première ligne vide après la ligne d'état (la ligne vide signalant la fin de l'en-tête section). Étant donné que HTTP/1.0 ne définissait aucun code d'état 1xx, un le serveur NE DOIT PAS envoyer une réponse 1xx à un client HTTP/1.0.

Un client DOIT être capable d'analyser une ou plusieurs réponses 1xx reçues avant une réponse finale, même si le client n'en attend pas. l'agent utilisateur PEUT ignorer les réponses 1xx inattendues.

Un proxy DOIT transmettre les réponses 1xx à moins que le proxy lui-même ne l'ait demandé

la génération de la réponse 1xx. Par exemple, si un proxy ajoute un Champ « Attendre : 100-continuer » lorsqu'il transmet une demande, il doit alors ne pas transmettre les 100 réponses (Continuer) correspondantes.

[6.2.1](#) . 100 Continue

Le code d'état 100 (Continuer) indique que la partie initiale d'un la demande a été reçue et n'a pas encore été rejetée par le serveur. Le serveur a l'intention d'envoyer une réponse finale après le la demande a été entièrement reçue et traitée.

Lorsque la requête contient un champ d'en-tête Expect qui inclut un 100-continue expectative, la réponse 100 indique que le serveur souhaite recevoir le corps de la charge utile de la requête, comme décrit dans

[Section 5.1.1](#) . Le client doit continuer à envoyer la demande et rejeter la réponse 100.

Si la demande ne contenait pas de champ d'en-tête Expect contenant le Attente de 100 %, le client peut simplement abandonner cet intermédiaire réponse.

[6.2.2](#) . 101 Protocoles de commutation

Le code d'état 101 (Switching Protocols) indique que le serveur comprend et est disposé à se conformer à la demande du client, via le champ d'en-tête de mise à niveau ([section 6.7 de \[RFC7230\]](#)), pour un changement de le protocole d'application utilisé sur cette connexion. Le serveur

Piste des normes Fielding & Reschke [Page 50]

[RFC 7231](#)

HTTP/1.1 Sémantique et contenu Juin 2014

DOIT générer un champ d'en-tête de mise à niveau dans la réponse qui indique vers quel(s) protocole(s) seront basculés immédiatement après le vide ligne qui termine la réponse 101.

On suppose que le serveur acceptera uniquement de changer de protocole lorsque cela est avantageux. Par exemple, passer à une version plus récente la version de HTTP pourrait être avantageuse par rapport aux versions plus anciennes, et

le passage à un protocole synchrone en temps réel pourrait être avantageux lors de la livraison de ressources qui utilisent de telles fonctionnalités.

[6.3](#) . Réussite 2xx

La classe de code d'état 2xx (réussi) indique que le client la demande a été reçue, comprise et acceptée avec succès.

[6.3.1](#) . 200 OK

Le code d'état 200 (OK) indique que la demande a réussi.

La charge utile envoyée dans une réponse 200 dépend de la méthode de demande.

Pour les méthodes définies par la présente spécification, la signification prévue

de la charge utile peut être résumée comme suit :

OBTENIR une représentation de la ressource cible ;

HEAD la même représentation que GET, mais sans la représentation données ;

PUBLIER une représentation de l'état ou des résultats obtenus à partir de l'action ;

METTRE, SUPPRIMER une représentation de l'état de l'action ;

OPTIONS une représentation des options de communication ;

TRACE une représentation du message de demande tel que reçu par la fin serveur.

Outre les réponses à CONNECT, une réponse 200 a toujours une charge utile, bien qu'un serveur d'origine PEUT générer un corps de charge utile de longueur nulle.

Si aucune charge utile n'est souhaitée, un serveur d'origine doit envoyer 204 (Non

Contenu). Pour CONNECT, aucune charge utile n'est autorisée car résultat réussi est un tunnel, qui commence immédiatement après le 200 section d'en-tête de réponse.

Une réponse 200 peut être mise en cache par défaut, sauf indication contraire.

indiqué par la définition de la méthode ou les contrôles de cache explicites (voir

[Section 4.2.2 de \[RFC7234\]](#)).

Piste des normes Fielding & Reschke [Page 51]

[RFC 7231](#)

HTTP/1.1 Sémantique et contenu Juin 2014

6.3.2 . 201 Créé

Le code d'état 201 (Créé) indique que la demande a été accompli et a abouti à l'obtention d'une ou plusieurs nouvelles ressources créées. La ressource principale créée par la requête est identifiée soit par un champ d'en-tête d'emplacement dans la réponse, soit, s'il n'y a pas d'emplacement le champ est reçu, par l'URI de demande effective.

La charge utile de réponse 201 décrit et relie généralement à l'Ressource(s) créée(s). Voir [la section 7.2](#) pour une discussion sur la signification. et le but des champs d'en-tête du validateur, tels que ETag et Dernière modification, dans une réponse 201.

6.3.3 . 202 Accepté

Le code d'état 202 (Accepté) indique que la demande a été accepté pour traitement, mais le traitement n'a pas été terminé. La demande pourrait ou non être finalement prise en compte, comme elle pourrait l'être être refusée lorsque le traitement a effectivement lieu. Il n'y a pas fonctionnalité dans HTTP pour renvoyer un code d'état à partir d'un protocole asynchrone opération.

La réponse 202 est intentionnellement évasive. Son but est de permettre à un serveur d'accepter une demande pour un autre processus (peut-être un processus par lots exécuté une seule fois par jour) sans exigeant que la connexion de l'agent utilisateur au serveur persiste jusqu'à la fin du processus. La déclaration jointe à ce la réponse doit décrire l'état actuel de la demande et indiquer (ou intégrer) un moniteur d'état qui peut fournir à l'utilisateur un estimation du moment où la demande sera satisfaite.

6.3.4 . 203 Informations non autorisées

Le code d'état 203 (Information non autorisée) indique que la demande a réussi mais la charge utile jointe a été modifiée à partir de celle de la réponse 200 (OK) du serveur d'origine par une transformation proxy ([section 5.7.2 de \[RFC7230\]](#)). Ce code d'état permet à proxy pour notifier les destinataires lorsqu'une transformation a été appliquée, car cette connaissance pourrait avoir un impact sur les décisions ultérieures concernant contenu. Par exemple, les futures demandes de validation du cache pour le le contenu peut être applicable uniquement le long du même chemin de requête (via les mêmes procurations).

La réponse 203 est similaire au code d'avertissement de la transformation 214

Appliqué ([Section 5.5 de \[RFC7234\]](#)), qui présente l'avantage d'être applicable aux réponses avec n'importe quel code de statut.

Piste des normes Fielding & Reschke [Page 52]

[RFC 7231](#)

HTTP/1.1 Sémantique et contenu Juin 2014

Une réponse 203 peut être mise en cache par défaut ; c'est-à-dire, sauf indication contraire
indiqué par la définition de la méthode ou les contrôles de cache explicites (voir [Section 4.2.2 de \[RFC7234\]](#)).

6.3.5 . 204 Aucun contenu

Le code d'état 204 (Aucun contenu) indique que le serveur a
a répondu avec succès à la demande et qu'il n'y a pas de demande
supplémentaire
contenu à envoyer dans le corps de la charge utile de la réponse. Métadonnées
dans le
les champs d'en-tête de réponse font référence à la ressource cible et à sa
ressource sélectionnée
représentation après que l'action demandée a été appliquée.

Par exemple, si un code d'état 204 est reçu en réponse à un PUT
la requête et la réponse contiennent un champ d'en-tête ETag, puis le PUT
a réussi et la valeur du champ ETag contient la balise d'entité pour
la nouvelle représentation de cette ressource cible.

La réponse 204 permet à un serveur d'indiquer que l'action a été
appliqué avec succès à la ressource cible, tout en impliquant que le
l'agent utilisateur n'a pas besoin de quitter son « document » actuel
vue" (le cas échéant). Le serveur suppose que l'agent utilisateur fournira
une indication du succès pour son utilisateur, en accord avec ses propres
interface et appliquer toutes les métadonnées nouvelles ou mises à jour dans
la réponse à
sa représentation active.

Par exemple, un code d'état 204 est couramment utilisé avec l'édition de
documents
interfaces correspondant à une action « enregistrer », telles que le document
L'enregistrement reste accessible à l'utilisateur pour modification. Il est
également
fréquemment utilisé avec des interfaces qui attendent des transferts de
données automatisés
être répandu, comme dans les systèmes de contrôle de version distribués.

Une réponse 204 se termine par la première ligne vide après l'en-tête
champs car il ne peut pas contenir de corps de message.

Une réponse 204 peut être mise en cache par défaut ; c'est-à-dire, sauf
indication contraire
indiqué par la définition de la méthode ou les contrôles de cache explicites
(voir [Section 4.2.2 de \[RFC7234\]](#)).

6.3.6 . 205 Réinitialiser le contenu

Le code d'état 205 (Réinitialiser le contenu) indique que le serveur a
a répondu à la demande et souhaite que l'agent utilisateur réinitialise le

« vue du document », qui a provoqué l'envoi de la demande, à son emplacement d'origine
état tel que reçu du serveur d'origine.

Cette réponse est destinée à prendre en charge un cas d'utilisation courant de saisie de données

où l'utilisateur reçoit du contenu qui prend en charge la saisie de données (un formulaire, bloc-notes, toile, etc.), saisit ou manipule des données dans cet espace,

Piste des normes Fielding & Reschke [Page 53]

[RFC 7231](#)

HTTP/1.1 Sémantique et contenu Juin 2014

provoque la soumission des données saisies dans une requête, puis le le mécanisme de saisie des données est réinitialisé pour la prochaine entrée afin que l'utilisateur puisse lancer facilement une autre action d'entrée.

Étant donné que le code d'état 205 implique qu'aucun contenu supplémentaire ne sera

à condition qu'un serveur NE DOIT PAS générer de charge utile dans une réponse 205.

en d'autres termes, un serveur DOIT effectuer l'une des opérations suivantes pour un 205

réponse : a) indiquer un corps de longueur nulle pour la réponse en y compris un champ d'en-tête Content-Length avec une valeur de 0 ; b) indiquer une charge utile de longueur nulle pour la réponse en incluant un Champ d'en-tête Transfer-Encoding avec une valeur chunked et un message corps constitué d'un seul morceau de longueur nulle ; ou, c) fermer le connexion immédiatement après l'envoi de la ligne vide mettant fin à la section d'en-tête.

[6.4](#) . Redirection 3xx

La classe de code d'état 3xx (Redirection) indique que d'autres une action doit être entreprise par l'agent utilisateur afin de remplir les demande. Si un champ d'en-tête Location ([Section 7.1.2](#)) est fourni, le l'agent utilisateur PEUT rediriger automatiquement sa requête vers l'URI référencé par la valeur du champ Emplacement, même si le statut spécifique Le code n'est pas compris. La redirection automatique doit être effectuée avec

prendre soin des méthodes qui ne sont pas connues pour être sûres, telles que définies dans [la section 4.2.1](#) ,

car l'utilisateur peut ne pas souhaiter rediriger une demande non sécurisée.

Il existe plusieurs types de redirections :

1. Redirections indiquant que la ressource pourrait être disponible à un URI différent, tel que fourni par le champ Emplacement, comme dans le codes d'état 301 (déplacé définitivement), 302 (trouvé) et 307 (Redirection temporaire).
2. Redirection qui offre un choix de ressources correspondantes, chacune capable de représenter la cible de la requête d'origine, comme dans le Code d'état 300 (choix multiples).
3. Redirection vers une ressource différente, identifiée par l'emplacement champ, qui peut représenter une réponse indirecte à la demande, comme dans le code d'état 303 (Voir Autre).
4. Redirection vers un résultat précédemment mis en cache, comme dans le 304 (Non Code d'état (modifié)).

Remarque : dans HTTP/1.0, les codes d'état 301 (déplacé définitivement) et 302 (Trouvé) ont été définis pour le premier type de redirection

([\[RFC1945\]](#), [Section 9.3](#)). Les premiers agents utilisateurs étaient divisés sur la question de savoir si
La méthode appliquée à la cible de redirection serait la même que la

Piste des normes Fielding & Reschke [Page 54]

[RFC 7231](#)

HTTP/1.1 Sémantique et contenu Juin 2014

requête originale ou serait réécrite en GET. Bien que HTTP défini à l'origine l'ancienne sémantique pour 301 et 302 (pour correspondre sa mise en œuvre originale au CERN), et défini 303 (Voir Autre) pour correspondre à cette dernière sémantique, la pratique dominante s'est progressivement ont convergé vers cette dernière sémantique pour 301 et 302 également. la première révision de HTTP/1.1 a ajouté 307 (Redirection temporaire) à indiquer l'ancienne sémantique sans être impacté par des divergences pratique. Plus de 10 ans plus tard, la plupart des agents utilisateurs utilisent encore la méthode réécriture pour 301 et 302 ; par conséquent, cette spécification rend ce comportement est conforme lorsque la requête d'origine est POST.

Un client DOIT détecter et intervenir dans les redirections cycliques (c.-à-d. boucles de redirection « infinies »).

Remarque : une version antérieure de cette spécification recommandait un maximum de cinq redirections ([\[RFC2068\]](#), [section 10.3](#)). Contenu les développeurs doivent être conscients que certains clients peuvent mettre en œuvre de telles une limitation fixe.

[6.4.1](#) . 300 choix multiples

Le code d'état 300 (choix multiples) indique que la cible la ressource a plus d'une représentation, chacune avec sa propre plus identifiant spécifique et des informations sur les alternatives sont en cours fourni pour que l'utilisateur (ou l'agent utilisateur) puisse sélectionner un représentation en redirigeant sa demande vers un ou plusieurs de ces identifiants. Autrement dit, le serveur souhaite que l'agent utilisateur s'engager dans une négociation réactive pour sélectionner la solution la plus appropriée représentation(s) pour ses besoins ([Section 3.4](#)).

Si le serveur a un choix préféré, le serveur DEVRAIT générer un Champ d'en-tête d'emplacement contenant la référence URI d'un choix préféré. L'agent utilisateur PEUT utiliser la valeur du champ Emplacement pour l'automatisation redirection.

Pour les méthodes de requête autres que HEAD, le serveur DOIT générer un charge utile dans la réponse 300 contenant une liste de représentation métadonnées et référence(s) URI à partir desquelles l'utilisateur ou l'agent utilisateur peut

Choisissez celui qui vous convient le mieux. L'agent utilisateur PEUT effectuer une sélection.

à partir de cette liste automatiquement s'il comprend le support fourni type. Un format spécifique pour la sélection automatique n'est pas défini par cette spécification car HTTP essaie de rester orthogonal à la définition de ses charges utiles. En pratique, la représentation est fournis dans un format facilement analysable et considéré comme acceptable

pour
l'agent utilisateur, tel que déterminé par la conception ou le contenu
partagé
négociation, ou dans un format hypertexte communément accepté.

Piste des normes Fielding & Reschke [Page 55]

[RFC 7231](#)

HTTP/1.1 Sémantique et contenu Juin 2014

Une réponse 300 peut être mise en cache par défaut ; c'est-à-dire, sauf indication contraire
indiqué par la définition de la méthode ou les contrôles de cache explicites (voir [Section 4.2.2 de \[RFC7234\]](#)).

Remarque : La proposition initiale pour le code de statut 300 définissait le

Champ d'en-tête URI fournissant une liste d'alternatives représentations, de telle sorte qu'il serait utilisable pour 200, 300 et 406 réponses et être transférées dans les réponses à la méthode HEAD. Cependant, le manque de déploiement et le désaccord sur la syntaxe ont conduit à
à la fois l'URI et les Alternates (une proposition ultérieure) étant abandonnés
cette spécification. Il est possible de communiquer la liste en utilisant un ensemble de champs d'en-tête de lien [[RFC5988](#)], chacun avec une relation de
« alternatif », bien que le déploiement soit un problème de poule et d'œuf.

[6.4.2](#) . 301 Déplacé définitivement

Le code d'état 301 (Déplacé définitivement) indique que la cible la ressource s'est vu attribuer un nouvel URI permanent et tout futur les références à cette ressource doivent utiliser l'un des URI inclus. Les clients disposant de capacités d'édition de liens doivent automatiquement se reconnecter
références à l'URI de demande effective à un ou plusieurs des nouveaux références envoyées par le serveur, lorsque cela est possible.

Le serveur DOIT générer un champ d'en-tête Location dans la réponse contenant une référence URI préférée pour le nouvel URI permanent. l'agent utilisateur PEUT utiliser la valeur du champ Emplacement pour l'automatisation
redirection. La charge utile de réponse du serveur contient généralement une courte
note hypertexte avec un hyperlien vers le(s) nouvel(aux) URI.

Remarque : pour des raisons historiques, un agent utilisateur PEUT modifier la demande
méthode de POST à GET pour la requête suivante. Si cela
le comportement est indésirable, le code d'état 307 (Redirection temporaire)
peut être utilisé à la place.

Une réponse 301 peut être mise en cache par défaut, sauf indication contraire.
indiqué par la définition de la méthode ou les contrôles de cache explicites (voir [Section 4.2.2 de \[RFC7234\]](#)).

[6.4.3](#) . 302 Trouvé

Le code d'état 302 (Trouvé) indique que la ressource cible réside temporairement sous un URI différent. Depuis la redirection peuvent être modifiés à l'occasion, le client doit continuer à utiliser le URI de demande efficace pour les demandes futures.

Piste des normes Fielding & Reschke [Page 56]

[RFC 7231](#)

HTTP/1.1 Sémantique et contenu Juin 2014

Le serveur DOIT générer un champ d'en-tête Location dans la réponse contenant une référence URI pour les différents URI. L'agent utilisateur PEUT Utiliser la valeur du champ Emplacement pour la redirection automatique. Le serveur

La charge utile de réponse contient généralement une courte note hypertexte avec un

lien hypertexte vers les différents URI.

Remarque : pour des raisons historiques, un agent utilisateur PEUT modifier la demande

méthode de POST à GET pour la requête suivante. Si cela le comportement est indésirable, le code d'état 307 (Redirection temporaire) peut être utilisé à la place.

[6.4.4](#) . 303 Voir Autre

Le code d'état 303 (Voir autre) indique que le serveur est rediriger l'agent utilisateur vers une ressource différente, comme indiqué par un

URI dans le champ d'en-tête Location, qui est destiné à fournir un réponse indirecte à la requête initiale. Un agent utilisateur peut effectuer une demande de récupération ciblant cet URI (une demande GET ou HEAD si utilisant HTTP), qui pourrait également être redirigé, et présenter l'éventuel

résultat en réponse à la requête initiale. Notez que le nouvel URI dans le champ d'en-tête Emplacement n'est pas considéré comme équivalent à URI de demande effective.

Ce code d'état s'applique à toute méthode HTTP. Il est principalement utilisé pour permettre la sortie d'une action POST pour rediriger l'agent utilisateur

à une ressource sélectionnée, car cela fournit les informations correspondant à la réponse POST sous une forme qui peut être séparée identifié, mis en signet et mis en cache, indépendamment de l'original demande.

Une réponse 303 à une requête GET indique que le serveur d'origine ne ne pas avoir une représentation de la ressource cible qui peut être transférées par le serveur via HTTP. Cependant, le champ Emplacement la valeur fait référence à une ressource qui décrit la cible ressource, de telle sorte que faire une demande de récupération sur cette autre ressource

pourrait aboutir à une représentation utile aux destinataires sans ce qui implique qu'il représente la ressource cible d'origine. Notez que réponses aux questions de ce qui peut être représenté, de ce qui les représentations sont adéquates, et ce qui pourrait être une description utile

sont en dehors du champ d'application de HTTP.

À l'exception des réponses à une requête HEAD, la représentation d'un 303 la réponse doit contenir une courte note hypertexte avec un lien hypertexte vers

la même référence URI fournie dans le champ d'en-tête Location.

Piste des normes Fielding & Reschke [Page 57]

[6.4.5](#) . **305 Utiliser un proxy**

Le code d'état 305 (Utiliser un proxy) a été défini dans une version précédente de cette spécification et est désormais obsolète (Annexe B).

[6.4.6](#) . **306 (inutilisé)**

Le code d'état 306 a été défini dans une version précédente de ce spécification, n'est plus utilisé et le code est réservé.

[6.4.7](#) . **307 Redirection temporaire**

Le code d'état 307 (Redirection temporaire) indique que la cible la ressource réside temporairement sous un URI différent et l'agent utilisateur

NE DOIT PAS modifier la méthode de requête si elle effectue une opération automatique

Redirection vers cet URI. La redirection pouvant évoluer au fil du temps, le client doit continuer à utiliser l'URI de demande effective d'origine pour les demandes futures.

Le serveur DOIT générer un champ d'en-tête Location dans la réponse contenant une référence URI pour les différents URI. L'agent utilisateur PEUT Utilisez la valeur du champ Emplacement pour la redirection automatique. Le serveur

La charge utile de réponse contient généralement une courte note hypertexte avec un

lien hypertexte vers les différents URI.

Remarque : ce code d'état est similaire au 302 (Trouvé), sauf qu'il ne permet pas de changer la méthode de requête de POST à GET. Ceci la spécification ne définit aucun équivalent pour 301 (Déplacé (De façon permanente) ([[RFC7238](#)], cependant, définit le code d'état 308 (Redirection permanente) à cet effet).

[6.5](#) . **Erreur client 4xx**

La classe de code d'état 4xx (erreur client) indique que le client semble avoir commis une erreur. Sauf en réponse à une requête HEAD, le serveur DEVRAIT envoyer une représentation contenant une explication du situation d'erreur, et si elle est temporaire ou permanente condition. Ces codes d'état s'appliquent à toute méthode de demande.

Les agents utilisateurs DOIVENT afficher toute représentation incluse à l'utilisateur.

[6.5.1](#) . **400 Mauvaise requête**

Le code d'état 400 (mauvaise demande) indique que le serveur ne peut pas ou ne traitera pas la demande en raison de quelque chose qui est perçu comme étant

une erreur client (par exemple, une syntaxe de requête mal formée, une requête non valide)

cadrage de message ou routage de requête trompeur).

Piste des normes Fielding & Reschke [Page 58]

[RFC 7231](#)

HTTP/1.1 Sémantique et contenu Juin 2014

6.5.2 . 402 Paiement requis

Le code d'état 402 (Paiement requis) est réservé pour une utilisation future.

6.5.3 . 403 Interdit

Le code d'état 403 (Interdit) indique que le serveur a compris la requête mais refuse de l'autoriser. Un serveur qui souhaite rendre public les raisons pour lesquelles la demande a été refusée peut décrire cela raison dans la charge utile de réponse (le cas échéant).

Si des informations d'authentification ont été fournies dans la demande, le serveur les considère comme insuffisants pour accorder l'accès. Le client NE DOIT PAS répéter automatiquement la demande avec le même informations d'identification. Le client PEUT répéter la demande avec des informations nouvelles ou différentes. informations d'identification. Cependant, une demande pourrait être refusée pour des raisons sans rapport avec les informations d'identification.

Un serveur d'origine qui souhaite « masquer » l'existence actuelle d'un la ressource cible interdite PEUT plutôt répondre avec un code d'état de 404 (non trouvé).

6.5.4 . 404 Non trouvé

Le code d'état 404 (non trouvé) indique que le serveur d'origine n'a pas ne trouve pas de représentation actuelle pour la ressource cible ou n'est pas prêt à divulguer son existence. Un code d'état 404 ne signifie pas indiquer si ce manque de représentation est temporaire ou permanent ; le code d'état 410 (Parti) est préféré au 404 si le le serveur d'origine sait, vraisemblablement par des moyens configurables, que il est probable que la condition soit permanente.

Une réponse 404 peut être mise en cache par défaut, sauf indication contraire. indiqué par la définition de la méthode ou les contrôles de cache explicites (voir [Section 4.2.2 de \[RFC7234\]](#)).

6.5.5 . 405 Méthode non autorisée

Le code d'état 405 (Méthode non autorisée) indique que la méthode reçu dans la ligne de requête est connu du serveur d'origine mais pas pris en charge par la ressource cible. Le serveur d'origine DOIT générer un Autoriser le champ d'en-tête dans une réponse 405 contenant une liste de la cible méthodes actuellement prises en charge par la ressource.

Une réponse 405 peut être mise en cache par défaut, sauf indication contraire.

indiqué par la définition de la méthode ou les contrôles de cache explicites (voir [Section 4.2.2 de \[RFC7234\]](#)).

Piste des normes Fielding & Reschke [Page 59]

[6.5.6](#) . 406 Non acceptable

Le code d'état 406 (non acceptable) indique que la cible la ressource n'a pas de représentation actuelle qui serait acceptable pour l'agent utilisateur, selon la négociation proactive champs d'en-tête reçus dans la requête ([Section 5.3](#)), et le serveur n'est pas disposé à fournir une représentation par défaut.

Le serveur DOIT générer une charge utile contenant une liste des ressources disponibles

caractéristiques de représentation et identifiants de ressources correspondants

parmi lesquels l'utilisateur ou l'agent utilisateur peut choisir celui qui lui convient le mieux

approprié. Un agent utilisateur PEUT sélectionner automatiquement le plus choix approprié dans cette liste. Cependant, cette spécification ne ne définit aucune norme pour une telle sélection automatique, comme décrit dans

[Section 6.4.1](#) .

[6.5.7](#) . 408 Délai d'expiration de la requête

Le code d'état 408 (Request Timeout) indique que le serveur n'a pas ne pas recevoir un message de demande complet dans le délai imparti Prêt à attendre. Un serveur DEVRAIT envoyer l'option de connexion « fermer » ([Section 6.1 de \[RFC7230\]](#)) dans la réponse, puisque 408 implique que le serveur a décidé de fermer la connexion plutôt que de continuer en attente. Si le client a une demande en attente en transit, le le client PEUT répéter cette demande sur une nouvelle connexion.

[6.5.8](#) . 409 Conflit

Le code d'état 409 (Conflit) indique que la demande n'a pas pu être traitée. être terminé en raison d'un conflit avec l'état actuel de la cible ressource. Ce code est utilisé dans les situations où l'utilisateur pourrait être

capable de résoudre le conflit et de soumettre à nouveau la requête. Le serveur

DEVRAIT générer une charge utile qui inclut suffisamment d'informations pour un utilisateur reconnaître la source du conflit.

Les conflits sont plus susceptibles de se produire en réponse à une requête PUT.

par exemple, si le contrôle de version était utilisé et que la représentation était

PUT a inclus des modifications à une ressource qui entrent en conflit avec celles apportées par

une demande antérieure (tierce), le serveur d'origine peut utiliser une erreur 409

réponse indiquant qu'il ne peut pas traiter la demande. Dans ce cas cas, la représentation de la réponse contiendrait probablement des informations

utile pour fusionner les différences en fonction de l'historique des révisions.

6.5.9 . 410 Disparu

Le code d'état 410 (Parti) indique que l'accès à la cible la ressource n'est plus disponible sur le serveur d'origine et que cela Cette condition est susceptible d'être permanente. Si le serveur d'origine ne

Piste des normes Fielding & Reschke [Page 60]

[RFC 7231](#)

HTTP/1.1 Sémantique et contenu Juin 2014

savoir, ou n'a aucune possibilité de déterminer, si la condition ou non est permanent, le code d'état 404 (non trouvé) doit être utilisé plutôt.

La réponse 410 est principalement destinée à faciliter la tâche du Web maintenance en notifiant au destinataire que la ressource est intentionnellement indisponible et que les propriétaires du serveur souhaitent que

Les liens distants vers cette ressource doivent être supprimés. Un tel événement est fréquent.

pour des services promotionnels à durée limitée et pour des ressources appartenant à

les individus ne sont plus associés au site du serveur d'origine.

il n'est pas nécessaire de marquer toutes les ressources définitivement indisponibles comme

« disparu » ou de conserver la marque pendant une durée donnée - c'est à dire la discrétion du propriétaire du serveur.

Une réponse 410 peut être mise en cache par défaut ; c'est-à-dire, sauf indication contraire

indiqué par la définition de la méthode ou les contrôles de cache explicites (voir

[Section 4.2.2 de \[RFC7234\]](#)).

[6.5.10](#) . 411 Longueur requise

Le code d'état 411 (longueur requise) indique que le serveur refuse d'accepter la demande sans une longueur de contenu définie ([Section 3.3.2 de \[RFC7230\]](#)). Le client PEUT répéter la demande si il ajoute un champ d'en-tête Content-Length valide contenant la longueur de le corps du message dans le message de demande.

[6.5.11](#) . 413 Charge utile trop grande

Le code d'état 413 (charge utile trop grande) indique que le serveur est refuser de traiter une demande car la charge utile de la demande est plus importante

que le serveur est disposé ou capable de traiter. Le serveur PEUT fermer la connexion pour empêcher le client de poursuivre la demande.

Si la condition est temporaire, le serveur DEVRAIT générer un Champ d'en-tête Retry-After pour indiquer qu'il est temporaire et après à quelle heure le client PEUT réessayer.

[6.5.12](#) . 414 URI trop long

Le code d'état 414 (URI trop long) indique que le serveur est refuser de traiter la demande car la cible de la demande ([section 5.3 de \[RFC7230\]](#)) est plus longue que ce que le serveur est prêt à interpréter.

Cette condition rare n'est susceptible de se produire que lorsqu'un client a conversion incorrecte d'une requête POST en requête GET avec une requête longue

informations, lorsque le client est descendu dans un « trou noir » de redirection (par exemple, un préfixe d'URI redirigé qui pointe vers un suffixe de lui-même) ou lorsque le serveur est attaqué par un client qui tente d'exploiter les failles de sécurité potentielles.

Piste des normes Fielding & Reschke [Page 61]

[RFC 7231](#)

HTTP/1.1 Sémantique et contenu Juin 2014

Une réponse 414 peut être mise en cache par défaut ; c'est-à-dire, sauf indication contraire
indiqué par la définition de la méthode ou les contrôles de cache explicites (voir [Section 4.2.2 de \[RFC7234\]](#)).

[6.5.13](#) . 415 Type de média non pris en charge

Le code d'état 415 (type de support non pris en charge) indique que le le serveur d'origine refuse de traiter la demande car la charge utile est dans un format non pris en charge par cette méthode sur la ressource cible.

Le problème de format peut être dû à la demande indiquée
Type de contenu ou codage de contenu, ou à la suite de l'inspection du données directement.

[6.5.14](#) . 417 Échec des attentes

Le code d'état 417 (échec de l'attente) indique que le attente donnée dans le champ d'en-tête Expect de la requête ([Section 5.1.1](#)) n'a pas pu être satisfaite par au moins un des entrants serveurs.

[6.5.15](#) . 426 Mise à niveau requise

Le code d'état 426 (mise à niveau requise) indique que le serveur refuse d'exécuter la requête en utilisant le protocole actuel mais pourrait être prêt à le faire après que le client soit passé à une version différente protocole. Le serveur DOIT envoyer un champ d'en-tête Upgrade dans une erreur 426

réponse pour indiquer le(s) protocole(s) requis ([section 6.7 de \[RFC7230\]](#)).

Exemple:

```
Mise à niveau HTTP/1.1 426 requise
Mise à niveau : HTTP/3.0
Connexion : Mise à niveau
Contenu-Longueur : 53
Type de contenu : texte/brut
```

Ce service nécessite l'utilisation du protocole HTTP/3.0.

[6.6](#) . Erreur de serveur 5xx

La classe de code d'état 5xx (erreur de serveur) indique que le serveur est conscient qu'il a commis une erreur ou est incapable d'exécuter la méthode demandée. Sauf en cas de réponse à une requête HEAD, la le serveur DEVRAIT envoyer une représentation contenant une explication du situation d'erreur, et si elle est temporaire ou permanente

Piste des normes Fielding & Reschke [Page 62]

condition. Un agent utilisateur DEVRAIT afficher toute représentation incluse à l'utilisateur. Ces codes de réponse s'appliquent à toute demande méthode.

[6.6.1](#) . 500 Erreur interne du serveur

Le code d'état 500 (erreur interne du serveur) indique que le serveur rencontré une condition inattendue qui l'a empêché de remplir la demande.

[6.6.2](#) . 501 Non implémenté

Le code d'état 501 (non implémenté) indique que le serveur ne prend pas en charge la fonctionnalité requise pour répondre à la demande. Ceci est la réponse appropriée lorsque le serveur ne reconnaît pas le méthode de requête et n'est pas capable de la prendre en charge pour aucune ressource.

Une réponse 501 peut être mise en cache par défaut, sauf indication contraire. indiqué par la définition de la méthode ou les contrôles de cache explicites (voir [Section 4.2.2 de \[RFC7234\]](#)).

[6.6.3](#) . 502 Mauvaise passerelle

Le code d'état 502 (Bad Gateway) indique que le serveur, bien que agissant comme une passerelle ou un proxy, a reçu une réponse invalide d'un serveur entrant auquel il a accédé en tentant de répondre à la demande.

[6.6.4](#) . 503 Service indisponible

Le code d'état 503 (Service indisponible) indique que le serveur est actuellement incapable de traiter la demande en raison d'une surcharge temporaire ou une maintenance programmée, qui sera probablement atténuée après un certain temps retard. Le serveur PEUT envoyer un champ d'en-tête Retry-After ([Section 7.1.3](#)) pour suggérer une durée appropriée pour le client doit attendre avant de réessayer la demande.

Remarque : l'existence du code d'état 503 n'implique pas qu'un Le serveur doit l'utiliser en cas de surcharge. Certains serveurs peuvent refusez simplement la connexion.

[6.6.5](#) . 504 Délai d'expiration de la passerelle

Le code d'état 504 (Délai d'expiration de la passerelle) indique que le serveur, tout en agissant comme passerelle ou proxy, n'a pas reçu de réponse en temps opportun à partir d'un serveur en amont auquel il devait accéder pour terminer le

demande.

Piste des normes Fielding & Reschke [Page 63]

6.6.6 . 505 Version HTTP non prise en charge

Le code d'état 505 (version HTTP non prise en charge) indique que le serveur ne prend pas en charge, ou refuse de prendre en charge, la version majeure de HTTP utilisé dans le message de requête. Le serveur indique qu'il est incapable ou ne veut pas répondre à la demande en utilisant le même version majeure en tant que client, comme décrit dans [la section 2.6 de \[RFC7230\]](#), sauf avec ce message d'erreur. Le serveur DEVRAIT générer une représentation pour la réponse 505 qui décrit pourquoi cette version n'est pas prise en charge et quels autres protocoles sont pris en charge par ce serveur.

7. Champs d'en-tête de réponse

Les champs d'en-tête de réponse permettent au serveur de transmettre des informations supplémentaires informations sur la réponse au-delà de ce qui est placé dans le ligne d'état. Ces champs d'en-tête fournissent des informations sur le serveur, concernant un accès ultérieur à la ressource cible, ou concernant des informations connexes ressources.

Bien que chaque champ d'en-tête de réponse ait une signification définie, En général, la sémantique précise pourrait être encore affinée par sémantique de la méthode de requête et/ou du code d'état de réponse.

7.1 . Données de contrôle

Les champs d'en-tête de réponse peuvent fournir des données de contrôle qui complètent les code d'état, dirige la mise en cache ou indique au client où aller suivant.

+-----+-----+-----+-----+-----+-----+	
Nom du champ d'en-tête	Défini dans...
+-----+-----+-----+-----+-----+-----+	
Âge	Section 5.1 de [RFC7234]
Cache-Control	Section 5.2 de [RFC7234]
Expire	Section 5.3 de [RFC7234]
Date	Section 7.1.1.2
Emplacement	Section 7.1.2
Réessayer après	Section 7.1.3
Varier	Section 7.1.4
Avertissement	Section 5.5 de [RFC7234]
+-----+-----+-----+-----+-----+-----+	

Piste des normes Fielding & Reschke [Page 64]

[RFC 7231](#)

HTTP/1.1 Sémantique et contenu Juin 2014

[7.1.1](#) . Date d'origine**[7.1.1.1](#) . Formats de date et d'heure**

Avant 1995, il y avait trois formats différents couramment utilisés par serveurs pour communiquer les horodatages. Pour la compatibilité avec les anciens implémentations, les trois sont définies ici. Le format préféré est un sous-ensemble de longueur fixe et à zone unique de la date et de l'heure spécification utilisée par le format de message Internet [[RFC5322](#)].

Date HTTP = date de fixation du FMI / date d'obs

Un exemple du format préféré est

Dim. 6 nov. 1994 08:49:37 GMT ; FMI-fixdate

Voici des exemples des deux formats obsolètes :

Dimanche 6 novembre 1994 08:49:37 GMT ; format [RFC 850 obsolète](#)
 Dim 6 novembre 08:49:37 1994 ; Format asctime() d'ANSI C

Un destinataire qui analyse une valeur d'horodatage dans un champ d'en-tête HTTP

DOIT accepter les trois formats de date HTTP. Lorsqu'un expéditeur génère une champ d'en-tête qui contient un ou plusieurs horodatages définis comme Date HTTP, l'expéditeur DOIT générer ces horodatages dans le Format de date fixe du FMI.

Une valeur de date HTTP représente l'heure en tant qu'instance de Coordonné Temps universel (UTC). Les deux premiers formats indiquent UTC par abréviation de trois lettres pour Greenwich Mean Time, « GMT », un prédécesseur du nom UTC ; les valeurs au format asctime sont supposées être en UTC. Un expéditeur qui génère des valeurs de date HTTP à partir d'une adresse locale

l'horloge doit utiliser NTP ([[RFC5905](#)]) ou un protocole similaire pour synchroniser son horloge sur UTC.

Format préféré :

IMF-fixdate = nom-du-jour "," SP date1 SP heure-du-jour SP GMT
 ; sous-ensemble de longueur/zone/majuscule fixe du format
 ; voir [la section 3.3 de \[RFC5322\]](#).

nom-jour = %x4D.6F.6E ; "Lun", sensible à la casse
 / %x54.75.65 ; "Mar", sensible à la casse
 / %x57.65.64 ; « Mer », sensible à la casse
 / %x54.68.75 ; "Jeu", sensible à la casse
 / %x46.72.69 ; « Ven », sensible à la casse
 / %x53.61.74 ; "Sam", sensible à la casse
 / %x53.75.6E ; « Soleil », sensible à la casse

Piste des normes Fielding & Reschke [Page 65]

[RFC 7231](#)

HTTP/1.1 Sémantique et contenu Juin 2014

date1 = jour SP mois SP année
; par exemple, 02 juin 1982

jour = 2 CHIFFRES
mois = %x4A.61.6E ; "Jan", sensible à la casse
/ %x46.65.62 ; "févr", sensible à la casse
/ %x4D.61.72 ; "Mar", sensible à la casse
/ %x41.70.72 ; "avr", sensible à la casse
/ %x4D.61.79 ; « Peut », sensible à la casse
/ %x4A.75.6E ; "Jun", sensible à la casse
/ %x4A.75.6C ; "Jul", sensible à la casse
/ %x41.75.67 ; "Août", sensible à la casse
/ %x53.65.70 ; "Sep", sensible à la casse
/ %x4F.63.74 ; "Oct", sensible à la casse
/ %x4E.6F.76 ; "Nov", sensible à la casse
/ %x44.65.63 ; "Dec", sensible à la casse
année = 4 CHIFFRES

GMT = %x47.4D.54 ; "GMT", sensible à la casse

heure du jour = heure ":" minute ":" seconde
; 00:00:00 - 23:59:60 (seconde intercalaire)

heure = 2 CHIFFRES
minute = 2 CHIFFRES
seconde = 2 CHIFFRES

Formats obsolètes :

obs-date = [rfc850](#) -date / asctime-date

[rfc850](#) -date = nom-jour-1 "," SP date2 SP heure-du-jour SP GMT
date2 = jour "-" mois "-" 2CHIFFRES
; par exemple, 02-juin-82

day-name-1 = %x4D.6F.6E.64.61.79 ; "Lundi", sensible à la casse
/ %x54.75.65.73.64.61.79 ; « Mardi », sensible à la casse
/ %x57.65.64.6E.65.73.64.61.79 ; « Mercredi », sensible à la casse
/ %x54.68.75.72.73.64.61.79 ; « Jeudi », sensible à la casse
/ %x46.72.69.64.61.79 ; « Vendredi », sensible à la casse
/ %x53.61.74.75.72.64.61.79 ; « Samedi », sensible à la casse
/ %x53.75.6E.64.61.79 ; « Dimanche », sensible à la casse

asctime-date = nom-du-jour SP date3 SP heure-du-jour SP année
date3 = mois SP (2 CHIFFRES / (SP 1 CHIFFRE))
; par exemple, le 2 juin

[RFC 7231](#)

HTTP/1.1 Sémantique et contenu Juin 2014

La date HTTP est sensible à la casse. L'expéditeur NE DOIT PAS générer de données supplémentaires.

espace blanc dans une date HTTP au-delà de celui spécifiquement inclus comme SP dans

la grammaire. La sémantique du nom du jour, du jour, du mois, de l'année et l'heure du jour est la même que celle définie pour le message Internet Formater les constructions avec le nom correspondant ([[RFC5322](#)], [Section 3.3](#)).

Destinataires d'une valeur d'horodatage au format [rfc850](#) -date, qui utilise un

année à deux chiffres, DOIT interpréter un horodatage qui semble être plus plus de 50 ans dans le futur comme représentant l'année la plus récente le passé qui avait les mêmes deux derniers chiffres.

Les destinataires des valeurs d'horodatage sont encouragés à faire preuve de robustesse dans l'analyse

horodatages, sauf restriction contraire définie par la définition du champ. Pour

par exemple, les messages sont parfois transmis via HTTP à partir d'un source non HTTP susceptible de générer n'importe quelle date et heure spécifications définies par le format de message Internet.

Remarque : les exigences HTTP pour le format d'horodatage s'appliquent uniquement

à leur utilisation dans le flux du protocole. Les implémentations sont il n'est pas nécessaire d'utiliser ces formats pour la présentation utilisateur, demande exploitation forestière, etc.

[7.1.1.2](#) . Date

Le champ d'en-tête « Date » représente la date et l'heure à laquelle le message a été créé, ayant la même sémantique que l'Origine Champ de date (orig-date) défini dans [la section 3.6.1 de \[RFC5322\]](#) . Le la valeur du champ est une date HTTP, telle que définie dans [la section 7.1.1.1](#) .

Date = date HTTP

Un exemple est

Date : mardi 15 novembre 1994 08:12:31 GMT

Lorsqu'un champ d'en-tête Date est généré, l'expéditeur DOIT générer son valeur du champ comme la meilleure approximation disponible de la date et de l'heure

de génération de messages. En théorie, la date devrait représenter moment juste avant la génération de la charge utile. En pratique, la date peut être généré à tout moment lors de la création du message.

Un serveur d'origine NE DOIT PAS envoyer un champ d'en-tête Date s'il ne le fait pas

disposer d'une horloge capable de fournir une approximation raisonnable de la instance actuelle en Temps Universel Coordonné. Un serveur d'origine PEUT envoyer un champ d'en-tête Date si la réponse est dans le 1xx (Informationnel) ou classe 5xx (Erreur du serveur) de codes d'état. Un le serveur d'origine DOIT envoyer un champ d'en-tête Date dans tous les autres cas.

A recipient with a clock that receives a response message without a Date header field MUST record the time it was received and append a corresponding Date header field to the message's header section if it is cached or forwarded downstream.

A user agent MAY send a Date header field in a request, though generally will not do so unless it is believed to convey useful information to the server. For example, custom applications of HTTP might convey a Date if the server is expected to adjust its interpretation of the user's request based on differences between the user agent and server clocks.

[7.1.2.](#) Location

The "Location" header field is used in some responses to refer to a specific resource in relation to the response. The type of relationship is defined by the combination of request method and status code semantics.

Location = URI-reference

The field value consists of a single URI-reference. When it has the form of a relative reference ([\[RFC3986\]](#), [Section 4.2](#)), the final value is computed by resolving it against the effective request URI ([\[RFC3986\]](#), [Section 5](#)).

For 201 (Created) responses, the Location value refers to the primary resource created by the request. For 3xx (Redirection) responses, the Location value refers to the preferred target resource for automatically redirecting the request.

If the Location value provided in a 3xx (Redirection) response does not have a fragment component, a user agent MUST process the redirection as if the value inherits the fragment component of the URI reference used to generate the request target (i.e., the redirection inherits the original reference's fragment, if any).

For example, a GET request generated for the URI reference "http://www.example.org/~tim" might result in a 303 (See Other) response containing the header field:

Location: /People.html#tim

which suggests that the user agent redirect to "http://www.example.org/People.html#tim"

Likewise, a GET request generated for the URI reference "http://www.example.org/index.html#larry" might result in a 301 (Moved Permanently) response containing the header field:

Location: http://www.example.net/index.html

which suggests that the user agent redirect to "http://www.example.net/index.html#larry", preserving the original fragment identifier.

There are circumstances in which a fragment identifier in a Location value would not be appropriate. For example, the Location header field in a 201 (Created) response is supposed to provide a URI that is specific to the created resource.

Note: Some recipients attempt to recover from Location fields that are not valid URI references. This specification does not mandate or define such processing, but does allow it for the sake of robustness.

Note: The Content-Location header field ([Section 3.1.4.2](#)) differs from Location in that the Content-Location refers to the most specific resource corresponding to the enclosed representation. It is therefore possible for a response to contain both the Location and Content-Location header fields.

[7.1.3.](#) **Retry-After**

Servers send the "Retry-After" header field to indicate how long the user agent ought to wait before making a follow-up request. When sent with a 503 (Service Unavailable) response, Retry-After indicates how long the service is expected to be unavailable to the client. When sent with any 3xx (Redirection) response, Retry-After indicates the minimum time that the user agent is asked to wait before issuing the redirected request.

The value of this field can be either an HTTP-date or a number of seconds to delay after the response is received.

Retry-After = HTTP-date / delay-seconds

A delay-seconds value is a non-negative decimal integer, representing time in seconds.

delay-seconds = 1*DIGIT

Two examples of its use are

```
Retry-After: Fri, 31 Dec 1999 23:59:59 GMT
Retry-After: 120
```

In the latter example, the delay is 2 minutes.

[7.1.4.](#) Vary

The "Vary" header field in a response describes what parts of a request message, aside from the method, Host header field, and request target, might influence the origin server's process for selecting and representing this response. The value consists of either a single asterisk ("*") or a list of header field names (case-insensitive).

```
Vary = "*" / 1#field-name
```

A Vary field value of "*" signals that anything about the request might play a role in selecting the response representation, possibly including elements outside the message syntax (e.g., the client's network address). A recipient will not be able to determine whether this response is appropriate for a later request without forwarding the request to the origin server. A proxy **MUST NOT** generate a Vary field with a "*" value.

A Vary field value consisting of a comma-separated list of names indicates that the named request header fields, known as the selecting header fields, might have a role in selecting the representation. The potential selecting header fields are not limited to those defined by this specification.

For example, a response that contains

```
Vary: accept-encoding, accept-language
```

indicates that the origin server might have used the request's Accept-Encoding and Accept-Language fields (or lack thereof) as determining factors while choosing the content for this response.

An origin server might send Vary with a list of fields for two purposes:

1. To inform cache recipients that they **MUST NOT** use this response to satisfy a later request unless the later request has the same values for the listed fields as the original request ([Section 4.1 of \[RFC7234\]](#)). In other words, Vary expands the cache key required to match a new request to the stored cache entry.

2. To inform user agent recipients that this response is subject to content negotiation ([Section 5.3](#)) and that a different representation might be sent in a subsequent request if additional parameters are provided in the listed header fields (proactive negotiation).

An origin server SHOULD send a Vary header field when its algorithm for selecting a representation varies based on aspects of the request message other than the method and request target, unless the variance cannot be crossed or the origin server has been deliberately configured to prevent cache transparency. For example, there is no need to send the Authorization field name in Vary because reuse across users is constrained by the field definition ([Section 4.2 of \[RFC7235\]](#)). Likewise, an origin server might use Cache-Control directives ([Section 5.2 of \[RFC7234\]](#)) to supplant Vary if it considers the variance less significant than the performance cost of Vary's impact on caching.

[7.2. Validator Header Fields](#)

Validator header fields convey metadata about the selected representation ([Section 3](#)). In responses to safe requests, validator fields describe the selected representation chosen by the origin server while handling the response. Note that, depending on the status code semantics, the selected representation for a given response is not necessarily the same as the representation enclosed as response payload.

In a successful response to a state-changing request, validator fields describe the new representation that has replaced the prior selected representation as a result of processing the request.

For example, an ETag header field in a 201 (Created) response communicates the entity-tag of the newly created resource's representation, so that it can be used in later conditional requests to prevent the "lost update" problem [[RFC7232](#)].

+-----+-----+	
Header Field Name Defined in...	
+-----+-----+	
ETag	Section 2.3 of [RFC7232]
Last-Modified	Section 2.2 of [RFC7232]
+-----+-----+	

7.3. Authentication Challenges

Authentication challenges indicate what mechanisms are available for the client to provide authentication credentials in future requests.

Header Field Name	Defined in...
WWW-Authenticate	Section 4.1 of [RFC7235]
Proxy-Authenticate	Section 4.3 of [RFC7235]

7.4. Response Context

The remaining response header fields provide more information about the target resource for potential use in later requests.

Header Field Name	Defined in...
Accept-Ranges	Section 2.3 of [RFC7233]
Allow	Section 7.4.1
Server	Section 7.4.2

7.4.1. Allow

The "Allow" header field lists the set of methods advertised as supported by the target resource. The purpose of this field is strictly to inform the recipient of valid request methods associated with the resource.

Allow = #method

Example of use:

Allow: GET, HEAD, PUT

The actual set of allowed methods is defined by the origin server at the time of each request. An origin server **MUST** generate an Allow field in a 405 (Method Not Allowed) response and **MAY** do so in any other response. An empty Allow field value indicates that the resource allows no methods, which might occur in a 405 response if the resource has been temporarily disabled by configuration.

A proxy **MUST NOT** modify the Allow header field -- it does not need to understand all of the indicated methods in order to handle them according to the generic message handling rules.

7.4.2. Server

The "Server" header field contains information about the software used by the origin server to handle the request, which is often used by clients to help identify the scope of reported interoperability problems, to work around or tailor requests to avoid particular server limitations, and for analytics regarding server or operating system use. An origin server MAY generate a Server field in its responses.

Server = product *(RWS (product / comment))

The Server field-value consists of one or more product identifiers, each followed by zero or more comments ([Section 3.2 of \[RFC7230\]](#)), which together identify the origin server software and its significant subproducts. By convention, the product identifiers are listed in decreasing order of their significance for identifying the origin server software. Each product identifier consists of a name and optional version, as defined in [Section 5.5.3](#).

Example:

Server: CERN/3.0 libwww/2.17

An origin server SHOULD NOT generate a Server field containing needlessly fine-grained detail and SHOULD limit the addition of subproducts by third parties. Overly long and detailed Server field values increase response latency and potentially reveal internal implementation details that might make it (slightly) easier for attackers to find and exploit known security holes.

8. IANA Considerations

8.1. Method Registry

The "Hypertext Transfer Protocol (HTTP) Method Registry" defines the namespace for the request method token ([Section 4](#)). The method registry has been created and is now maintained at <http://www.iana.org/assignments/http-methods>.

8.1.1. Procedure

HTTP method registrations MUST include the following fields:

- o Method Name (see [Section 4](#))
- o Safe ("yes" or "no", see [Section 4.2.1](#))
- o Idempotent ("yes" or "no", see [Section 4.2.2](#))
- o Pointer to specification text

Values to be added to this namespace require IETF Review (see [\[RFC5226\]](#), [Section 4.1](#)).

8.1.2. Considerations for New Methods

Standardized methods are generic; that is, they are potentially applicable to any resource, not just one particular media type, kind of resource, or application. As such, it is preferred that new methods be registered in a document that isn't specific to a single application or data format, since orthogonal technologies deserve orthogonal specification.

Since message parsing ([Section 3.3 of \[RFC7230\]](#)) needs to be independent of method semantics (aside from responses to HEAD), definitions of new methods cannot change the parsing algorithm or prohibit the presence of a message body on either the request or the response message. Definitions of new methods can specify that only a zero-length message body is allowed by requiring a Content-Length header field with a value of "0".

A new method definition needs to indicate whether it is safe ([Section 4.2.1](#)), idempotent ([Section 4.2.2](#)), cacheable ([Section 4.2.3](#)), what semantics are to be associated with the payload body if any is present in the request and what refinements the method makes to header field or status code semantics. If the new method is cacheable, its definition ought to describe how, and under what conditions, a cache can store a response and use it to satisfy a subsequent request. The new method ought to describe whether it can be made conditional ([Section 5.2](#)) and, if so, how a server responds when the condition is false. Likewise, if the new method might have some use for partial response semantics ([\[RFC7233\]](#)), it ought to document this, too.

Note: Avoid defining a method name that starts with "M-", since that prefix might be misinterpreted as having the semantics assigned to it by [\[RFC2774\]](#).

8.1.3. Registrations

The "Hypertext Transfer Protocol (HTTP) Method Registry" has been populated with the registrations below:

Method	Safe	Idempotent	Reference
CONNECT	no	no	Section 4.3.6
DELETE	no	yes	Section 4.3.5
GET	yes	yes	Section 4.3.1
HEAD	yes	yes	Section 4.3.2
OPTIONS	yes	yes	Section 4.3.7
POST	no	no	Section 4.3.3
PUT	no	yes	Section 4.3.4
TRACE	yes	yes	Section 4.3.8

8.2. Status Code Registry

The "Hypertext Transfer Protocol (HTTP) Status Code Registry" defines the namespace for the response status-code token ([Section 6](#)). The status code registry is maintained at <http://www.iana.org/assignments/http-status-codes>.

This section replaces the registration procedure for HTTP Status Codes previously defined in [Section 7.1 of \[RFC2817\]](#).

8.2.1 . Procédure

Une inscription DOIT inclure les champs suivants :

- o Code d'état (3 chiffres)
- o Brève description
- o Pointeur vers le texte de spécification

Les valeurs à ajouter à l'espace de noms du code d'état HTTP nécessitent l'IETF

Révision (voir [\[RFC5226\]](#), [Section 4.1](#)).

8.2.2 . Considérations relatives aux nouveaux codes d'état

Lorsqu'il est nécessaire d'exprimer une sémantique pour une réponse qui n'est pas défini par les codes d'état actuels, un nouveau code d'état peut être enregistré.

Les codes d'état sont génériques ; ils sont potentiellement applicables à n'importe quel ressource, pas seulement un type de média particulier, un genre de ressource ou application du protocole HTTP. Il est donc préférable que les nouveaux codes d'état être enregistré dans un document qui n'est pas spécifique à une seule personne application.

Les nouveaux codes de statut doivent appartenir à l'une des catégories défini dans [la section 6](#). Pour permettre aux analyseurs existants de traiter les message de réponse, les nouveaux codes d'état ne peuvent pas interdire une charge utile, bien qu'ils puissent exiger un corps de charge utile de longueur nulle.

Les propositions de nouveaux codes de statut qui ne sont pas encore largement déployés devraient pour éviter d'attribuer un numéro spécifique au code jusqu'à ce qu'il y ait consensus clair sur le fait qu'il sera enregistré ; au lieu de cela, les premières ébauches peuvent utilisez une notation telle que « 4NN », ou « 3N0 » .. « 3N9 », pour indiquer le classe du ou des codes d'état proposés sans consommer de numéro prématurément.

La définition d'un nouveau code d'état doit expliquer la demande conditions qui provoqueraient une réponse contenant ce code d'état (par exemple, des combinaisons de champs d'en-tête de requête et/ou de méthodes) avec toutes les dépendances sur les champs d'en-tête de réponse (par exemple, quels champs sont obligatoires, quels champs peuvent modifier la sémantique et quel en-tête la sémantique du champ est encore affinée lorsqu'elle est utilisée avec le nouveau statut code).

La définition d'un nouveau code d'état doit préciser si oui ou non Il est cachable. Notez que tous les codes d'état peuvent être mis en cache si le la réponse dans laquelle ils se produisent contient des informations explicites sur la fraîcheur ; cependant, les codes d'état définis comme pouvant être mis en cache peuvent être mis en cache sans information explicite de fraîcheur. De même, la définition d'un code d'état peut imposer des contraintes au cache

comportement. Voir [[RFC7234](#)] pour plus d'informations.

Enfin, la définition d'un nouveau code de statut devrait indiquer si la charge utile a une association implicite avec un ressource ([Section 3.1.4.1](#)).

[8.2.3](#) . Inscriptions

Le registre des codes d'état a été mis à jour avec les enregistrements ci-dessous:

Piste des normes Fielding & Reschke [Page 76]

[RFC 7231](#)

HTTP/1.1 Sémantique et contenu Juin 2014

+-----+-----+-----+-----+			
Valeur Description Référence			
+-----+-----+-----+-----+			
100	Continuer	Section 6.2.1	
101	Protocoles de commutation	Section 6.2.2	
200	OK	Section 6.3.1	
201	Créé	Section 6.3.2	
202	Accepté	Section 6.3.3	
203	Informations non autorisées	Section 6.3.4	
204	Aucun contenu	Section 6.3.5	
205	Réinitialiser le contenu	Section 6.3.6	
300	Choix multiples	Section 6.4.1	
301	Déplacé définitivement	Section 6.4.2	
302	Trouvé	Section 6.4.3	
303	Voir Autre	Section 6.4.4	
305	Utiliser un proxy	Section 6.4.5	
306	(Inutilisé)	Section 6.4.6	
307	Redirection temporaire	Section 6.4.7	
400	Mauvaise requête	Section 6.5.1	
402	Païement requis	Section 6.5.2	
403	Interdit	Section 6.5.3	
404	Non trouvé	Section 6.5.4	
405	Méthode non autorisée	Section 6.5.5	
406	Non acceptable	Section 6.5.6	
408	Délai d'expiration de la demande	Section 6.5.7	
409	Conflit	Section 6.5.8	
410	Disparu	Section 6.5.9	
411	Longueur requise	Section 6.5.10	
413	Charge utile trop importante	Section 6.5.11	
414	URI trop long	Section 6.5.12	
415	Type de média non pris en charge	Section 6.5.13	
417	Attente échouée	Section 6.5.14	
426	Mise à niveau requise	Section 6.5.15	
500	Erreur interne du serveur	Section 6.6.1	
501	Non implémenté	Section 6.6.2	
502	Mauvaise passerelle	Section 6.6.3	
503	Service indisponible	Section 6.6.4	
504	Délai d'expiration de la passerelle	Section 6.6.5	
505	Version HTTP non prise en charge	Section 6.6.6	
+-----+-----+-----+-----+			

8.3 . Registre des champs d'en-tête

Les champs d'en-tête HTTP sont enregistrés dans les « En-têtes de message » registre situé à

< <http://www.iana.org/assignments/message-headers> >, tel que défini par [[BCP90](#)].

8.3.1 . Considérations relatives aux nouveaux champs d'en-tête

Les champs d'en-tête sont des paires clé:valeur qui peuvent être utilisées pour communiquer données sur le message, sa charge utile, la ressource cible ou le connexion (c'est-à-dire les données de contrôle). Voir [la section 3.2 de \[RFC7230\]](#) pour une définition générale de la syntaxe du champ d'en-tête dans les messages HTTP.

Les exigences relatives aux noms de champs d'en-tête sont définies dans [[BCP90](#)].

Il est conseillé aux auteurs de spécifications définissant de nouveaux domaines de conserver les nom aussi court que possible et ne pas préfixer le nom par « X- » à moins que le champ d'en-tête ne soit jamais utilisé sur Internet. (Le L'idiome du préfixe « X- » a été largement mal utilisé dans la pratique ; il a été destiné à être utilisé uniquement comme mécanisme pour éviter les collisions de noms à l'intérieur d'un logiciel propriétaire ou d'un traitement intranet, puisque le préfixe garantirait que les noms privés n'entrent jamais en conflit avec un nom nouvellement enregistré Nom Internet ; voir [[BCP178](#)] pour plus d'informations).

Les nouvelles valeurs de champ d'en-tête ont généralement leur syntaxe définie à l'aide de ABNF ([[RFC5234](#)]), en utilisant l'extension définie dans [la section 7 de \[RFC7230\]](#) si nécessaire, et sont généralement limités à la plage de Caractères US-ASCII. Champs d'en-tête nécessitant une plus grande plage de les caractères peuvent utiliser un codage tel que celui défini dans [[RFC5987](#)].

Les espaces de début et de fin dans les valeurs de champ brutes sont supprimés lors de Analyse de champ ([section 3.2.4 de \[RFC7230\]](#)). Définitions de champ où les espaces de début ou de fin dans les valeurs sont significatifs et devront utiliser une syntaxe de conteneur telle que quoted-string ([Section 3.2.6 de \[RFC7230\]](#)).

Parce que les virgules (",") sont utilisées comme délimiteur générique entre field-values, they need to be treated with care if they are allowed in the field-value. Typically, components that might contain a comma are protected with double-quotes using the quoted-string ABNF production.

For example, a textual date and a URI (either of which might contain a comma) could be safely carried in field-values like these:

```
Example-URI-Field: "http://example.com/a.html,foo",
                  "http://without-a-comma.example.com/"
Example-Date-Field: "Sat, 04 May 1996", "Wed, 14 Sep 2005"
```

Note that double-quote delimiters almost always are used with the quoted-string production; using a different syntax inside double-quotes will likely cause unnecessary confusion.

Many header fields use a format including (case-insensitively) named parameters (for instance, Content-Type, defined in [Section 3.1.1.5](#)). Allowing both unquoted (token) and quoted (quoted-string) syntax for the parameter value enables recipients to use existing parser components. When allowing both forms, the meaning of a parameter value ought to be independent of the syntax used for it (for an example, see the notes on parameter handling for media types in [Section 3.1.1.1](#)).

Authors of specifications defining new header fields are advised to consider documenting:

- o Whether the field is a single value or whether it can be a list (delimited by commas; see [Section 3.2 of \[RFC7230\]](#)).

If it does not use the list syntax, document how to treat messages where the field occurs multiple times (a sensible default would be to ignore the field, but this might not always be the right choice).

Note that intermediaries and software libraries might combine multiple header field instances into a single one, despite the field's definition not allowing the list syntax. A robust format enables recipients to discover these situations (good example: "Content-Type", as the comma can only appear inside quoted strings; bad example: "Location", as a comma can occur inside a URI).

- o Under what conditions the header field can be used; e.g., only in responses or requests, in all messages, only on responses to a particular request method, etc.
- o Whether the field should be stored by origin servers that understand it upon a PUT request.
- o Si la sémantique du champ est encore affinée par le contexte, comme par exemple par des méthodes de demande existantes ou des codes d'état.
- o S'il est approprié de répertorier le nom du champ dans la connexion champ d'en-tête (c'est-à-dire si le champ d'en-tête doit être exécuté saut par saut ; voir [Section 6.1 de \[RFC7230\]](#)).
- o Dans quelles conditions les intermédiaires sont autorisés à insérer, supprimer ou modifier la valeur du champ.

Piste des normes Fielding & Reschke [Page 79]

[RFC 7231](#)

HTTP/1.1 Sémantique et contenu Juin 2014

- o S'il est approprié de lister le nom du champ dans une variable champ d'en-tête de réponse (par exemple, lorsque le champ d'en-tête de demande est utilisé par l'algorithme de sélection de contenu d'un serveur d'origine ; voir [Section 7.1.4](#)).
- o Si le champ d'en-tête est utile ou autorisé dans les bandes-annonces (voir [Section 4.1 de \[RFC7230\]](#)).
- o Si le champ d'en-tête doit être conservé lors des redirections.
- o Si cela introduit des considérations de sécurité supplémentaires, telles que comme la divulgation de données relatives à la confidentialité.

8.3.2 . Inscriptions

Le registre « En-têtes de message » a été mis à jour avec les éléments suivants
immatriculations permanentes :

Nom du champ d'en-tête	Protocole	Statut	Référence
Accepter	http	standard	Section 5.3.2
Accept-Charset	http	standard	Section 5.3.3
Accept-Encoding	http	standard	Section 5.3.4
Accept-Language	http	standard	Section 5.3.5
Autoriser	http	standard	Section 7.4.1
Codage de contenu	http	norme	Section 3.1.2.2
Contenu-Langue	http	norme	Section 3.1.3.2
Emplacement du contenu	http	standard	Section 3.1.4.2
Type de contenu	http	standard	Section 3.1.1.5
Date	http	norme	Section 7.1.1.2
Attendre	http	standard	Section 5.1.1
De	http	norme	Section 5.5.1
Emplacement	http	norme	Section 7.1.2
Max-Forwards	http	standard	Section 5.1.2
Version MIME	http	standard	Annexe A.1
Référent	http	norme	Section 5.5.2
Réessayer après	http	standard	Section 7.1.3
Serveur	http	standard	Section 7.4.2
Agent utilisateur	http	standard	Section 5.5.3
Varier	http	norme	Section 7.1.4

Le responsable du changement pour les enregistrements ci-dessus est : « IETF (iesg@ietf.org) - Groupe de travail sur l'ingénierie Internet".

Piste des normes Fielding & Reschke [Page 80]

8.4 . Registre de codage de contenu

Le « registre de codage de contenu HTTP » définit l'espace de noms pour le contenu
noms de codage ([section 4.2 de la RFC 7230](#)). Le registre de codage de contenu
est maintenu à < <http://www.iana.org/assignments/http-parameters> >.

8.4.1 . Procédure

Les enregistrements de codage de contenu DOIVENT inclure les champs suivants :

- o Nom
- o Description
- o Pointeur vers le texte de spécification

Les noms des codages de contenu NE DOIVENT PAS chevaucher les noms de transfert
codages ([section 4 de \[RFC7230\]](#)), à moins que la transformation de codage est identique (comme c'est le cas pour les codages de compression définis dans
[Section 4.2 de \[RFC7230\]](#)).

Les valeurs à ajouter à cet espace de noms nécessitent une révision de l'IETF (voir [la section 4.1 de \[RFC5226\]](#)) et DOIVENT être conformes à l'objectif du codage du contenu.
défini dans cette section.

8.4.2 . Inscriptions

Le « Registre de codage de contenu HTTP » a été mis à jour avec le inscriptions ci-dessous :

+-----+-----+-----+-----+-----+-----+		
Nom	Description	Référence
+-----+-----+-----+-----+-----+-----+		
identité	Réservé (synonyme de « pas d'encodage » dans	Section 5.3.4
	Accepter-Encodage)	
+-----+-----+-----+-----+-----+-----+		

9. Considérations de sécurité

Cette section est destinée à informer les développeurs, les fournisseurs d'informations,
et les utilisateurs de problèmes de sécurité connus liés à la sémantique HTTP et
son utilisation pour transférer des informations sur Internet.
Les considérations liées à la syntaxe, à l'analyse et au routage des messages sont

discuté dans [la section 9 de \[RFC7230\]](#) .

La liste des considérations ci-dessous n'est pas exhaustive. La plupart des mesures de sécurité

les préoccupations liées à la sémantique HTTP concernent la sécurisation côté serveur

applications (code derrière l'interface HTTP), sécurisation de l'agent utilisateur

Piste des normes Fielding & Reschke [Page 81]

traitement des charges utiles reçues via HTTP, ou utilisation sécurisée du Internet en général, plutôt que la sécurité du protocole. Divers les organisations maintiennent des informations d'actualité et des liens vers des recherche sur la sécurité des applications Web (par exemple, [[OWASP](#)]).

[9.1](#) . Attaques basées sur les noms de fichiers et de chemins d'accès

Les serveurs d'origine utilisent fréquemment leur système de fichiers local pour gérer le mappage de l'URI de requête effective à la ressource représentations. La plupart des systèmes de fichiers ne sont pas conçus pour protéger contre les noms de fichiers ou de chemins d'accès malveillants. Par conséquent, un serveur d'origine doit éviter d'accéder à des noms qui ont une signification particulière pour le système lors du mappage de la cible de la requête vers des fichiers, des dossiers ou répertoires.

Par exemple, UNIX, Microsoft Windows et d'autres systèmes d'exploitation utilisent "." comme composant de chemin pour indiquer un niveau de répertoire supérieur au actuel, et ils utilisent des chemins ou des noms de fichiers spécialement nommés pour envoyer données aux périphériques système. Des conventions de dénomination similaires peuvent exister. dans d'autres types de systèmes de stockage. De même, le stockage local Les systèmes ont une fâcheuse tendance à privilégier la convivialité à la sécurité lors du traitement de caractères invalides ou inattendus, recomposition de caractères décomposés et normalisation de la casse noms insensibles à la casse.

Les attaques basées sur de tels noms spéciaux ont tendance à se concentrer soit sur le déni, hors service (par exemple, demander au serveur de lire à partir d'un port COM) ou divulgation de fichiers de configuration et de fichiers sources qui ne sont pas censés être servi.

[9.2](#) . Attaques basées sur l'injection de commandes, de code ou de requêtes

Les serveurs d'origine utilisent souvent des paramètres dans l'URI comme moyen de identifier les services système, sélectionner les entrées de la base de données ou choisir une source de données. Cependant, les données reçues dans une requête ne peuvent pas être fiable. Un attaquant pourrait construire n'importe laquelle des données de la requête éléments (méthode, cible de la requête, champs d'en-tête ou corps) à contenir

données qui pourraient être mal interprétées comme une commande, un code ou une requête lorsque transmis via une invocation de commande, un interpréteur de langage ou interface de base de données.

Par exemple, l'injection SQL est une attaque courante dans laquelle des données supplémentaires le langage de requête est inséré dans une partie de la cible de la requête ou champs d'en-tête (par exemple, hôte, référent, etc.). Si les données reçues sont utilisé directement dans une instruction SELECT, le langage de requête peut être interprété comme une commande de base de données plutôt que comme une simple valeur de chaîne. Ce type de vulnérabilité d'implémentation est extrêmement courant, même s'il est facile de les prévenir.

Piste des normes Fielding & Reschke [Page 82]

En général, les implémentations de ressources doivent éviter l'utilisation de requêtes

data in contexts that are processed or interpreted as instructions. Parameters ought to be compared to fixed strings and acted upon as a result of that comparison, rather than passed through an interface that is not prepared for untrusted data. Received data that isn't based on fixed parameters ought to be carefully filtered or encoded to avoid being misinterpreted.

Similar considerations apply to request data when it is stored and later processed, such as within log files, monitoring tools, or when included within a data format that allows embedded scripts.

[9.3. Disclosure of Personal Information](#)

Clients are often privy to large amounts of personal information, including both information provided by the user to interact with resources (e.g., the user's name, location, mail address, passwords, encryption keys, etc.) and information about the user's browsing activity over time (e.g., history, bookmarks, etc.). Implementations need to prevent unintentional disclosure of personal information.

[9.4. Disclosure of Sensitive Information in URIs](#)

URIs are intended to be shared, not secured, even when they identify secure resources. URIs are often shown on displays, added to templates when a page is printed, and stored in a variety of unprotected bookmark lists. It is therefore unwise to include information within a URI that is sensitive, personally identifiable, or a risk to disclose.

Authors of services ought to avoid GET-based forms for the submission of sensitive data because that data will be placed in the request-target. Many existing servers, proxies, and user agents log or display the request-target in places where it might be visible to third parties. Such services ought to use POST-based form submission instead.

Since the Referer header field tells a target site about the context that resulted in a request, it has the potential to reveal information about the user's immediate browsing history and any personal information that might be found in the referring resource's URI. Limitations on the Referer header field are described in [Section 5.5.2](#) to address some of its security considerations.

9.5. Disclosure of Fragment after Redirects

Although fragment identifiers used within URI references are not sent in requests, implementers ought to be aware that they will be visible to the user agent and any extensions or scripts running as a result of the response. In particular, when a redirect occurs and the original request's fragment identifier is inherited by the new reference in Location ([Section 7.1.2](#)), this might have the effect of disclosing one site's fragment to another site. If the first site uses personal information in fragments, it ought to ensure that redirects to other sites include a (possibly empty) fragment component in order to block that inheritance.

9.6. Disclosure of Product Information

The User-Agent ([Section 5.5.3](#)), Via ([Section 5.7.1 of \[RFC7230\]](#)), and Server ([Section 7.4.2](#)) header fields often reveal information about the respective sender's software systems. In theory, this can make it easier for an attacker to exploit known security holes; in practice, attackers tend to try all potential holes regardless of the apparent software versions being used.

Proxies that serve as a portal through a network firewall ought to take special precautions regarding the transfer of header information that might identify hosts behind the firewall. The Via header field allows intermediaries to replace sensitive machine names with pseudonyms.

9.7. Browser Fingerprinting

Browser fingerprinting is a set of techniques for identifying a specific user agent over time through its unique set of characteristics. These characteristics might include information related to its TCP behavior, feature capabilities, and scripting environment, though of particular interest here is the set of unique characteristics that might be communicated via HTTP. Fingerprinting is considered a privacy concern because it enables tracking of a user agent's behavior over time without the corresponding controls that the user might have over other forms of data collection (e.g., cookies). Many general-purpose user agents (i.e., Web browsers) have taken steps to reduce their fingerprints.

There are a number of request header fields that might reveal information to servers that is sufficiently unique to enable fingerprinting. The From header field is the most obvious, though it is expected that From will only be sent when self-identification is desired by the user. Likewise, Cookie header fields are deliberately

designed to enable re-identification, so fingerprinting concerns only apply to situations where cookies are disabled or restricted by the user agent's configuration.

The User-Agent header field might contain enough information to uniquely identify a specific device, usually when combined with other characteristics, particularly if the user agent sends excessive details about the user's system or extensions. However, the source of unique information that is least expected by users is proactive negotiation ([Section 5.3](#)), including the Accept, Accept-Charset, Accept-Encoding, and Accept-Language header fields.

In addition to the fingerprinting concern, detailed use of the Accept-Language header field can reveal information the user might consider to be of a private nature. For example, understanding a given language set might be strongly correlated to membership in a particular ethnic group. An approach that limits such loss of privacy would be for a user agent to omit the sending of Accept-Language except for sites that have been whitelisted, perhaps via interaction after detecting a Vary header field that indicates language negotiation might be useful.

In environments where proxies are used to enhance privacy, user agents ought to be conservative in sending proactive negotiation header fields. General-purpose user agents that provide a high degree of header field configurability ought to inform users about the loss of privacy that might result if too much detail is provided. As an extreme privacy measure, proxies could filter the proactive negotiation header fields in relayed requests.

[10.](#) Acknowledgments

See [Section 10 of \[RFC7230\]](#).

[11.](#) References

[11.1.](#) Normative References

- [RFC2045] Freed, N. and N. Borenstein, "Multipurpose Internet Mail Extensions (MIME) Part One: Format of Internet Message Bodies", [RFC 2045](#), November 1996.
- [RFC2046] Freed, N. and N. Borenstein, "Multipurpose Internet Mail Extensions (MIME) Part Two: Media Types", [RFC 2046](#), November 1996.
- [RFC2119] Bradner, S., "Key words for use in RFCs to Indicate Requirement Levels", [BCP 14](#), [RFC 2119](#), March 1997.

- [RFC3986] Berners-Lee, T., Fielding, R., and L. Masinter, "Uniform Resource Identifier (URI): Generic Syntax", STD 66, [RFC 3986](#), January 2005.
- [RFC4647] Phillips, A., Ed. and M. Davis, Ed., "Matching of Language Tags", [BCP 47](#), [RFC 4647](#), September 2006.
- [RFC5234] Crocker, D., Ed. and P. Overell, "Augmented BNF for Syntax Specifications: ABNF", STD 68, [RFC 5234](#), January 2008.
- [RFC5646] Phillips, A., Ed. and M. Davis, Ed., "Tags for Identifying Languages", [BCP 47](#), [RFC 5646](#), September 2009.
- [RFC6365] Hoffman, P. and J. Klensin, "Terminology Used in Internationalization in the IETF", [BCP 166](#), [RFC 6365](#), September 2011.
- [RFC7230] Fielding, R., Ed. and J. Reschke, Ed., "Hypertext Transfer Protocol (HTTP/1.1): Message Syntax and Routing", [RFC 7230](#), June 2014.
- [RFC7232] Fielding, R., Ed. and J. Reschke, Ed., "Hypertext Transfer Protocol (HTTP/1.1): Conditional Requests", [RFC 7232](#), June 2014.
- [RFC7233] Fielding, R., Ed., Lafon, Y., Ed., and J. Reschke, Ed., "Hypertext Transfer Protocol (HTTP/1.1): Range Requests", [RFC 7233](#), June 2014.
- [RFC7234] Fielding, R., Ed., Nottingham, M., Ed., and J. Reschke, Ed., "Hypertext Transfer Protocol (HTTP/1.1): Caching", [RFC 7234](#), June 2014.
- [RFC7235] Fielding, R., Ed. and J. Reschke, Ed., "Hypertext Transfer Protocol (HTTP/1.1): Authentication", [RFC 7235](#), June 2014.

[11.2.](#) Informative References

- [BCP13] Freed, N., Klensin, J., and T. Hansen, "Media Type Specifications and Registration Procedures", [BCP 13](#), [RFC 6838](#), January 2013.
- [BCP178] Saint-Andre, P., Crocker, D., and M. Nottingham, "Deprecating the "X-" Prefix and Similar Constructs in Application Protocols", [BCP 178](#), [RFC 6648](#), June 2012.

- [BCP90] Klyne, G., Nottingham, M., and J. Mogul, "Registration Procedures for Message Header Fields", [BCP 90](#), [RFC 3864](#), September 2004.
- [OWASP] van der Stock, A., Ed., "A Guide to Building Secure Web Applications and Web Services", The Open Web Application Security Project (OWASP) 2.0.1, July 2005, <<https://www.owasp.org/>>.
- [REST] Fielding, R., "Architectural Styles and the Design of Network-based Software Architectures", Doctoral Dissertation, University of California, Irvine, September 2000, <<http://roy.gbiv.com/pubs/dissertation/top.htm>>.
- [RFC1945] Berners-Lee, T., Fielding, R., and H. Nielsen, "Hypertext Transfer Protocol -- HTTP/1.0", [RFC 1945](#), May 1996.
- [RFC2049] Freed, N. and N. Borenstein, "Multipurpose Internet Mail Extensions (MIME) Part Five: Conformance Criteria and Examples", [RFC 2049](#), November 1996.
- [RFC2068] Fielding, R., Gettys, J., Mogul, J., Nielsen, H., and T. Berners-Lee, "Hypertext Transfer Protocol -- HTTP/1.1", [RFC 2068](#), January 1997.
- [RFC2295] Holtman, K. and A. Mutz, "Transparent Content Negotiation in HTTP", [RFC 2295](#), March 1998.
- [RFC2388] Masinter, L., "Returning Values from Forms: multipart/form-data", [RFC 2388](#), August 1998.
- [RFC2557] Palme, F., Hopmann, A., Shelness, N., and E. Stefferud, "MIME Encapsulation of Aggregate Documents, such as HTML (MHTML)", [RFC 2557](#), March 1999.
- [RFC2616] Fielding, R., Gettys, J., Mogul, J., Frystyk, H., Masinter, L., Leach, P., and T. Berners-Lee, "Hypertext Transfer Protocol -- HTTP/1.1", [RFC 2616](#), June 1999.
- [RFC2774] Frystyk, H., Leach, P., and S. Lawrence, "An HTTP Extension Framework", [RFC 2774](#), February 2000.
- [RFC2817] Khare, R. and S. Lawrence, "Upgrading to TLS Within HTTP/1.1", [RFC 2817](#), May 2000.
- [RFC2978] Freed, N. and J. Postel, "IANA Charset Registration Procedures", [BCP 19](#), [RFC 2978](#), October 2000.

- [RFC5226] Narten, T. and H. Alvestrand, "Guidelines for Writing an IANA Considerations Section in RFCs", [BCP 26](#), [RFC 5226](#), May 2008.
- [RFC5246] Dierks, T. and E. Rescorla, "The Transport Layer Security (TLS) Protocol Version 1.2", [RFC 5246](#), August 2008.
- [RFC5322] Resnick, P., "Internet Message Format", [RFC 5322](#), October 2008.
- [RFC5789] Dusseault, L. and J. Snell, "PATCH Method for HTTP", [RFC 5789](#), March 2010.
- [RFC5905] Mills, D., Martin, J., Ed., Burbank, J., and W. Kasch, "Network Time Protocol Version 4: Protocol and Algorithms Specification", [RFC 5905](#), June 2010.
- [RFC5987] Reschke, J., "Character Set and Language Encoding for Hypertext Transfer Protocol (HTTP) Header Field Parameters", [RFC 5987](#), August 2010.
- [RFC5988] Nottingham, M., "Web Linking", [RFC 5988](#), October 2010.
- [RFC6265] Barth, A., "HTTP State Management Mechanism", [RFC 6265](#), April 2011.
- [RFC6266] Reschke, J., "Use of the Content-Disposition Header Field in the Hypertext Transfer Protocol (HTTP)", [RFC 6266](#), June 2011.
- [RFC7238] Reschke, J., "The Hypertext Transfer Protocol (HTTP) Status Code 308 (Permanent Redirect)", [RFC 7238](#), June 2014.

[Appendix A](#). Differences between HTTP and MIME

HTTP/1.1 uses many of the constructs defined for the Internet Message Format [[RFC5322](#)] and the Multipurpose Internet Mail Extensions (MIME) [[RFC2045](#)] to allow a message body to be transmitted in an open variety of representations and with extensible header fields. However, [RFC 2045](#) is focused only on email; applications of HTTP have many characteristics that differ from email; hence, HTTP has features that differ from MIME. These differences were carefully chosen to optimize performance over binary connections, to allow greater liberté dans l'utilisation des nouveaux types de médias, pour faire des comparaisons de dates plus facile, et de reconnaître la pratique de certains premiers serveurs HTTP et clients.

Cette annexe décrit les domaines spécifiques dans lesquels HTTP diffère de MIME.

Les proxys et les passerelles vers et depuis des environnements MIME stricts doivent être conscients de ces différences et fournissent les conversions appropriées si nécessaire.

[A.1](#) . Version MIME

HTTP n'est pas un protocole compatible MIME. Cependant, les messages peuvent contenir

un seul champ d'en-tête MIME-Version pour indiquer quelle version du Le protocole MIME a été utilisé pour construire le message.

Le champ d'en-tête MIME-Version indique que le message est complet conformité avec le protocole MIME (tel que défini dans [[RFC2045](#)]).

Les expéditeurs sont responsables de garantir la pleine conformité (le cas échéant).

(possible) lors de l'exportation de messages HTTP vers des environnements MIME stricts.

[A.2](#) . Conversion en forme canonique

MIME exige qu'une partie du corps d'un courrier Internet soit convertie en forme canonique avant d'être transféré, comme décrit dans [la section 4 de \[RFC2049\]](#) . [La section 3.1.1.3](#) de ce document décrit les formes autorisé pour les sous-types du type de média « texte » lors de la transmission sur

HTTP. [[RFC2046](#)] exige que le contenu de type « texte » représente les sauts de ligne comme CRLF et interdit l'utilisation de CR ou LF en dehors

des séquences de saut de ligne. HTTP autorise CRLF, CR nu et LF nu indiquer un saut de ligne dans le contenu du texte.

Un proxy ou une passerelle de HTTP vers un environnement MIME strict doit traduire tous les sauts de ligne dans les types de supports de texte décrits dans

[Section 3.1.1.3](#) de ce document à la forme canonique [RFC 2049 de](#) CRLF. Notez cependant que cela pourrait être compliqué par la présence d'un

Content-Encoding et par le fait que HTTP permet l'utilisation de certains jeux de caractères qui n'utilisent pas les octets 13 et 10 pour représenter CR et LF, respectivement.

Piste des normes Fielding & Reschke [Page 89]

[RFC 7231](#)

HTTP/1.1 Sémantique et contenu Juin 2014

La conversion brisera toutes les sommes de contrôle cryptographiques appliquées au contenu original sauf si le contenu original est déjà en canonique forme. Par conséquent, la forme canonique est recommandée pour tout contenu qui utilise de telles sommes de contrôle dans HTTP.

[A.3](#) . Conversion des formats de date

HTTP/1.1 utilise un ensemble restreint de formats de date ([Section 7.1.1.1](#)) pour simplifier le processus de comparaison des dates. Les proxys et passerelles de d'autres protocoles doivent garantir que tout champ d'en-tête Date présent dans un message est conforme à l'un des formats HTTP/1.1 et réécrit le date si nécessaire.

[A.4](#) . Conversion du codage de contenu

MIME n'inclut aucun concept équivalent à celui de HTTP/1.1 Champ d'en-tête Content-Encoding. Comme il agit comme un modificateur sur le type de média, proxys et passerelles de HTTP à MIME les protocoles doivent soit modifier la valeur de l'en-tête Content-Type champ ou décoder la représentation avant de transmettre le message. (Quelques applications expérimentales de Content-Type pour le courrier Internet ont utilisé un paramètre de type de média de « ;conversions=<content-coding> » pour exécuter une fonction équivalente à Content-Encoding. Cependant, ceci le paramètre ne fait pas partie des normes MIME).

[A.5](#) . Conversion du codage de transfert de contenu

HTTP n'utilise pas le champ Content-Transfer-Encoding de MIME. Les proxys et les passerelles des protocoles compatibles MIME vers HTTP doivent supprimer tout codage de transfert de contenu avant de livrer la réponse message à un client HTTP.

Les proxys et passerelles allant des protocoles HTTP aux protocoles compatibles MIME sont responsable de s'assurer que le message est au bon format et le codage pour un transport sécurisé sur ce protocole, où « sûr Le « transport » est défini par les limitations du protocole utilisé. Un tel proxy ou passerelle devrait transformer et étiqueter les données avec un un codage de transfert de contenu approprié si cela améliore la probabilité d'un transport sûr via le protocole de destination.

[A.6](#) . MHTML et limitations de longueur de ligne

Implémentations HTTP qui partagent du code avec MHTML [[RFC2557](#)] les implémentations doivent être conscientes des limitations de longueur de

ligne MIME.

Étant donné que HTTP n'a pas cette limitation, HTTP ne se replie pas longtemps
lignes. Les messages MHTML transportés par HTTP suivent tous
conventions du MHTML, y compris les limitations de longueur de ligne et le
pliage,
canonisation, etc., puisque HTTP transfère les corps des messages sous forme

Piste des normes Fielding & Reschke [Page 90]

[RFC 7231](#)

HTTP/1.1 Sémantique et contenu Juin 2014

charge utile et, en dehors du type « multipart/byteranges » (Annexe A de [[RFC7233](#)]), n'interprète pas le contenu ni aucun en-tête MIME lignes qui pourraient y être contenues.

[Annexe B. Modifications par rapport à la RFC 2616](#)

Les principaux changements apportés à cette révision sont de nature rédactionnelle :

extraire la syntaxe de messagerie et partitionner la sémantique HTTP en documents séparés pour les fonctionnalités principales, les demandes conditionnelles,

requêtes partielles, mise en cache et authentification. La conformité le langage a été révisé pour cibler clairement les exigences et les la terminologie a été améliorée pour distinguer la charge utile de représentations et représentations à partir des ressources.

Une nouvelle exigence a été ajoutée selon laquelle la sémantique intégrée dans un URI doit être

désactivé lorsque cette sémantique est incompatible avec la demande méthode, car il s'agit d'une cause fréquente d'échec d'interopérabilité. ([Section 2](#))

Un algorithme a été ajouté pour déterminer si une charge utile est associé à un identifiant spécifique. ([Section 3.1.4.1](#))

Le jeu de caractères par défaut ISO-8859-1 pour les types de supports texte a été

supprimé ; la valeur par défaut est désormais celle indiquée dans la définition du type de média.

De même, le traitement spécial de la norme ISO-8859-1 a été supprimé de la Champ d'en-tête Accept-Charset. ([Section 3.1.1.3](#) et [Section 5.3.3](#))

La définition de Content-Location a été modifiée pour ne plus affecter l'URI de base pour résoudre les références URI relatives, en raison de

un soutien à la mise en œuvre médiocre et l'effet indésirable de rompre les liens relatifs dans les ressources négociées en termes de contenu. ([Section 3.1.4.2](#))

Pour être cohérent avec l'algorithme d'analyse neutre de la méthode [[RFC7230](#)], la définition de GET a été assouplie afin que les requêtes peut avoir un corps, même si un corps n'a aucune signification pour GET. ([Section 4.3.1](#))

Les serveurs ne sont plus tenus de gérer tous les champs d'en-tête Content-* et l'utilisation de Content-Range a été explicitement interdite dans les requêtes PUT. ([Section 4.3.4](#))

La définition de la méthode CONNECT a été déplacée de [[RFC2817](#)] à cette spécification. ([Section 4.3.6](#))

Les méthodes de requête OPTIONS et TRACE ont été définies comme étant

sûr. ([Section 4.3.7](#) et [Section 4.3.8](#))

Piste des normes Fielding & Reschke [Page 91]

[RFC 7231](#)

HTTP/1.1 Sémantique et contenu Juin 2014

Le mécanisme d'extension du champ d'en-tête Expect a été supprimé en raison de implémentations brisées largement déployées. ([Section 5.1.1](#))

Le champ d'en-tête Max-Forwards a été limité aux OPTIONS et Méthodes TRACE ; auparavant, les méthodes d'extension auraient pu l'utiliser comme bien. ([Section 5.1.2](#))

L'URI « about:blank » a été suggérée comme valeur pour le référent champ d'en-tête lorsqu'aucun URI de référence n'est applicable, ce qui distingue ce cas parmi d'autres où le champ Référent n'est pas envoyé ou a été supprimé. ([Section 5.5.2](#))

Les codes d'état suivants peuvent désormais être mis en cache (c'est-à-dire qu'ils peuvent être stocké et réutilisé par un cache sans informations de fraîcheur explicites présent) : 204, 404, 405, 414, 501. ([Section 6](#))

La description du statut 201 (Créé) a été modifiée pour permettre la possibilité que plus d'une ressource ait été créée. ([Section 6.3.2](#))

La définition de 203 (Informations non autorisées) a été élargi pour inclure également les cas de transformations de charge utile. ([Section 6.3.4](#))

L'ensemble des méthodes de requête qui peuvent être redirigées automatiquement en toute sécurité est n'est plus fermé ; les agents utilisateurs sont en mesure de prendre cette décision

basé sur la sémantique de la méthode de requête. Les codes d'état de redirection

Les articles 301, 302 et 307 n'ont plus d'exigences normatives en matière de réponse

charges utiles et interaction utilisateur. ([Section 6.4](#))

Les codes d'état 301 et 302 ont été modifiés pour permettre aux agents utilisateurs

pour réécrire la méthode de POST à GET. (Sections [6.4.2](#) et [6.4.3](#))

La description du code d'état 303 (Voir autre) a été modifiée pour permettre sa mise en cache si des informations de fraîcheur explicites sont fournies,

et une définition spécifique a été ajoutée pour une réponse 303 à GET.

([Section 6.4.4](#))

Le code d'état 305 (Utiliser un proxy) est obsolète pour des raisons de sécurité

préoccupations concernant la configuration en bande d'un proxy. ([Section 6.4.5](#))

Le code d'état 400 (mauvaise demande) a été assoupli afin qu'il ne soit pas limité aux erreurs de syntaxe. ([Section 6.5.1](#))

Le code d'état 426 (mise à niveau requise) a été intégré à partir de [[RFC2817](#)]. ([Section 6.5.15](#))

Piste des normes Fielding & Reschke [Page 92]

[RFC 7231](#)

HTTP/1.1 Sémantique et contenu Juin 2014

La cible des exigences sur HTTP-date et le champ d'en-tête Date ont été réduits à ces systèmes générant la date, plutôt que tous les systèmes envoyant une date. ([Section 7.1.1](#))

La syntaxe du champ d'en-tête Emplacement a été modifiée pour autoriser tous les Références URI, y compris les références relatives et les fragments, ainsi que avec quelques précisions quant aux cas où l'utilisation de fragments ne serait pas approprié. ([Section 7.1.2](#))

Autoriser a été reclassé comme champ d'en-tête de réponse, supprimant le possibilité de le spécifier dans une requête PUT. Exigences relatives à Le contenu de Allow a été assoupli ; en conséquence, les clients ne sont pas il est nécessaire de toujours faire confiance à sa valeur. ([Section 7.4.1](#))

Un registre de méthodes a été défini. ([Section 8.1](#))

Le registre des codes d'état a été redéfini par cette spécification ; auparavant, il était défini dans [la section 7.1 de \[RFC2817\]](#) . ([Section 8.2](#))

L'enregistrement des codages de contenu a été modifié pour exiger l'IETF Révision. ([Section 8.4](#))

Le champ d'en-tête Content-Disposition a été supprimé car il est désormais défini par [[RFC6266](#)].

Le champ d'en-tête Content-MD5 a été supprimé car il était mise en œuvre de manière incohérente en ce qui concerne les réponses partielles.

[Annexe C. ABNF importé](#)

Les règles fondamentales suivantes sont incluses par référence, telles que définies dans

[Annexe B.1 de \[RFC5234\]](#) : ALPHA (lettres), CR (retour chariot), CRLF (CR LF), CTL (contrôles), DIGIT (décimal 0-9), DQUOTE (double citation), HEXDIG (hexadécimal 0-9/AF/af), HTAB (onglet horizontal), LF (saut de ligne), OCTET (toute séquence de données de 8 bits), SP (espace) et VCHAR (tout caractère US-ASCII visible).

Les règles ci-dessous sont définies dans [[RFC7230](#)] :

BWS = <BWS, voir [\[RFC7230\], Section 3.2.3](#) >
OWS = <OWS, voir [\[RFC7230\], Section 3.2.3](#) >
RWS = <RWS, voir [\[RFC7230\], Section 3.2.3](#) >
Référence URI = <Référence URI, voir [\[RFC7230\], Section 2.7](#) >
absolute-URI = <absolute-URI, voir [\[RFC7230\], Section 2.7](#) >
commentaire = <commentaire, voir [\[RFC7230\], Section 3.2.6](#) >
field-name = <commentaire, voir [\[RFC7230\], Section 3.2](#) >
partial-URI = <partial-URI, voir [\[RFC7230\], Section 2.7](#) >

Piste des normes Fielding & Reschke [Page 93]

[RFC 7231](#)

HTTP/1.1 Sémantique et contenu Juin 2014

chaîne entre guillemets = <chaîne entre guillemets, voir [\[RFC7230\], Section 3.2.6](#) >

jeton = <jeton, voir [\[RFC7230\], Section 3.2.6](#) >

[Annexe D. ABNF collecté](#)

Dans l'ABNF collecté ci-dessous, les règles de liste sont développées conformément à [la section 1.2 de \[RFC7230\]](#) .

```
Accepter = [ ( "," / ( plage-média [ paramètres-acceptables ] ) ) *( OWS ","
[
    OWS ( plage de médias [ accept-params ] ) ] ) ]
Accept-Charset = *( "," OWS ) ( ( charset / "*" ) [ poids ] ) *( OWS
    "," [ OWS ( ( jeu de caractères / "*" ) [ poids ] ) ] )
Accept-Encoding = [ ( "," / ( codages [ poids ] ) ) *( OWS "," [ OWS
    ( codages [ poids ] ) ] ) ]
Accept-Language = *( "," OWS ) ( plage de langues [ poids ] ) *( OWS
    "," [ OWS ( plage de langues [ poids ] ) ] )
Autoriser = [ ( "," / méthode ) *( OWS "," [ méthode OWS ] ) ]
```

BWS = <BWS, voir [\[RFC7230\], Section 3.2.3](#) >

```
Codage de contenu = *( "," OWS ) codage de contenu *( OWS "," [ OWS
    codage de contenu ] )
Contenu-Langue = *( "," OWS ) balise-langue *( OWS "," [ OWS
    balise de langue ] )
Emplacement du contenu = URI absolu / URI partiel
Type de contenu = type de média
```

Date = date HTTP

Attendre = "100-continuer"

De = boîte aux lettres

GMT = %x47.4D.54 ; GMT

Date HTTP = date de fixation du FMI / date d'obs

IMF-fixdate = nom-du-jour "," SP date1 SP heure-du-jour SP GMT

Emplacement = référence URI

Max-Forwards = 1*DIGIT

OWS = <OWS, voir [\[RFC7230\], Section 3.2.3](#) >

RWS = <RWS, voir [\[RFC7230\], Section 3.2.3](#) >

Référent = URI-absolu / URI-partiel

Retry-After = HTTP-date / delay-secondes

Piste des normes Fielding & Reschke [Page 94]

[RFC 7231](#)

HTTP/1.1 Sémantique et contenu Juin 2014

Serveur = produit *(RWS (produit / commentaire))

Référence URI = <Référence URI, voir [\[RFC7230\]](#), [Section 2.7](#) >

User-Agent = produit *(RWS (produit / commentaire))

Vary = "*" / (*("," OWS) nom-champ *(OWS "," [OWS nom-champ]))

absolute-URI = <absolute-URI, voir [\[RFC7230\]](#), [Section 2.7](#) >

accept-ext = OWS ";" jeton OWS ["=" (jeton / chaîne entre guillemets)]

accept-params = poids *accept-ext

asctime-date = nom-du-jour SP date3 SP heure-du-jour SP année

jeu de caractères = jeton

codages = codage de contenu / "identité" / "*"

commentaire = <commentaire, voir [\[RFC7230\]](#), [Section 3.2.6](#) >

codage de contenu = jeton

date1 = jour SP mois SP année

date2 = jour "-" mois "-" 2CHIFFRES

date3 = mois SP (2 CHIFFRES / (CHIFFRE SP))

jour = 2 CHIFFRES

nom-jour = %x4D.6F.6E ; Lun

/ %x54.75.65 ; mar.

/ %x57.65.64 ; Mer

/ %x54.68.75 ; Jeu

/ %x46.72.69 ; Ven

/ %x53.61.74 ; Sam

/ %x53.75.6E ; Soleil

nom-jour-l = %x4D.6F.6E.64.61.79 ; lundi

/ %x54.75.65.73.64.61.79 ; Mardi

/ %x57.65.64.6E.65.73.64.61.79 ; Mercredi

/ %x54.68.75.72.73.64.61.79 ; Jeudi

/ %x46.72.69.64.61.79 ; vendredi

/ %x53.61.74.75.72.64.61.79 ; samedi

/ %x53.75.6E.64.61.79 ; dimanche

délai-secondes = 1*CHIFFRE

field-name = <commentaire, voir [\[RFC7230\]](#), [Section 3.2](#) >

heure = 2 CHIFFRES

language-range = <language-range, voir [\[RFC4647\]](#), [Section 2.1](#) >

language-tag = <Language-Tag, voir [\[RFC5646\]](#), [Section 2.1](#) >

boîte aux lettres = <boîte aux lettres, voir [\[RFC5322\]](#), [Section 3.4](#) >

plage-média = ("*" / (type "/") / (type "/" sous-type)) *(OWS ";" paramètre OWS)

[RFC 7231](#)

HTTP/1.1 Sémantique et contenu Juin 2014

```

type-média = type "/" sous-type *( OWS ";" paramètre OWS )
méthode = jeton
minute = 2 CHIFFRES
mois = %x4A.61.6E ; janv.
      / %x46.65.62 ; Fév
      / %x4D.61.72 ; Mars
      / %x41.70.72 ; avr.
      / %x4D.61.79 ; mai
      / %x4A.75.6E ; juin
      / %x4A.75.6C ; juil.
      / %x41.75.67 ; Août
      / %x53.65.70 ; sept.
      / %x4F.63.74 ; Oct
      / %x4E.6F.76 ; novembre
      / %x44.65.63 ; Déc

```

```
obs-date = rfc850 -date / asctime-date
```

```
paramètre = jeton "=" ( jeton / chaîne entre guillemets )
```

```
partial-URI = <partial-URI, voir \[RFC7230\], Section 2.7 >
```

```
produit = jeton [ "/" version-du-produit ]
```

```
version-du-produit = jeton
```

```
chaîne entre guillemets = <chaîne entre guillemets, voir \[RFC7230\], Section
```

```
3.2.6 >
```

```
qvalue = ( "0" [ "." *3DIGIT ] ) / ( "1" [ "." *3"0" ] )
```

```
rfc850 -date = nom-jour-1 "," SP date2 SP heure-du-jour SP GMT
```

```
seconde = 2 CHIFFRES
```

```
sous-type = jeton
```

```
heure du jour = heure ":" minute ":" seconde
```

```
jeton = <jeton, voir \[RFC7230\], Section 3.2.6 >
```

```
type = jeton
```

```
poids = OWS ";" OWS "q=" qvalue
```

```
année = 4 CHIFFRES
```

[RFC 7231](#)

HTTP/1.1 Sémantique et contenu Juin 2014

Indice

- 1
 - 1xx Informatif (classe de code d'état) 50
- 2
 - 2xx Réussi (classe de code d'état) 51
- 3
 - Redirection 3xx (classe de code d'état) 54
- 4
 - Erreur client 4xx (classe de code d'état) 58
- 5
 - Erreur du serveur 5xx (classe de code d'état) 62
- 1
 - 100 Continuer (code d'état) 50
 - 100-continuer (valeur attendue) 34
 - 101 Protocoles de commutation (code d'état) 50
- 2
 - 200 OK (code d'état) 51
 - 201 Créé (code d'état) 52
 - 202 Accepté (code d'état) 52
 - 203 Informations non autorisées (code d'état) 52
 - 204 Aucun contenu (code d'état) 53
 - 205 Réinitialiser le contenu (code d'état) 53
- 3
 - 300 choix multiples (code d'état) 55
 - 301 Déplacé définitivement (code d'état) 56
 - 302 Trouvé (code d'état) 56
 - 303 Voir Autre (code d'état) 57
 - 305 Utiliser un proxy (code d'état) 58
 - 306 (non utilisé) (code d'état) 58
 - 307 Redirection temporaire (code d'état) 58
- 4
 - 400 Mauvaise demande (code d'état) 58
 - 402 Paiement requis (code d'état) 59
 - 403 Interdit (code d'état) 59
 - 404 Non trouvé (code d'état) 59
 - 405 Méthode non autorisée (code d'état) 59
 - 406 Non acceptable (code d'état) 59
 - 408 Délai d'expiration de la demande (code d'état) 60
 - 409 Conflit (code d'état) 60

[RFC 7231](#)

HTTP/1.1 Sémantique et contenu Juin 2014

410 Parti (code d'état) 60
411 Longueur requise (code d'état) 61
413 Charge utile trop importante (code d'état) 61
414 URI trop long (code d'état) 61
415 Type de média non pris en charge (code d'état) 62
417 Échec de l'attente (code d'état) 62
426 Mise à niveau requise (code d'état) 62

5

Erreur interne du serveur 500 (code d'état) 63
501 Non implémenté (code d'état) 63
502 Mauvaise passerelle (code d'état) 63
503 Service indisponible (code d'état) 63
504 Délai d'expiration de la passerelle (code d'état) 63
505 Version HTTP non prise en charge (code d'état) 64

UN

Accepter le champ d'en-tête 38
Champ d'en-tête Accept-Charset 40
Champ d'en-tête Accept-Encoding 41
Champ d'en-tête Accept-Language 42
Autoriser le champ d'en-tête 72

C

pouvant être mis en cache 24
compresser (codage de contenu) 11
demande conditionnelle 36
Méthode CONNECT 30
codage de contenu 11
négociation de contenu 6
Champ d'en-tête Content-Encoding 12
Champ d'en-tête Content-Language 13
Champ d'en-tête Content-Location 15
Champ d'en-tête Content-Transfer-Encoding 89
Champ d'en-tête Content-Type 10

D

Champ d'en-tête de date 67
dégonfler (codage de contenu) 11
Méthode DELETE 29

E

Attendez-vous au champ d'en-tête 34

F

À partir du champ d'en-tête 44

[RFC 7231](#)

HTTP/1.1 Sémantique et contenu Juin 2014

G

Méthode GET 24

Grammaire

Accepter 38

Accepter-Charset 40

Accepter-Encodage 41

accepter-ext 38

Accepter-Langue 42

accepter-params 38

Autoriser 72

asctime-date 66

jeu de caractères 9

codages 41

codage de contenu 11

Codage de contenu 12

Contenu-Langue 13

Contenu-Emplacement 15

Type de contenu 10

Date 67

date1 65

jour 65

nom du jour 65

jour-nom-1 65

délai-secondes 69

Attendez-vous à 34

À partir de 44

GMT 65

heure 65

Date HTTP 65

FMI-date fixe 65

gamme de langues 42

balise de langue 13

Emplacement 68

Max-Avants 36

gamme multimédia 38

type de média 8

méthode 21

minute 65

mois 65

date d'observation 66

paramètre 8

produit 46

version du produit 46

qvalue 38

Référent 45

Réessayer après 69

[rfc850](#) -date 66

deuxième 65

[RFC 7231](#)

HTTP/1.1 Sémantique et contenu Juin 2014

- Serveur 73
 - sous-type 8
 - heure du jour 65
 - type 8
 - Agent utilisateur 46
 - Varie 70
 - poids 38
 - année 65
- gzip (codage de contenu) 11
- H
 - Méthode HEAD 25
- je
 - idempotent 23
- L
 - Champ d'en-tête d'emplacement 68
- M
 - Champ d'en-tête Max-Forwards 36
 - Champ d'en-tête MIME-Version 89
- O
 - Méthode OPTIONS 31
- P
 - charge utile 17
 - Méthode POST 25
 - Méthode PUT 26
- R
 - Champ d'en-tête de référence 45
 - représentation 7
 - Champ d'en-tête Retry-After 69
- S
 - coffre-fort 22
 - représentation sélectionnée 7, 71
 - Champ d'en-tête du serveur 73
 - Classes de codes d'état
 - 1xx Informatif 50
 - 2xx Réussi 51
 - Redirection 3xx 54
 - Erreur client 4xx 58
 - Erreur de serveur 5xx 62

[RFC 7231](#)

HTTP/1.1 Sémantique et contenu Juin 2014

T

Méthode TRACE 32

Tu

Champ d'en-tête User-Agent 46

V

Varier le champ d'en-tête 70

X

x-compress (codage de contenu) 11

x-gzip (codage de contenu) 11

Adresses des auteurs

Roy T. Fielding (éditeur)
Adobe Systems Incorporated
345, avenue Park
San José, CA 95110
USA

Courriel : fielding@gbiv.com
URI : <http://roy.gbiv.com/>

Julian F. Reschke (éditeur)
greenbytes GmbH
Hafenweg 16
Münster, NW 48155
Allemagne

Courriel : julian.reschke@greenbytes.de
URI : <http://greenbytes.de/tech/webdav/>